ONTOLOGY MODELING GUIDE

本体建模与应用指南

从概念体系到形式化模型，再到数据与应用

共 43 节 · 覆盖 RDF / OWL 2 / SKOS / SHACL / SPARQL、建模方法、实现架构与应用落地

由站点内容自动生成 · 2026年11月4日

GIT loongWoong/ontoPlantform

GIT 用户 loong

https://github.com/loongWoong/ontoPlantform/tree/main/ontology-guide

目录

1. 总览
2. 本体与语义网历史
3. 本体模型
4. 标准与语言
5. RDF 数据模型
6. OWL 2 本体语言
7. SKOS 词表
8. SHACL 形状约束
9. SPARQL 查询
10. 建模
11. 方法论
12. 能力问题（CQ）
13. 建模流程与规范
14. 质量评估与反模式
15. 本体元素
16. 概念与类
17. 对象属性与关系
18. 数据属性与取值
19. 个体与实例数据
20. 公理
21. 约束与业务规则
22. 模块化与复用
23. 本体设计模式
24. 实现
25. 架构模式
26. 分层与六边形架构
27. 存储选型
28. 推理与物化
29. 版本与演进
30. 实现本体对象

31. 技术栈
32. JVM 技术栈
33. Python 技术栈
34. JavaScript/TypeScript 技术栈
35. 应用
36. 知识图谱构建
37. 语义检索与 RAG
38. 数据治理与主数据
39. 业务规则与校验
40. 语义集成与互操作
41. 运营与生命周期
42. 落地路线图
43. 端到端案例

本体（Ontology）建模是什么?

本体（Ontology）源自哲学中的“本体论”，研究存在及其本质。在信息科学中，本体指对一个领域中概念、关系、约束的**显式、形式化**描述。最常被引用的定义来自 Gruber（1993）：本体是“对概念化的显式规范”（an explicit specification of a conceptualization）；Studer 等（1998）进一步把它表述为“对共享概念化的形式化、显式规范”（a formal, explicit specification of a shared conceptualization）。

通俗地说：本体把一个领域里“有哪些概念、概念之间是什么关系、什么情况一定成立、什么情况不允许发生”讲清楚，并且用机器能理解的方式表达出来，让不同系统、不同团队对同一批数据有一致的理解。

本体建模的核心方法论是什么?

本体建模的核心方法论，可以概括为：**从领域问题出发，建立共享概念体系，把概念体系形式化，再用本体驱动数据与应用。**它和 DDD“领域问题驱动模型设计，模型驱动软件设计”的思路是一致的，只是对象从“业务行为模型”扩展到了“共享语义与数据”。

整个过程大致分三个阶段：

1. **概念化**：与领域专家一起，从业务问题和能力问题（Competency Questions）中提取概念、关系、规则，形成术语表和概念草图。
2. **形式化**：用 RDF、OWL、SKOS、SHACL 等标准表达概念与约束，让语义精确、可推理、可校验。
3. **应用**：用本体组织数据（知识图谱）、驱动查询与推理、做数据校验、支撑语义检索、系统集成与 AI 应用。

概念化决定本体有没有用，形式化决定本体能不能被机器使用，应用决定本体有没有价值。三者缺一不可。具体的流程、元素与设计模式见本体建模，这个学科的发展脉络见本体与语义网历史。

本体有什么优势?

- **消除歧义、统一语义**：同一个词在不同系统里可能含义不同（“客户”是签约主体还是联系人？“订单”是购物车还是已支付单据？）。本体用显式定义和关系把语义固定下来，减少沟通和数据对不上的问题。
- **跨系统互操作**：用共享的词汇表与 IRI 标识概念，不同来源的数据可以对齐、合并、联合查询，而不是靠人工写映射脚本。
- **让数据可推理、可校验、可解释**：本体加上推理可以得到隐含知识（如传递关系、分类归属），加上 SHACL 等约束可以做数据质量与业务规则校验；推理过程可解释。
- **支撑知识图谱、语义检索、RAG、数据治理**：本体是这些应用共同的语义底座，避免每个项目重复造一套“概念约定”。具体场景与判断标准见应用场景。
- **沉淀组织知识，形成长期资产**：领域术语、关系、规则一旦形式化，就可以被复用、被审核、被版本管理。
- **与 LLM 互补**：大模型擅长理解自然语言但不保证事实与一致性；本体提供确定性的语义约束、结构化事实与校验能力，可以用来做检索增强、结果校验与可解释性支撑。

同时要清楚边界：本体不是银弹。它擅长表达“概念与语义”，不擅长表达算法和业务流程；建模与维护有成本；没有应用场景的本体，本质上只是一份文档。

如何判断一个项目是否真的用了本体?

满足以下三点，才算真正把本体用起来了：

1. **有显式的概念体系**：有术语表、受控词表或形式化模型，而不是只存在于少数人的脑袋里。
2. **形式化模型被系统使用**：数据按本体组织，查询、推理、校验或系统集成真的依赖它。
3. **有治理机制**：有版本、评审、变更流程，本体能随业务演进。

如果只是画了一张概念关系图放进 PPT，或者把数据库表名换一套“专业词汇”，那不是本体实践。

本体和知识图谱是什么关系?

知识图谱通常可以理解为：**本体（模式层）+ 实例数据（实体、关系、属性值）+ 可选的推理**。本体描述“可以说什么”，数据描述“实际有什么”。

知识图谱可以只有很轻的模式，甚至接近 schema-less；但一旦要做语义互操作、推理、数据校验和跨系统集成，就需要显式的本体作为骨架。反过来，只有本体没有数据，也只是空架子。

本体必须用 OWL 吗?

不是。本体是“形式化的共享概念体系”，具体用什么语言取决于需求：

- 只需要术语表、分类体系：SKOS 或简单 RDFS 就够。
- 需要描述逻辑推理（分类、一致性检查、隐含关系）：OWL 2。
- 主要需要数据完整性校验与业务规则：SHACL。
- 工程团队使用属性图：LPG 模式 + 应用层约束也可以承载一部分本体职责。
- 面向 Web 的轻量标注：schema.org + JSON-LD。

形式化程度越高，表达与推理能力越强，但建模成本、学习成本和推理成本也越高。够用即可，不要为了“标准”而标准。各种标准的关系见标准与语言。

什么是概念体系？和 DDD 的通用语言是什么关系?

概念体系是一个领域里被认可的概念、术语、定义、分类和关系的集合。DDD 的通用语言（Ubiquitous Language）要求团队用统一的术语交流；本体把通用语言进一步显式化、形式化，让机器也能使用。

可以这样理解：通用语言是本体的人话版本，本体是通用语言的机器版本。两者目标一致，但不是一一对应——通用语言允许口语化、局部简写，本体需要更严格的边界与定义。

什么是本体模型?

本体模型是针对领域问题设计的语义模型：它用类、关系、属性、公理、约束描述领域中的概念及其语义，忽略具体的表结构、编程语言和存储实现。它既是团队共识的载体，也是数据与应用的语义契约。参见本体模型是什么、不是什么。

有哪些错误的本体实践?

- **只建模不落地**：本体做完就放进文档库，没有数据、没有查询、没有应用，无法验证也无法产生价值。

- **把 ER 图/类图当本体**：只描述数据结构，没有语义公理和约束，也没有形式化表达，机器无法推理和校验。
- **迷信形式化，全量 OWL 公理**：为了“严谨”把能写的公理全写上，导致推理慢、难维护、数据稍有不符就整体不一致。
- **一次建完、不做治理**：没有版本、评审和变更管理，本体很快与真实数据脱节。
- **追求大而全**：试图一次覆盖整个企业所有领域，战线过长，最后无人维护。
- **用 LLM 自动生成本体却不做领域评审**：生成结果看起来合理，但概念边界、关系方向、公理强度都可能有错，必须由领域专家审核。
- **没有能力问题与验收标准**：无法判断本体好坏，只能凭感觉争论。

只有复杂项目或大企业才需要本体吗？

判断标准不是项目规模，而是**语义复杂度**。出现以下信号时，值得引入本体：

- 数据来自多个系统，同一个概念有多种叫法和含义，集成成本高；
- 业务规则密集、跨实体、经常变化，需要可维护的规则表达；
- 需要知识推理、语义检索、智能问答、推荐或决策支持；
- 需要对数据做统一治理、口径对齐、血缘追踪；
- 需要为 AI 应用提供可靠、可解释的语义基础。

小项目、简单 CRUD 通常不必一上来就做重本体。可以先从轻量词表、字段级语义约定或 SHACL 校验起步，等需求增长再逐步形式化。

本体与 DDD 是什么关系？可以一起用吗？

两者是互补关系，不冲突：

- DDD 关注业务边界（限界上下文）、行为（聚合、命令、领域事件）与业务不变量；本体关注跨边界共享的概念、数据语义与互操作。
- 用 DDD 的通用语言作为本体术语的重要来源；每个限界上下文可以维护自己的局部本体或词表。
- 聚合的不变量、数据完整性规则可以用 SHACL 等约束实现；领域事件可以用来驱动知识图谱或语义层更新。
- 跨上下文、跨系统的集成，用本体做语义映射和对齐，比点对点写映射更可持续。

不要用本体替代领域模型，也不要用领域模型替代本体。两者解决的是不同层次的问题。DDD 部分请阅读领域驱动设计指南。

谁来建本体？需要哪些角色？

- **本体工程师/知识工程师**：负责建模、形式化、工具链与质量把关。
- **领域专家**：提供概念、关系、规则与验收标准，必须深度参与。
- **数据工程师/数据管家**：负责数据接入、映射、质量与日常运营。
- **应用方**：提出场景与验收标准，确保本体服务于真实需求。
- **管理者**：提供治理机制与长期投入的保障。

单靠程序员闭门造车，或者把建模整体外包给“专家”而业务方不参与，都很难成功。

投入大吗？应该怎么开始？

本体是长期资产，但起步可以很小：

1. 选一个有明确痛点、范围可控的场景（例如商品主数据统一、设备知识库、合规规则校验）。
2. 写 3–10 个能力问题（CQ），明确“本体要能回答什么”。
3. 先做轻量词表或小本体，优先复用 schema.org 等成熟标准。
4. 用真实数据跑通一个闭环：查询、校验或检索任选其一。
5. 通过 CQ 验收后再扩展，逐步形式化、模块化。

详见能力问题、落地路线图与端到端案例。落进系统时的架构取舍见实现总览。

本体与语义网的发展历史

本体和语义网不是一夜之间出现的。从早期的知识表示、语义网络，到 W3C 的标准栈，再到知识图谱与 LLM 时代的语义层，这条脉络里积累的经验与教训，正是今天做本体建模的基础。以下按时间顺序列出与本体建模和应用相关的主要技术、标准与事件，以及它们对本指南的价值。

---

1. 语义网络（Semantic Networks）

- **时间**：1960 年代
- **贡献者**：Ross Quillian（1968 年提出语义网络模型）等
- **对本指南的价值**：用“概念节点 + 关系边”表示知识，是本体与知识图谱的思想源头；同时暴露了没有形式语义时推理不可靠的问题，后来才催生了描述逻辑与 OWL。

---

2. 框架与知识表示（Frames）

- **时间**：1970 年代
- **贡献者**：Marvin Minsky（1974 年提出框架理论）等
- **对本指南的价值**：用带槽位的框架描述概念、属性和默认值，影响了后来“类—属性—继承”的建模方式；本体中的类、数据属性、默认值等概念都能看到它的影子。

---

3. 专家系统与知识工程

- **时间**：1970–1980 年代
- **贡献者**：Edward Feigenbaum 等
- **对本指南的价值**：把领域知识显式化、可维护化，形成了知识获取、知识表示、知识验证的工程方法，是现代本体工程方法论的先声。

---

4. Cyc 常识知识库

- **时间**：1984 年启动
- **贡献者**：Doug Lenat 与 MCC（Microelectronics and Computer Technology Corporation）
- **对本指南的价值**：大规模常识本体工程的最早实践之一，证明了本体工程的长期性与难度，也让“复用已有本体、不要从零造”成为共识。

---

5. WordNet

- **时间**：1985 年
- **贡献者**：George Miller，普林斯顿大学

- **对本指南的价值**：按词义组织名词、动词等，形成大规模词汇网络，是词汇本体与词表设计的经典参考，也常被用于实体链接与语义相似度计算。

---

6. TOVE 与企业本体

- **时间**：1990 年代
- **贡献者**：Mark Fox 等，多伦多大学
- **对本指南的价值**：面向企业建模的本体工程实践；Gruninger 与 Fox（1995）提出用“能力问题”（Competency Questions）驱动本体设计与验收，这正是本指南推荐的核心方法之一。

---

7. RDF 与 RDFS

- **时间**：1998–1999 年（RDF 1.0 成为 W3C 推荐标准）；2014 年发布 RDF 1.1
- **贡献者**：W3C
- **对本指南的价值**：提供“主语—谓语—宾语”三元组数据模型和全局 IRI 标识，是语义网与本体数据的底座；今天所有主流语义技术都建立在它之上。

---

8. OIL / DAML+OIL

- **时间**：1999–2001 年
- **贡献者**：欧洲 OIL 项目、美国 DARPA DAML 项目
- **对本指南的价值**：把描述逻辑与 Web 标准结合起来，是 OWL 的直接前身；它确立了“本体语言要有形式语义、要可推理”的方向。

---

9. 语义网愿景

- **时间**：2001 年
- **贡献者**：Tim Berners-Lee、James Hendler、Ora Lassila（《The Semantic Web》，Scientific American）
- **对本指南的价值**：提出让 Web 数据“机器可理解”的愿景与分层标准栈，推动了 RDF、OWL、SPARQL 等标准体系的形成。

---

10. OWL

- **时间**：2004 年 OWL 成为 W3C 推荐标准；2009 年发布 OWL 2
- **贡献者**：W3C Web Ontology 工作组
- **对本指南的价值**：基于描述逻辑的本体语言，支持类表达式、属性特征、一致性检查与推理；OWL 2 增加了 EL/QL/RL profile，让推理能力与工程成本可以按需权衡。

---

11. 关联数据（Linked Data）

- **时间**：2006 年
- **贡献者**：Tim Berners-Lee
- **对本指南的价值**：提出用 URI 发布和链接数据的四条原则，让本体与数据可以在 Web 上互联；今天做跨系统语义集成，仍然遵循“用全局标识、可链接、可解引用”的思路。

---

12. SKOS

- **时间**：2009 年（W3C 推荐标准）
- **贡献者**：W3C
- **对本指南的价值**：为受控词表、分类法、叙词表提供轻量标准，降低了“先做重本体”的门槛，让术语治理可以先从 SKOS 起步。

---

13. schema.org

- **时间**：2011 年
- **贡献者**：Google、Bing、Yahoo!、Yandex
- **对本指南的价值**：大规模轻量词汇表，让 Web 内容以 JSON-LD 等方式结构化标注，是“轻量本体、广泛复用”的典型代表。

---

14. 知识图谱

- **时间**：2007 年 DBpedia；2012 年 Google Knowledge Graph；2012 年 Wikidata
- **贡献者**：莱比锡大学、柏林自由大学等；Google；Wikimedia
- **对本指南的价值**：把本体与语义数据推向工程化和产品化，“知识图谱”成为跨领域的热门实践，也带来了从本体到数据流水线的工程方法。

---

15. RDF 1.1

- **时间**：2014 年
- **贡献者**：W3C
- **对本指南的价值**：完善了三元组、命名图、数据类型等规范，巩固了 RDF 生态的标准化基础，也让不同工具之间的互操作更有保障。

---

16. SHACL

- **时间**：2017 年（W3C 推荐标准）
- **贡献者**：W3C Data Shapes 工作组
- **对本指南的价值**：补齐了 RDF 数据的完整性校验能力，让“本体 + 约束”可以真正落到数据质量与业务规则；它和 OWL 的分工（推理 vs 校验）也是今天的实践重点。

17. JSON-LD 1.1

- **时间**：2020 年（W3C 推荐标准）
- **贡献者**：W3C JSON-LD 工作组
- **对本指南的价值**：让 RDF 以 JSON 形式融入 Web 与前端生态，降低了语义数据的接入门槛，是前后端共享语义的常用序列化方式。

---

18. 知识图谱工程化与 LLM 时代

- **时间**：2010 年代中后期至今
- **贡献者**：学术界与工业界（例如微软研究院等）
- **对本指南的价值**：知识图谱成为数据治理、搜索推荐、风控与 AI 的基础设施；大模型兴起后，本体与知识图谱被用于检索增强生成（RAG）、GraphRAG、语义约束与结果校验。相关实践仍在快速演进，需要谨慎判断、小步验证。

从历史看，具体的技术和标准一直在演进，但“显式、共享、形式化”这个核心没有变。今天做本体不必从零发明：优先复用成熟标准与词表，把精力放在领域概念、能力问题与应用闭环上。沿着这条线索往下读，可以看本体建模总览了解怎么建，也可以对照 DDD 的历史看两条思想脉络如何互补。

本体模型是数据库表的结构吗？是 ER 图、类图吗？

不是。数据库表、ER 图、类图描述的是“数据怎么存、结构怎么摆”，本体模型描述的是“概念是什么、概念之间有什么语义关系、什么必然成立”。ER 图和类图通常没有统一的形式逻辑语义，一般按封闭世界理解；本体（尤其是 OWL）有明确的形式语义（开放世界假设、无唯一名称假设），可以被推理机用于分类、一致性检查和隐含知识推导。两者可以互相映射，但不能画等号。

本体就是分类法（Taxonomy）吗？

不是。分类法只是概念的 is-a 层次，是本体的一部分。完整的本体还包含对象属性、数据属性、公理、约束，乃至个体数据。如果只需要维护一棵“类目树”，通常用 SKOS 或简单 RDFS 就够了，不必上升到 OWL。

本体就是知识图谱吗？

不完全等同。知识图谱通常可以理解为：本体（模式/术语层）+ 实例数据（实体、关系、属性值）+ 可选的推理。本体回答“可以说什么”，知识图谱还回答“实际有什么”。反过来，没有显式本体的知识图谱也可以存在（轻量 schema 或 schema-less），只是语义互操作与推理能力会弱一些。

本体必须用 OWL 吗？

不是。本体是“形式化的共享概念体系”，RDFS、SKOS、SHACL，甚至属性图的 schema 与应用层约束，都可以承担本体的部分职责。选择取决于需要多少推理与互操作能力：

- 只要术语与分类：SKOS 或 RDFS。
- 需要一致性检查、类推理、隐含关系推导：OWL 2。
- 主要需要数据完整性校验与业务规则：SHACL。
- 团队使用属性图：LPG 模式 + 应用层约束。
- 面向 Web 的轻量标注：schema.org + JSON-LD。

形式化程度越高，表达与推理能力越强，但建模成本、学习成本和推理成本也越高。各标准的分工与选型见标准与语言总览。

本体模型与 DDD 的领域模型是什么关系？

两者都从领域问题出发，但侧重点不同：

- 领域模型关注如何解决业务问题，强调行为、聚合、不变量与限界上下文。
- 本体模型关注共享概念与数据语义，强调显式定义、互操作、推理与校验。

它们可以共存：用领域模型实现业务，用本体统一语义和集成数据。参见总览与DDD 指南。

本体模型要描述到什么程度?

够用即可：能支撑能力问题、能指导数据与应用即可。不是越形式化越好，也不是越细越好。过度建模会带来维护负担、评审困难和推理成本，反而拖慢落地。

标准与语言总览

本体不是一张画在 PPT 里的概念图。它要被工具读取、被推理器检查、被程序查询、被校验器验证，就必须落到一组公共的语法与语义标准上。W3C 的语义网标准栈就是为此存在的：不同团队、不同产品只要遵守同一套标准，本体的交换和复用才不需要每次重写。

本页给出标准全景、它们之间的组合方式，以及按场景选型的思路。各标准的细节分别见 RDF、OWL 2、SKOS、SHACL、SPARQL。

为什么本体需要标准

没有标准时，团队通常用三种方式表达概念体系：数据库表与注释、私有 JSON Schema、图数据库里约定俗成的节点结构。它们都能在一个系统内工作，但一旦要跨系统交换，就会出现三个问题：

- **语法不统一**：A 团队用 `parent_id` 表示上下级，B 团队用 `belongsTo`，映射代码越写越多。
- **语义不共享**：字段名叫 `status`，但一方指订单状态，另一方指客户状态，机器无法判断。
- **工具不通用**：换一个图数据库或校验工具，模型和规则就得重写一遍。

标准解决的是这三个层面的问题：RDF 统一数据表示，RDFS/OWL 统一语义表达，SHACL 统一校验方式，SPARQL 统一查询语言。代价是学习曲线，收益是长期可交换、可推理、可治理。

标准全景

标准	版本年份	解决什么问题	在本指南中的位置
RDF 1.1	2014	数据模型：三元组、IRI、字面量、命名图	RDF 数据模型与序列化
RDFS	随 RDF 1.1 更新	轻量模式：类层次、属性层次、domain/range	RDF 数据模型与序列化
OWL 2	2009（2012 第二版）	形式化本体：类表达式、属性特征、公理、推理	OWL 2 本体语言
SKOS	2009	受控词表、分类法、叙词表	SKOS 词表与分类法
SHACL	2017	封闭世界的数据形状校验	SHACL 形状约束
SPARQL 1.1	2013	查询与更新 RDF 数据	SPARQL 查询与更新
JSON-LD 1.1	2020	用 JSON 表达 RDF，便于 Web 与前端使用	RDF 数据模型与序列化

此外还要知道两个正在演进的方向：RDF-star（把语句当作节点来引用）目前是社区规范，尚未成为 W3C 正式推荐标准；RDF 1.2 正在标准化过程中。生产系统不要把未定稿的特性当作基础依赖。

它们如何组合

标准不是一个替代另一个，而是一条流水线：

text

```
概念体系 ──SKOS/OWL──▶ 模式（TBox）
                          |
```



一个典型的最小组合是：用 RDF/Turtle 写模式和样例数据，用 OWL 表达需要推理的公理，用 SHACL 表达必填、范围、格式等业务约束，用 SPARQL 实现查询与验收，用 JSON-LD 与前端或其他系统交换。SKOS 只在需要受控词表时加入，不必每页都用。

turtle

```
@prefix ex:   <https://example.com/ontology/order#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

ex:Order a owl:Class ;
    rdfs:label "订单"@zh .

ex:placedBy a owl:ObjectProperty ;
    rdfs:domain ex:Order ;
    rdfs:range  ex:Customer .

ex:orderAmount a owl:DatatypeProperty ;
    rdfs:domain ex:Order ;
    rdfs:range  xsd:decimal .
```

这段 Turtle 已经用到了 RDF 的语法、RDFS 的词汇、OWL 的声明。注意 `rdfs:domain` 的含义是“推理出类型”，不是“校验报错”，这一点在 OWL 页会重点解释。

怎么选：按问题选标准

不要问“哪个标准更先进”，要问“我现在要解决什么问题”。

你的主要诉求	优先使用	说明
术语统一、分类导航、同义词	SKOS	轻量、工具支持好，不引入推理负担
共享概念模型、需要推理或一致性检查	OWL 2	从 EL/RL 等 profile 起步，避免全量 DL 公理
数据完整性、业务规则校验	SHACL	封闭世界，天然适合“必填、范围、格式”
查询、报表、验收测试	SPARQL 1.1	与 OWL 解耦，查询不依赖推理器也能跑
Web API、前端消费	JSON-LD 1.1	用 `@context` 把 JSON 映射为 RDF
只是内部图结构、不需要语义互操作	属性图（LPG）也可	但要清楚它不解决语义共享问题

一个务实的判断顺序是：先确认是否真的需要跨系统语义互操作；需要则从 RDF + SPARQL + SHACL 起步，再按需要加 OWL；如果只是词表管理，SKOS 就够了。

与属性图的关系

行业里常把“图数据库”和“本体”混为一谈。RDF 与属性图（Labeled Property Graph，LPG）是两种图范式：RDF 以全局 IRI 标识资源、边不带属性、强调标准与推理；属性图允许边上带属性、工程上手快、生态偏应用。选型取决于你是否需要语义互操作、标准查询与推理，而不是取决于“图”这个字。详细对比见 RDF 页。

常见误区

- **标准越重越好**：一上来就全量 OWL DL + 复杂公理，推理慢、维护贵，业务方还看不懂。先轻后重。
- **用 OWL 做校验**：OWL 是开放世界，缺数据不等于错误；必填、范围、格式应交给 SHACL。
- **把 SKOS 当简化版 OWL**：SKOS 的 `broader` 不是 `subClassOf`，不能当类层次推理用。
- **只学语法不学语义**：会写 Turtle 不等于会建模；语法只是载体，取舍在语义层。
- **以为 JSON-LD 是另一套体系**：它只是 RDF 的一种序列化，`@context` 决定映射，不是新模型。

小结

标准栈的价值在于分工：RDF 管表示，OWL/SKOS 管语义，SHACL 管校验，SPARQL 管访问，JSON-LD 管交换。多数项目不需要一次用全，先从最小组合开始，随着问题复杂度增加再引入更重的标准，这个顺序比反过来容易成功得多。

RDF 数据模型与序列化

RDF（Resource Description Framework）是整套语义网标准的地基。它只有一句话：任何知识都可以拆成“主语—谓语—宾语”的三元组。理解这句话背后的 IRI、字面量、空白节点和命名图，就理解了为什么本体可以跨系统合并、推理和查询。

三元组：RDF 的全部

一个三元组（triple）描述一个事实：

text

```
<https://example.com/order/1001>  <https://example.com/ontology/order#placedBy>
<https://example.com/customer/C-42> .
```

- **主语**必须是 IRI 或空白节点，不能是字面量；
- **谓语**必须是 IRI；
- **宾语**可以是 IRI、空白节点或字面量。

RDF 没有“表”“行”“字段”的概念，只有一堆三元组。把它们放在一起就是一个图（graph）：主语和宾语是节点，谓语是边。这也是为什么 RDF 数据天然可以合并——两个图取并集即可，不需要对齐 schema。

IRI、字面量与空白节点

IRI（Internationalized Resource Identifier）是全局标识。RDF 用完整 IRI 保证不同来源的数据不会撞名。日常写 Turtle 用前缀缩写，但展开后必须是绝对 IRI。注意 IRI 区分大小写，`ex:Order` 和 `ex:order` 是两个东西。

字面量（literal）是值，分三种：

turtle

```
@prefix ex:    <https://example.com/ontology/order#> .
@prefix order: <https://example.com/order/> .
@prefix xsd:   <http://www.w3.org/2001/XMLSchema#> .

order:1001 ex:orderNo   "SO-1001" .
order:1001 ex:note      "加急"@zh .
order:1001 ex:itemCount "3"^^xsd:integer .
```

在 RDF 1.1 中，`"abc"` 等价于 `"abc"^^xsd:string`。数字、日期、布尔值都应该显式标注 XSD 类型，否则查询里的数值比较会得到意外结果。

空白节点（blank node，写作 `_:b1`）表示“存在某个东西，但我不知道也不关心它的 IRI”。它适合表示没有独立身份的结构性节点，比如一个内嵌地址。但空白节点不能被外部引用，标签只在当前文档内有效，跨文档合并时会重新分配。需要稳定标识的实体，一律用 IRI。

用 Turtle 写数据

Turtle 是手写 RDF 最常用的语法，可读性最好：

turtle

```
@prefix ex:   <https://example.com/ontology/order#> .
@prefix order: <https://example.com/order/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

order:1001
    a               ex:Order ;
    rdfs:label      "订单 1001"@zh ;
    ex:orderNo      "SO-1001" ;
    ex:orderAmount  "1280.00"^^xsd:decimal ;
    ex:placedBy     order:C-42 ;
    ex:contains     [ a ex:OrderLine ; ex:product order:P-7 ; ex:quantity 2 ] .
```

要点：`a` 是 `rdf:type` 的缩写；`;` 续写同一主语，`,` 续写同一谓语的多个宾语；`[...]` 是空白节点属性列表。这些缩写只是语法糖，展开后仍是普通三元组。

其他序列化

同一份数据可以写成多种格式，语义完全等价。

N-Triples：一行一个三元组，无前缀，适合流式处理和 diff：

text

```
<https://example.com/order/1001> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
<https://example.com/ontology/order#Order> .
<https://example.com/order/1001> <https://example.com/ontology/order#orderAmount>
"1280.00"^^<http://www.w3.org/2001/XMLSchema#decimal> .
```

JSON-LD：用 JSON 表达 RDF，通过 `@context` 建立映射，适合 Web API：

json

```
{
  "@context": {
    "ex": "https://example.com/ontology/order#",
    "xsd": "http://www.w3.org/2001/XMLSchema#",
    "orderNo": "ex:orderNo",
    "orderAmount": { "@id": "ex:orderAmount", "@type": "xsd:decimal" }
  },
  "@id": "https://example.com/order/1001",
  "@type": "ex:Order",
  "orderNo": "SO-1001",
  "orderAmount": "1280.00"
}
```

选择依据：人工编辑与代码审查用 Turtle；大数据交换与流处理用 N-Triples/N-Quads；前后端 API 用 JSON-LD；带命名图的场景用 TriG 或 N-Quads。

RDFS：最轻的模式词汇

RDFS 提供了一小组建模词汇，它们不是“注释”，而是带推理语义的公理：

词汇	含义	推理后果
`rdfs:subClassOf`	子类关系	传递；子类实例可被推断为父类实例
`rdfs:subPropertyOf`	子属性关系	子属性断言可推断出父属性断言
`rdfs:domain`	定义域	若 `x p y` 且 `p` 的 domain 是 `C`，则推出 `x a C`
`rdfs:range`	值域	若 `x p y` 且 `p` 的 range 是 `C`，则推出 `y a C`
`rdfs:label` / `rdfs:comment`	人类可读标签与说明	无逻辑后果，供展示和检索

turtle

```
@prefix ex:   <https://example.com/ontology/order#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

ex:PaidOrder rdfs:subClassOf ex:Order .
ex:Order     rdfs:subClassOf ex:BusinessDocument .

# 由传递性可推出：ex:PaidOrder rdfs:subClassOf ex:BusinessDocument
```

最常见的坑是把 `domain` / `range` 当校验规则。它们**只产生类型推断，不会报错**：如果一条订单没有 `placedBy`，推理器不会说“订单缺少客户”；如果 `placedBy` 的对象是商品，推理器会把它推断成客户，而不是判定数据非法。校验要用 SHACL。

RDF 数据集与命名图

单个图是最常见的单位，但 RDF 数据集（dataset）可以包含一个默认图和若干命名图（named graph）。命名图让同一份数据集里可以按来源、版本或用途隔离三元组，SPARQL 里用 `GRAPH` 访问，见 SPARQL 页。典型用法是按数据来源分图（如 `<https://example.com/graph/sap>`、`<https://example.com/graph/crm>`），既保留来源，又能在查询时合并。

RDF 与属性图怎么选

维度	RDF	属性图（LPG）
标识	全局 IRI，天然可合并	通常为内部 ID，跨库需对齐
边属性	不支持，需要具体化为节点	原生支持
模式	RDFS/OWL/SHACL 标准化	多为应用层约定，标准较弱
推理	有成熟标准与推理器	一般依赖应用逻辑或规则引擎
查询	SPARQL（标准）	Cypher/Gremlin 等（各产品不同）
工程体验	学习曲线较陡，工具链成熟	上手快，生态偏应用
适合	跨系统语义互操作、知识图谱、数据治理	路径查询、推荐、图算法密集场景

判断标准不是“哪个技术更先进”，而是“我的问题里，语义共享与标准化占多大比重”。只是做社交关系查询，属性图更省事；要做跨部门主数据统一并保留可推理的语义层，RDF 更合适。也可以组合：RDF 做语义层，属性图或搜索引擎做服务层。

入门命令

用 Apache Jena 的 `riot` 可以完成语法校验与格式转换：

bash

```
# 校验 Turtle 语法
riot --validate data.ttl

# 把 N-Triples 转为 Turtle
riot --output=TURTLE data.nt
```

用 `rapper`（raptor 工具集）也可以互转：

bash

```
rapper -i turtle -o ntriples data.ttl > data.nt
```

常见坑

- **把字符串当实体**：`ex:customer "C-42"` 与 `ex:customer <https://example.com/order/C-42>` 语义完全不同，前者无法参与实体链接。
- **类型标注不一致**：一处写 `"3"`，一处写 `"3"^^xsd:integer`，聚合和比较会出问题。约定好每个数据属性用哪种 XSD 类型。
- **语言标签缺失**：多语言站点要统一用 `@zh`、`@en`，不要一半带标签一半不带。
- **前缀冲突**：合并多个文件时同一前缀指向不同命名空间，展开后才知道撞了。评审时检查前缀表。
- **空白节点充当实体**：需要被查询、被引用、被更新的对象必须有稳定 IRI。

OWL 2 本体语言

OWL 2 是 W3C 于 2009 年发布的正式推荐标准（2012 年出版第二版），用于在 RDF 之上表达比 RDFS 更丰富的本体公理。它的理论根基是描述逻辑（Description Logic），所以既能表达“订单必须由客户下达”这类结构知识，也能让推理器据此推导出新事实或发现矛盾。

但要先记住一句话：**OWL 是用来推理和检查一致性的，不是用来校验业务数据的。**理解这一点，比记住所有构造子更重要。

OWL 2 的定位

OWL 2 有三种“方言”层面的选择：

- **OWL 2 DL**：表达力强，有可判定的推理，主流推理器支持；
- **OWL 2 Full**：在 RDF 上不加限制，表达力最强但没有推理保证，实践中很少作为目标；
- **OWL 2 Profiles（EL / QL / RL）**：牺牲部分表达力换取可计算性与工程性能，是落地时的重点。

OWL 2 常用的序列化是 Turtle（RDF 语法），也可以用 OWL/XML、Manchester 语法。无论哪种语法，模型语义相同。

类表达式：把概念组合起来

类表达式让“类”不再只是一个名字，而是可以由其他类、属性和值定义：

turtle

```
@prefix ex:  <https://example.com/ontology/order#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:PaidOrder a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            ex:Order
            [ a owl:Restriction ;
              owl:onProperty ex:paymentStatus ;
              owl:hasValue ex:Paid ]
        )
    ] .

ex:ExpressOrder a owl:Class ;
    owl:equivalentClass [
        a owl:Restriction ;
        owl:onProperty ex:deliveryOption ;
        owl:someValuesFrom ex:ExpressDelivery
    ] .
```

常用构造子包括：

构造	含义	典型用途
`owl:intersectionOf`	交集（and）	组合特征定义子类
`owl:unionOf`	并集（or）	多个来源合并为一个概念
`owl:complementOf`	补集（not）	互斥分类
`owl:oneOf`	枚举	状态、等级等有限取值
`owl:someValuesFrom`	至少有一个值属于某类	存在性要求
`owl:allValuesFrom`	所有值都属于某类	值域限制（仅在属性出现时生效）
`owl:hasValue`	必须取某个具体值	固定状态或标志
`owl:minCardinality` / `maxCardinality`	基数限制	数量约束
`owl:minQualifiedCardinality` + `owl:onClass`	限定基数	“至少两个不同商品”

属性特征

对象属性的特征决定了推理能力：

特征	推理效果
`owl:inverseOf`	双向关系可互推
`owl:TransitiveProperty`	传递闭包，如“属于上级组织”
`owl:SymmetricProperty`	对称关系，如“与...合作”
`owl:FunctionalProperty`	至多一个值，常表达唯一标识
`owl:InverseFunctionalProperty`	值唯一确定主语
`owl:ReflexiveProperty` / `owl:IrreflexiveProperty`	自反/非自反
`owl:AsymmetricProperty`	非对称
`owl:propertyDisjointWith`	两个属性不能同时成立
`owl:propertyChainAxiom`	属性链，如“父之兄为伯父”
`owl:hasKey`	用一组属性作为键，键相同则个体相同

turtle

```
@prefix ex:   <https://example.com/ontology/order#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

ex:hasEmployeeNo a owl:DatatypeProperty , owl:FunctionalProperty ;
    rdfs:domain ex:Employee ;
    rdfs:range  xsd:string .

ex:Employee owl:hasKey ( ex:hasEmployeeNo ) .
```

```
ex:managerOf a owl:ObjectProperty ;
    owl:inverseOf ex:reportsTo .
```

注意 `owl:hasKey` 与 `owl:FunctionalProperty` 的差别：前者用于“键相同则同一个人”，后者用于“一个主语最多一个值”，二者不能互相替代。

个体同一性：sameAs 与不同个体

OWL 默认**没有唯一名称假设（no UNA）**：两个不同的 IRI 完全可能指同一个体。要表达“这是同一个人”，写 `ex:A owl:sameAs ex:B`；要表达“这是两个人”，必须显式写 `owl:differentFrom` 或 `owl:AllDifferent`。

这带来两个实践后果：

- 不写 `differentFrom` 不代表不同，推理器不会因为你用了两个 IRI 就认为它们是两个人；
- `sameAs` 会合并两个个体的所有断言，误用会污染数据。实体链接得到的 `sameAs` 必须经过审核，且最好保留来源与置信度。

开放世界假设的后果

OWL 采用开放世界假设（OWA）：**没有说的不等于假的。**因此：

- 订单没有 `placedBy`，推理器不会报错，只会认为“尚未知道”；
- 要表达“订单必须由客户下达”，`someValuesFrom` 只能推出“存在一个客户”，推不出“缺失即错误”；
- 若两个类声明为 `owl:disjointWith`，而数据迫使某个个体同时属于两者，推理器会判定本体或数据**不一致**（inconsistent），这是唯一会“报错”的机制。

turtle

```
ex:Customer a owl:Class .
ex:Supplier a owl:Class ;
    owl:disjointWith ex:Customer .
```

如果一条数据同时断言某个主体既属于 `Customer` 又属于 `Supplier`，推理器会发现矛盾。但如果只是“某个属性没有填写”，它什么也不会说——这正是需要 SHACL 的地方。

Profile 与推理复杂度

Profile	表达力取舍	适合场景	代表推理器
OWL 2 EL	支持存在量词、交、类/属性层次、部分属性链，不支持全称、析取、否定	大规模术语分类、SNOMED 类本体	ELK
OWL 2 QL	面向查询重写，可把 SPARQL 改写到关系数据库	OBDA、数据虚拟化	查询重写引擎（如 Ontop）
OWL 2 RL	可用规则引擎/Datalog 实现，适合流式与增量推理	规则密集型系统、生产环境	规则引擎
OWL 2 DL	表达力最强，复杂度高	复杂约束建模、学术与深度推理	HermiT、Pellet

实践建议：能用 EL 表达就用 EL，需要规则就用 RL，只有确实需要复杂否定/析取时才上完整 DL。全量 DL 公理在大数据上很容易推理爆炸。

OWL、RDFS 与 SHACL 的边界

维度	RDFS	OWL 2	SHACL
主要用途	轻量类层次	形式化语义与推理	数据校验
世界假设	开放	开放	封闭
缺数据	不报错	不报错	可报错
典型问题	这是子类吗	这个概念能推出什么、是否矛盾	这条数据合法吗

三者可以共存：RDFS/OWL 定义语义，SHACL 定义完整性要求。不要在 OWL 里硬造校验，也不要用 SHACL 表达推理规则。

过度公理化的坑

- **用 `allValuesFrom` 当必填**：属性不出现时约束自动成立，永远校验不出缺失。
- **用不相交公理做业务规则**：一旦数据有噪声，整个本体被判定不一致，难以定位。
- **把唯一性寄托在 `InverseFunctionalProperty`**：OWL 推的是“相等”，不是“重复数据报错”。
- **属性链、传递属性滥用**：推理闭包膨胀，查询变慢。
- **用 `sameAs` 做 ETL 对齐**：合并后难以撤销，应由受控的实体链接流程负责。

建议：公理只保留真正有推理价值的少数几条，其余约束交给 SHACL；每次变更后跑推理器做一致性检查，并保留解释（explanation）；用样例实例回归测试验证推理结果。进一步的质量方法见 质量评估与反模式。

SKOS 词表与分类法

SKOS（Simple Knowledge Organization System）是 W3C 于 2009 年发布的推荐标准，用来在 RDF 上表达受控词表、分类法、叙词表和术语表。它和 OWL 长得像（都用 RDF 三元组、都定义类与属性），但目标完全不同：**SKOS 关心的是一组概念如何被命名、组织和互相映射，而不是概念之间的逻辑蕴含。**

如果你的问题只是“把术语统一起来、给它们分类、支持多语言与同义词”，用 SKOS 比 OWL 更轻、更合适。

三件容易混淆的东西

- **受控词表（controlled vocabulary）**：一组经过批准的术语，比如订单状态、产品类别、风险等级。
- **分类法（taxonomy）**：带层次结构的词表，每个条目通常只有一个父节点，比如商品类目树。
- **叙词表（thesaurus）**：更正式的词表，含同义词、相关词、上下位关系与范围注释，常用于图书馆与知识组织。

SKOS 对三者都给出一套统一建模方式，区别只在于你用了多少特性。

SKOS 核心词汇

概念体系与概念：`skos:ConceptScheme` 是词表容器，`skos:Concept` 是词表里的一个条目，用 `skos:inScheme` 归属，用 `skos:topConceptOf` / `skos:hasTopConcept` 标记顶层。

标签（都带语言标签）：

属性	用途	约束
`skos:prefLabel`	首选标签	同一语言下每个概念至多一个
`skos:altLabel`	同义词、缩写、别名	可多个，用于检索扩展
`skos:hiddenLabel`	拼写变体、错拼	不展示，只用于匹配

语义关系：

- `skos:broader` / `skos:narrower`：上下位关系，二者互为逆关系，`broader` 具有传递性；
- `skos:related`：相关但不构成层次的关系，是对称的。

其他常用属性：`skos:notation`（编码，如 `"PAID"`）、`skos:definition`（定义）、`skos:scopeNote`（使用范围说明）、`skos:example`、`skos:changeNote`；集合用 `skos:Collection`、有序集合用 `skos:OrderedCollection`。

turtle

```
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix ex:   <https://example.com/vocab/order-status#> .

ex:StatusScheme a skos:ConceptScheme ;
    skos:prefLabel "订单状态词表"@zh ;
    skos:hasTopConcept ex:Status .

ex:Status a skos:Concept ;
    skos:prefLabel "订单状态"@zh ;
```

```
    skos:topConceptOf ex:StatusScheme .

ex:Paid a skos:Concept ;
    skos:inScheme ex:StatusScheme ;
    skos:prefLabel "已支付"@zh ;
    skos:altLabel "付款完成"@zh , "paid"@en ;
    skos:notation "PAID" ;
    skos:broader ex:Status ;
    skos:definition "订单款项已收到，等待发货。"@zh .

ex:Shipped a skos:Concept ;
    skos:inScheme ex:StatusScheme ;
    skos:prefLabel "已发货"@zh ;
    skos:notation "SHIPPED" ;
    skos:broader ex:Status ;
    skos:related ex:Paid .
```

映射关系用于跨词表对齐：`skos:exactMatch`（可互换）、`skos:closeMatch`（近似）、`skos:broadMatch` / `skos:narrowMatch`（宽/窄于对方）、`skos:relatedMatch`。它们是对称或可逆的，但都只是“词表层面”的断言，不能推出实例层面的等价。

SKOS 不是 OWL 的子集替代品

这是最常见的误解。SKOS 本身是用 OWL 定义的词汇表，但它的语义很轻：`skos:Concept` 的实例之间只有 `broader`、`related` 这些关系，**`skos:broader` 不是 `rdfs:subClassOf`**。

差别在于：

- OWL 的 `subClassOf` 作用在**类**上，会影响实例的类型推断；
- SKOS 的 `broader` 作用在**概念个体**上，只表达“这两个词条有上下位关系”；
- 因此不能通过 `ex:Paid skos:broader ex:Status` 推出“任何已支付订单都是订单状态”，也不能把 `skos:Concept` 当业务实体的类型。

SKOS 里唯一的“推理”是少量内建语义：`broader` 传递、`broader` / `narrower` 互逆、`related` 对称、映射关系对称等。它不会替你做类分类，也不会检查一致性。

适用场景与反例

适合用 SKOS 的：

- 企业术语表、数据字典的分类层、指标口径词表；
- 商品类目、行业代码、地区代码等受控取值；
- 多语言标签与同义词检索扩展；
- 跨标准词表的映射（如内部状态码与行业标准码）。

不适合用 SKOS 的：

- 需要表达“订单必须由客户下达”这类结构约束；
- 需要按属性关系做推理（如组织汇报链、BOM 展开）；
- 需要类型层次与实例类型推断；

- 需要数据校验（那是 SHACL 的职责）。

判断方法很简单：如果你主要在管理“词”和“词的归属”，用 SKOS；如果开始描述“实体有哪些属性、彼此什么关系”，就该上 OWL。

与本体配合的三种方式

1. **词表作为本体的标签体系**：业务类用 OWL 定义，属性值用 SKOS 概念表示；对需要多语言、同义词的枚举，用 `skos:Concept` 个体比裸字符串更可控。
2. **用注释属性建立关联**：在概念和类之间加自定义注释属性（例如 `ex:representsClass`），或用 `rdfs:seeAlso` 指向类。不要用 `skos:exactMatch` 直接连“概念”和“类”，那会混淆两个层次。
3. **值划分（value partition）**：把 SKOS 词表当作枚举值的来源，实例通过对象属性指向 `skos:Concept` 个体，而不是用字符串存状态码。这样状态的含义、同义词、多语言标签都能复用。

常见坑

- **把 `broader` 当 `subClassOf`**：模型看起来对，推理结果全错；这是 SKOS 与 OWL 混用时的头号错误。
- **同一语言多个 `prefLabel`**：SKOS 要求每种语言至多一个首选标签，多了会导致展示与检索歧义。
- **用标签当标识**：概念必须有稳定 IRI；靠 `prefLabel` 做匹配，改名就断链。
- **`altLabel` 乱塞**：同义词、错拼、旧名要区分 `altLabel` 与 `hiddenLabel`，否则检索噪音大。
- **映射关系用得太重**：`exactMatch` 表示可互换，跨系统对齐时优先 `closeMatch`，把审核结论留在流程里而不是模型里。
- **缺少 `skos:definition` 与 `scopeNote`**：词表最终要给人用，没有定义和适用范围说明，词表会退化成一张标签表。

小结

SKOS 的定位是“把词组织好”，不是“把世界建模好”。它是本体项目里极好的第一步：成本低、业务方能参与、能快速统一术语；等到需要结构和推理时，再让 OWL 接管语义层，SKOS 继续负责标签与词表治理。两者分工清楚，比强行用一套语言解决所有问题更可持续。

SHACL 形状约束

SHACL（Shapes Constraint Language）是 W3C 于 2017 年发布的推荐标准，用来校验 RDF 数据是否满足一组“形状”（shape）。它是 OWL 的天然搭档：OWL 负责表达语义和推理，SHACL 负责回答“这份数据完整吗、格式对吗、值合法吗”。

记住它的世界假设是**封闭世界（CWA）**：没有出现的属性就是缺失，可以直接判为不符合。这正是业务校验需要的行为。

SHACL 解决什么问题

OWL 的开放世界假设决定了它无法表达“必填”。`owl:someValuesFrom` 在属性缺失时不会报错，只会认为“可能存在但未知”。而业务上“订单必须有客户、金额必须大于零、状态必须在枚举内”都是硬约束。

SHACL 的做法是：给数据图（data graph）配一个形状图（shapes graph），执行校验后返回一份校验报告（validation report），报告以 `sh:conforms true/false` 表示是否全部通过，并列出每个失败的焦点节点、路径、来源形状与消息。

形状与目标

形状分两类：

- **节点形状（NodeShape）**：直接约束一个节点；
- **属性形状（PropertyShape）**：约束某个路径上的值，用 `sh:path` 指定。

形状通过 target 决定“校验谁”：

目标	含义
`sh:targetClass ex:Order`	所有 `rdf:type ex:Order` 的节点
`sh:targetNode <https://example.com/order/1001>`	指定节点
`sh:targetSubjectsOf ex:placedBy`	所有以该属性为主语的节点
`sh:targetObjectsOf ex:placedBy`	所有以该属性为宾语的节点

也可以把形状直接声明为类（`ex:Order a sh:NodeShape, owl:Class`），此时该类的实例自动成为目标，这叫隐式类目标。

常用约束

约束	作用
`sh:datatype`	字面量必须是某 XSD 类型
`sh:class`	值必须是某类的实例（取决于是否启用推理）
`sh:nodeKind`	`sh:IRI` / `sh:Literal` / `sh:BlankNode` 等
`sh:minCount` / `sh:maxCount`	值的数量
`sh:minInclusive` / `sh:maxInclusive`	数值/日期范围
`sh:minLength` / `sh:maxLength`	字符串长度
`sh:pattern`	正则匹配（可配 `sh:flags`）
`sh:in`	值必须在给定列表中
`sh:hasValue`	必须包含指定值
`sh:languageIn`	语言标签必须在允许集合内
`sh:uniqueLang`	同一属性下语言标签不得重复
`sh:equals` / `sh:disjoint` / `sh:lessThan`	两个路径的值相等/不相交/有序
`sh:closed` + `sh:ignoredProperties`	只允许显式声明的属性
`sh:node`	值必须满足另一个形状

示例：订单必填与金额范围

turtle

```
@prefix sh:  <http://www.w3.org/ns/shacl#> .
@prefix ex:  <https://example.com/ontology/order#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:OrderShape a sh:NodeShape ;
    sh:targetClass ex:Order ;
    sh:closed true ;
    sh:ignoredProperties ( <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> ) ;
    sh:property [
        sh:path     ex:placedBy ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
        sh:class    ex:Customer ;
        sh:message  "订单必须且只能有一个下单客户。"@zh
    ] ;
    sh:property [
        sh:path          ex:orderAmount ;
        sh:minCount      1 ;
        sh:datatype      xsd:decimal ;
        sh:minInclusive  0 ;
        sh:message       "订单金额必填且不能为负数。"@zh
    ] ;
```

```
    sh:property [
        sh:path     ex:status ;
        sh:minCount 1 ;
        sh:in       ( ex:Pending ex:Paid ex:Shipped ex:Closed ) ;
        sh:message  "订单状态必须是预定义值之一。"@zh
    ] .
```

几个细节：`sh:closed true` 后，除了 `sh:path` 声明的属性和 `sh:ignoredProperties` 里的属性，其他属性一律判失败；`sh:in` 的值是有序列表；`sh:datatype xsd:decimal` 对 `"1280"^^xsd:integer` 也会失败，因为类型不精确匹配。

严重级别与消息

每个约束可设 `sh:severity`：`sh:Violation`（默认，失败即不通过）、`sh:Warning`、`sh:Info`。`sh:message` 支持多语言标签，用 `{?var}` 可以引用焦点节点或值。生产系统应把消息写成业务方能理解的话，而不是“Constraint violation”。

对同一批数据，可以组合多份形状图：数据质量形状、发布门禁形状、对外 API 形状各管一段，按场景加载。

SPARQL 约束

当内置约束表达不了跨字段规则时，用 `sh:sparql` 写 SPARQL 查询。约定是：查询中用 `$this` 指代当前焦点节点，返回的每一行代表一个违规。

turtle

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix ex: <https://example.com/ontology/order#> .

ex:ShipDateShape a sh:NodeShape ;
    sh:targetClass ex:Order ;
    sh:sparql [
        sh:message "发货时间不能早于下单时间。"@zh ;
        sh:select """
            SELECT $this WHERE {
                $this <https://example.com/ontology/order#orderDate> ?d ;
                      <https://example.com/ontology/order#shippedAt> ?s .
                FILTER (?s < ?d)
            }
        """
    ] .
```

这条规则用内置约束写不出来，因为它比较同一节点上两个属性的值。SPARQL 约束的代价是可移植性与性能：不同引擎的优化能力不同，复杂查询应加上目标类缩小范围。

与 OWL 推理如何配合

SHACL 校验的是“图里实际有的三元组”。如果先推理再校验，形状能看到推断出的类型和关系；如果直接校验原始数据，`sh:class` 可能因为子类未被推断而不通过。两种顺序都合理，但必须在流水线里固定，否则同一份数据在不同环境下结果不一致。

注意不要在推理后校验“缺失”类规则：推理不会凭空生成 `placedBy`，`minCount` 仍会失败，这是正确行为。反过来，也不要指望 SHACL 推断出新事实——它的输出是校验报告，不是新三元组。

运行校验

bash

```
# 使用 pyshacl 校验
pyshacl -s shapes.ttl -d data.ttl -f human

# 使用 Apache Jena 的 shacl 命令
shacl validate --shapes shapes.ttl --data data.ttl
```

`pyshacl` 支持 OWL 推理前置（按需开启）与多种报告格式；Jena 的 `shacl` 命令适合在 JVM 构建流水线中做门禁。

常见坑

- **忘记 target**：形状写了但没有任何节点被选中，校验永远通过。
- `sh:class` **不考虑子类**：未开启推理时，`ex:ExpressOrder` 的实例不会被认为满足 `sh:class ex:Order`。
- `closed` **漏掉** `rdf:type`：几乎所有数据都会因此失败，记得放进 `ignoredProperties`。
- **把 SHACL 当推理解释器**：它不生成数据、不判断语义蕴含，只判断形状满足性。
- `uniqueLang` **数据没语言标签**：没有标签的字面量不算语言重复，但也不满足 `languageIn`。
- **只在最后校验**：入库、发布、API 三个环节都应校验，越早发现成本越低。

SHACL 与 OWL 的分工可以概括为：OWL 回答“这个模型意味着什么”，SHACL 回答“这份数据够不够好”。两者都做，且各司其职，是语义层稳定运行的前提。约束如何与业务规则、聚合不变量配合，见 约束与业务规则 与 业务规则与校验。

SPARQL 查询与更新

SPARQL 1.1 是 W3C 于 2013 年发布的 RDF 查询语言标准，包含查询（Query）与更新（Update）两部分。它相当于 RDF 世界的 SQL：数据存成三元组后，靠 SPARQL 取数、统计、验收和对外提供接口。

与 SQL 最大的不同在于，SPARQL 匹配的是图模式而不是表。你写的是“子图长什么样”，引擎去找所有匹配该子图的绑定组合。

基本图模式

sparql

```
PREFIX ex: <https://example.com/ontology/order#>

SELECT ?order ?customerName
WHERE {
  ?order a ex:Order ;
         ex:placedBy ?customer ;
         ex:orderAmount ?amount .
  ?customer ex:name ?customerName .
}
```

`WHERE` 里的三元组模式用变量连接：`?order` 是订单节点，`a` 是 `rdf:type` 的缩写。这条查询等价于 SQL 的两表连接，但连接条件由共享变量隐式表达。

常用能力

构造	作用
`FILTER`	过滤表达式，如数值比较、正则、`BOUND`、`EXISTS`
`OPTIONAL`	左连接；匹配失败时变量为空
`UNION`	并集
`MINUS`	差集
`BIND (... AS ?x)`	在查询中计算新变量
`VALUES ?x { ... }`	内联数据，常用于参数化
`GROUP BY` / `HAVING` / 聚合	`COUNT`、`SUM`、`AVG`、`MIN`、`MAX`、`GROUP_CONCAT`
子查询	嵌套 `SELECT`，先聚合再连接
属性路径	`ex:partOf+`、`^ex:placedBy`、`ex:a/ex:b`，以及用竖线表示的多条路径并集
`GRAPH` / `FROM NAMED`	访问命名图，见 RDF 数据集
`SERVICE <endpoint>`	联邦查询，跨端点取数
`ORDER BY` / `LIMIT` / `OFFSET`	排序与分页

属性路径是一个高频能力。`ex:partOf+` 表示沿 `partOf` 传递闭包向上找，适合组织架构、BOM 展开、类目祖先这类层次查询；`^ex:placedBy` 表示反向沿边查找。注意无界路径（`*`、`+`）在大图上代价很高，务必配合 `LIMIT` 与起点约束。

从能力问题到验收查询

能力问题（CQ）的落地方式之一，就是把每个 CQ 写成一条 SPARQL 查询。例如 CQ：“列出 2024 年以来下单、尚未发货且总额超过 1000 元的订单，按金额倒序”。

sparql

```
PREFIX ex:  <https://example.com/ontology/order#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT ?order ?customerName (SUM(?amount) AS ?total)
WHERE {
  ?order a ex:Order ;
         ex:placedBy   ?customer ;
         ex:orderAmount ?amount ;
         ex:orderDate   ?date .
  ?customer ex:name ?customerName .
  OPTIONAL { ?order ex:shippedAt ?shipped }
  FILTER (?date >= "2024-01-01"^^xsd:date && !BOUND(?shipped))
}
GROUP BY ?order ?customerName
HAVING (SUM(?amount) > 1000)
ORDER BY DESC(?total)
LIMIT 20
```

这条查询同时用到了图模式、`OPTIONAL`、`FILTER`、聚合、`HAVING` 与排序。它能跑通，就说明模型里有对应的类和属性，且数据填得进去——这比“模型看起来不错”可靠得多。

其他查询形式

- **ASK**：返回布尔值，适合存在性检查；
- **CONSTRUCT**：返回新三元组，适合生成派生图或做格式转换；
- **DESCRIBE**：返回某资源的描述，具体内容由实现决定，不宜依赖其精确语义。

sparql

```
PREFIX ex: <https://example.com/ontology/order#>

CONSTRUCT {
  ?customer ex:hasHighValueOrder ?order .
}
WHERE {
  ?order a ex:Order ;
         ex:placedBy    ?customer ;
         ex:orderAmount ?amount .
  FILTER (?amount > 10000)
}
```

SPARQL Update

Update 用于写入、修改与删除，语法上独立于 Query：

sparql

```
PREFIX ex:  <https://example.com/ontology/order#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

DELETE { ?order ex:status ex:Pending . }
INSERT { ?order ex:status ex:Closed . }
WHERE  {
  ?order a ex:Order ;
         ex:status    ex:Pending ;
         ex:orderDate ?date .
  FILTER (?date < "2024-01-01"^^xsd:date)
}
```

常用操作包括 `INSERT DATA`、`DELETE DATA`、`DELETE WHERE`、`DELETE/INSERT ... WHERE`、`LOAD <file>`、`CLEAR GRAPH`。批量导入优先用 `LOAD` 或批量 `INSERT DATA`，逐条更新在远程端点上很慢。事务语义由具体存储决定，跨多个图的更新不保证原子，写操作前要评估回滚策略。

与 SQL 的对照

维度	SPARQL	SQL
数据模型	图（三元组）	表（行与列）
模式	可选，运行时匹配	强 schema，建表时定义
连接	共享变量	`JOIN ... ON`
可空/缺失	天然允许，`OPTIONAL` 表达	`NULL`
路径查询	属性路径原生支持	需要递归 CTE
演进	加属性不影响已有数据	通常要改表结构
生态	三元组库、推理器	关系数据库占绝对主流

两者不是替代关系。常见架构是业务数据在关系库，语义层用 RDF/SPARQL 做集成与查询，必要时通过联邦或物化同步。

命名图与推理边界

RDF 数据可以组织成数据集：默认图加若干命名图。SPARQL 用 `FROM` 指定默认图、`FROM NAMED` 声明可被访问的命名图，再用 `GRAPH` 在查询里切换；不写 `FROM` 时，查询作用于端点配置的默认数据集。按来源分图（ERP、CRM、人工补录）既保留出处，也能在一个查询里合并。

sparql

```
PREFIX ex: <https://example.com/ontology/order#>

SELECT ?order ?status
```

```
FROM <https://example.com/graph/orders>
FROM NAMED <https://example.com/graph/erp>
WHERE {
  ?order a ex:Order .
  GRAPH <https://example.com/graph/erp> { ?order ex:erpStatus ?status }
}
```

推理不属于 SPARQL 规范：一条查询能否看到推断出的三元组，取决于端点是否启用了 RDFS/OWL 推理、是物化还是查询时推理。同一端点改配置，结果就可能变化。接口契约要写明推理级别；需要稳定结果时，宁可在查询里显式写出所需的连接，也不要把语义押在端点的隐藏推理上。

性能与安全注意

- **无界属性路径**：`ex:partOf*` 在百万级图上有爆炸风险，尽量限定起点类型与深度。
- `SELECT *`：返回所有变量，网络与序列化开销大，生产查询应显式列出变量。
- **无法索引的过滤**：`FILTER(regex(...))`、函数计算通常无法走索引，先缩小图模式再过滤。
- **联邦查询**：`SERVICE` 依赖远端可用性与延迟，且会把数据暴露给对端，需评估信任边界。
- **开放端点风险**：可写的 SPARQL 端点是高危面。生产环境应默认只读、加认证与限流、设查询超时与结果上限；公开端点尤其要防止 `LOAD` 与大批量 `DELETE`。
- **注入风险**：不要把用户输入直接拼接进查询字符串。用 `VALUES` 绑定参数，或在应用层做 IRI 校验与转义。
- **推理与查询分离**：查询是否看到推断出的三元组，取决于端点是否开启推理。接口契约里要写清楚。

小结

SPARQL 是本体从“模型文件”变成“可用数据服务”的关键一环。写模型时就该同步写查询：查询写不出来，往往说明模型缺少必要的属性或方向；查询性能差，往往说明图结构或索引设计有问题。把 CQ 查询纳入自动化测试，是最划算的本体质量保障手段之一。

建模总览

本体建模不是“画一张概念图”，而是把团队共享的概念体系从文档、表格和人脑里提取出来，写成语义明确、机器可处理、能被多个系统复用的形式化模型。判断一次建模是否真的成立，只看三条：概念被显式定义、定义能被机器解释、模型确实驱动了数据或应用。

这和 DDD 的建模目标一致——用模型解决业务问题——但本体建模多了两个约束：模型必须用标准语言表达，且要能跨系统共享。

建模的两个产物：模式与数据

本体建模同时产出两类东西，理解这个划分是后续所有工作的前提：

- **模式（TBox）**：类、对象属性、数据属性、公理。回答“这个领域里有哪些概念、它们如何关联、有什么逻辑规律”。例如“订单至少包含一个订单行”“已取消订单不能发货”。
- **数据（ABox）**：个体与断言。回答“具体有哪些对象、它们之间是什么关系”。例如“订单 ORD-20250301 由客户 C-1001 在 2025-03-01 下单，含两个订单行”。

TBox 与 ABox 的边界不是绝对的。一个“订单状态”集合，既可以建成类层次，也可以建成一组个体，取决于你打算用它做分类推理还是做取值校验。做这个选择时，先问“谁会用它、怎么用”，而不是先问“哪种更标准”。

推荐流程：能力问题先行

本指南推荐的轻量流程如下，每一步都有可验收的产物，不追求一次到位：

步骤	产物	完成标志
1. 定范围	一页范围说明	明确“覆盖什么、不覆盖什么”
2. 写能力问题（CQ）	5–10 条可判定问题	每条都能翻译成查询或校验
3. 定复用策略	复用清单	决定哪些用标准词表、哪些自建
4. 概念草图	术语表 + 类与关系清单	领域专家能读懂并纠正
5. 形式化	Turtle/OWL 文件	能加载、能通过一致性检查
6. 样例实例	小规模 ABox	CQ 能跑出预期结果
7. 验证与评审	CQ 通过率、问题清单	关键 CQ 全部通过
8. 发布与迭代	版本化本体 + 变更记录	有维护人、有变更流程

关键在于第 2 步和第 6 步：没有 CQ，建模就没有验收标准，最后只能靠“看起来对不对”来评审；没有样例实例，模型就永远停留在纸面，无法证明可用。详细操作见从零到一本体：建模流程与规范与能力问题（CQ）。

建模与 DDD 建模是什么关系

两者互补，不互相替代：

维度	DDD 建模	本体建模
解决的问题	边界内的行为、聚合不变量	跨边界的共享语义、数据互操作
核心产物	领域模型 + 代码	形式化模式（TBox）+ 数据（ABox）
一致性手段	聚合内不变量、事务	推理器一致性检查 + SHACL 校验
验证方式	用例、测试	能力问题、SPARQL 查询、样例数据
迭代节奏	随需求迭代	随需求与数据源变化迭代

实践中的常见做法是：DDD 负责“业务怎么变、流程怎么走”，本体负责“概念是什么、数据怎么换算”。同类概念可以对齐，例如类与实体/值对象类型、个体与实例、模块与限界上下文、约束与聚合不变量，但不要在一个聚合内部引入 OWL 推理来维护不变量——那是过度设计，见本体模型是什么、不是什么。

建模到什么程度算够

从最小可用本体开始，而不是从全量公理开始。一个能支撑 3–5 条 CQ 的本体，通常只需 5–20 个类、10–30 个属性。先让这条链路跑通：模式 → 样例数据 → 查询/校验 → 服务或报表。之后再按需增加类、公理和 SHACL 形状。

需要回答“模型质量如何评估”时，见质量评估与反模式；需要决定用 SKOS、OWL 还是 SHACL 时，见标准与语言总览。

建模最常犯的四个错误

1. **把 ER 图或类图当本体。**图形本身没有共享 IRI、没有形式语义、不能被推理，也无法跨系统复用。它们可以是建模过程中的草图，但不是交付物。
2. **只画图不落地。**没有样例数据、没有查询、没有接入任何系统，模型无法被验证，也无法证明价值。
3. **过度公理化。**一上来写大量等价类、不相交、属性链和基数限制，推理变慢且难以维护；开放世界假设下模型“永远不完整”，靠公理补完整性是错误方向。
4. **无人维护。**本体是长期资产，一次性交付后无人评审、无版本记录，半年后连原建模者都说不清某个类为什么存在。

FAQ

不写代码、只画概念图可以吗?

不可以作为交付形态。概念图可以作为评审材料，但本体必须有可加载的形式化文件，并且至少有一组样例数据和查询能证明它可用。否则无法区分“模型”和“幻灯片”。

必须用 OWL 吗?

不一定。只做词表与分类，用 SKOS 更简单；只做数据校验，SHACL 就足够；需要类层次与推理才上 OWL。选型判断见标准与语言总览，不要为了“标准”而堆标准。

必须一次建全整个领域吗?

不需要，也不可能。领域本体会长期演进，第一版覆盖核心场景即可。预留扩展点（模块划分、命名空间）比提前建全更重要，见模块化与复用。

没有领域专家怎么建模?

领域专家不一定是“行业专家”，可以是产品经理、资深业务人员，甚至是熟悉业务的工程师。关键是有人能确认术语含义和业务规则。若确实找不到人，用现有系统数据、单据和报表反推，并把假设写进注释，待评审时确认。

怎么判断模型够好了?

首要是能力问题通过率，其次是一致性检查通过、术语可理解、复用方愿意用。不要追求公理数量或类图美观，见质量评估与反模式。

建模要用什么工具?

编辑器用 Protégé 或 WebProtégé；校验与查询用 Apache Jena、RDF4J 或 pyshacl；存库选三元组库或文件。工具只影响效率，不影响建模是否成立。平台选型见技术栈与平台总览。

接下来读什么

- 想知道元素怎么组织：读本体元素总览，再按概念与类、对象属性与关系、数据属性与取值、个体与实例数据、公理、约束与业务规则、模块化与复用顺序推进。
- 想先看结果：读端到端案例：电商商品与订单本体，从 CQ 到查询完整走一遍。
- 想先补基础：读本体工程方法论。
- 想了解常见可复用套路：读本体设计模式。
- 对照 DDD：读建模与实现。

本体工程方法论

本体不是一次性的建模作业，而是要被系统长期使用、被多人维护、被反复扩展的资产。既然它是资产，就需要工程方法：明确范围、有验收标准、能评审、可版本化。否则最常见的结局是“画了半年类图，上线时没人用”。

本页给出方法论谱系、推荐的轻量流程、角色分工，以及与 DDD 建模流程的对照。具体操作见 建模流程与规范，验收见 能力问题，质量见 质量评估与反模式。

为什么本体需要工程方法

直接打开 Protégé 开始建类，通常会在三个地方出问题：

- **范围失控**：什么都想建，类越来越多，却回答不了任何一个具体业务问题；
- **没有验收**：模型“看起来对”，但没有可执行的检查，缺陷要到数据接入后才暴露；
- **无法演进**：没有版本、没有评审、没有文档，第二次修改就没人敢动。

工程方法解决的正是这三件事：先定边界，再定验收，最后用流程保证可持续。

方法论谱系

方法论	大致年代	核心主张	对本指南的价值
Ontology Development 101	2001	从领域与范围出发，列术语、建类层次、定属性、造实例，迭代推进	最小可操作的入门流程
METHONTOLOGY	1997	覆盖规格说明、概念化、形式化、实现、维护的完整生命周期	提醒我们维护阶段同样重要
On-To-Knowledge	1999–2002	面向知识管理，强调应用场景驱动	本体要服务于具体应用
NeOn	2012	网络化本体工程，强调复用、分布式协作与场景化方法	复用优先、模块化协作
敏捷 / XD	2000s 末起	用例驱动、小步迭代、模式复用	小团队落地最实用

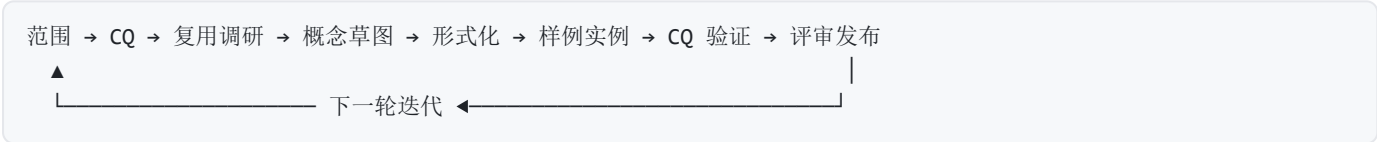
这些方法论并不互斥。实践中推荐“101 的骨架 + 敏捷的节奏 + NeOn 的复用意识”：流程轻、反馈快、能长期维护。

推荐轻量流程

步骤	产出	退出条件
1. 确定范围	领域边界、目标应用、不做什么	一句话说清“这个本体为谁回答什么问题”
2. 编写能力问题	CQ 清单（首批 5–10 条，扩展期 10–30 条）	每条都能判定、能追溯到来源
3. 复用调研	可复用标准/本体清单与取舍	明确“用哪些、不用哪些、为什么”
4. 概念草图	术语表、类与关系草图	领域专家认可术语与主要关系
5. 形式化	Turtle/OWL 模型、命名空间与 IRI	语法通过、可被工具加载
6. 样例实例	覆盖 CQ 的样例数据	每条 CQ 都有可验证的数据
7. CQ 验证	SPARQL 验收查询与结果	目标 CQ 通过率达到约定阈值
8. 评审发布	评审记录、版本与变更说明	评审通过并打版本号

八个步骤不是瀑布，而是螺旋：每轮聚焦一批 CQ，发布一个小版本，再进入下一轮。第一轮的目标不是“建全”，而是“打通一条从模型到查询的链路”。

text



与 DDD 建模流程的对照

维度	DDD 建模	本体工程
起点	业务问题与领域专家	能力问题与应用场景
主要产出	领域模型、聚合、领域事件、限界上下文	类、属性、公理、个体、形状约束
关注点	行为、边界、一致性、演进	共享语义、数据互操作、推理与校验
典型工作坊	事件风暴	CQ 工作坊、术语对齐会
验证方式	代码实现、测试、可运行系统	SPARQL CQ 验收、推理一致性、SHACL 校验
关系	管“系统怎么做事”	管“数据与概念是什么意思”

两者是互补关系。事件风暴产出的领域事件、聚合和限界上下文，可以作为本体的输入；本体产出的统一术语与语义层，也能反过来约束 DDD 的通用语言。熟悉 DDD 的读者可以把本体理解为通用语言的形式化延伸，但不要把聚合边界直接画成类层次：DDD 关注行为与边界，本体关注数据与概念的含义。

角色分工

- **本体工程师**：负责形式化、命名规范、推理与校验流水线，是模型质量的第一责任人。
- **领域专家**：提供术语、口径、边界与反例，参与 CQ 评审与模型验收。
- **数据工程师**：负责数据映射、ETL/ELT 管道、实例质量与性能。

- **架构师**：决定语义层在系统中的位置、存储与推理策略，见 实现总览。
- **产品/治理角色**：决定优先级、预算与推广节奏，见 落地路线图。

小团队可以一人多角色，但“谁对术语拍板”必须明确；没有拍板人，术语会随会议记录漂移。

FAQ

能不能不写 CQ，直接写 OWL？ 小范围、个人项目可以，但团队协作和长期维护建议首批至少写 5–10 条 CQ。CQ 是最便宜的需求文档，也是唯一能自动验收的部分。

必须写文档吗？ 至少要有：IRI 规范、类与属性的定义、典型实例、CQ 清单。只靠模型文件，新成员无法理解设计意图。

领域专家没时间怎么办？ 用 CQ 访谈代替长会：每次一小时，只确认 5–10 条问题与术语，回去建模，下次带结果来。不要等“专家有空”才开始。

第一版要建多全？ 只建能通过第一批 CQ 所需的最小集合。模型是长出来的，不是一次设计完的。

要不要追求完美模型？ 不要。不存在唯一正确的本体，只有对当前问题够用、且能演进的模型。过度设计比不设计更难维护。

常见失败模式

- **一次性项目**：交付模型文件后无人维护，半年后与业务脱节。
- **建模与数据脱节**：模型里有的属性，源系统根本没有数据；或数据有，模型没建。
- **只建模不验收**：没有 CQ 查询、没有校验，问题全部推迟到上线后。
- **过度形式化**：全量 OWL DL 公理，推理慢、没人看得懂，最后被弃用。
- **跳过复用**：把 schema.org、行业标准里已有的概念重新造一遍。

识别这些信号并及时回退，比严格执行某个方法论更重要。接下来从 能力问题 开始动手。

能力问题（CQ）

能力问题（Competency Question，CQ）是本体必须能够回答的问题。它是需求与验收之间的桥梁：向上承接业务诉求，向下变成可执行的 SPARQL 查询与样例数据。没有 CQ 的本体项目，最后只能靠“大家觉得模型对不对”来验收。

这个概念最早出现在 TOVE 项目中，后来成为本体工程的标准实践。方法很简单：先写出你想让本体回答的问题，再检查模型是否回答得出来。

什么是 CQ

一条好的 CQ 长这样：

> 给定一个客户，列出他最近 90 天内下单、尚未发货的订单及其金额。

它同时具备三个特征：有明确的输入（客户）、有明确的输出（订单列表与金额）、结果可以被验证（查出来的数据对不对）。CQ 不是抽象目标（“支持客户管理”），也不是技术任务（“建立客户表”），而是**一个能被具体回答的领域问题。**

CQ 清单本质上就是本体的功能需求文档。模型建成什么样，取决于你要回答哪些问题；反过来，每条 CQ 都应该能在模型上写出查询。

好 CQ 的标准

- **面向用户语言**：用业务方的说法描述，不出现 `owl:Class`、`rdf:type` 这类词。
- **可判定**：能写出查询，或有明确的判定方式；含糊的“是否合理”不合格。
- **范围明确**：输入、输出、时间范围、边界条件写清楚。
- **覆盖边界**：除了正常情况，还要有“没有数据”“跨多个对象”“不满足条件”的问题。
- **粒度适中**：一条 CQ 对应一个查询或一组紧密相关的查询，不要一条包住整个系统。
- **可追溯**：标明来源（谁提的、对应哪个业务场景），便于后续评审。

反例与改写的对比：

不合格	问题	改写
系统要支持商品管理	不可判定	给定商品 ID，返回它的类目路径与当前售价
列出所有类	技术查询，不是业务问题	某商品属于哪些一级类目
订单是否合规	太笼统	某订单是否存在“未支付却已发货”的明细
客户重要吗	无客观标准	近 12 个月消费额排名前 10% 的客户有哪些

示例一：电商

- **CQ-01**：给定客户，列出其近 90 天内下单且尚未发货的订单及金额。

- **CQ-02**：给定商品，找出所有包含该商品的进行中订单（反向查询）。
- **CQ-03**：某订单的应付总额是否等于其明细金额之和？列出不一致的订单。
- **CQ-04**：哪些客户在同一订单中同时购买了商品 A 与商品 B？
- **CQ-05**：给定仓库，列出受其库存影响的未发货订单。

CQ-03 是典型的“跨对象一致性”问题，它逼迫模型同时具备订单总额与明细金额两个属性，并能做聚合比较：

sparql

```
PREFIX ex: <https://example.com/ontology/order#>

SELECT ?order ?declared (SUM(?lineAmount) AS ?calculated)
WHERE {
  ?order a ex:Order ;
         ex:orderAmount ?declared .
  OPTIONAL {
    ?order ex:contains ?line .
    ?line  ex:lineAmount ?lineAmount .
  }
}
GROUP BY ?order ?declared
HAVING (SUM(?lineAmount) != ?declared)
```

如果模型里没有 `ex:lineAmount`，或者明细与订单之间没有可连接的属性，这条查询就写不出来——这就是 CQ 在暴露模型缺口。

示例二：制造

- **CQ-11**：给定成品，列出其 BOM 的全部下级零件及累计用量（多级展开）。
- **CQ-12**：某零件发生工程变更时，影响哪些在产工单与已排产订单？
- **CQ-13**：给定设备与工序，当前适用的工艺参数版本是哪一个？
- **CQ-14**：某供应商的来料最近一次质检不合格，涉及哪些批次与成品？

制造场景的 CQ 往往涉及层次结构与时间版本：BOM 展开需要属性路径或递归查询，工艺参数需要区分“生效区间”，供应商追溯需要串起批次与成品。这些正是 设计模式 中的部分-整体模式与时间区间模式要解决的问题。

CQ 与用例、用户故事、验收标准的关系

形式	关注点	与本体的关系
用户故事	谁、想要什么价值	提供 CQ 的来源与优先级
用例	交互流程与系统行为	CQ 通常对应用例中的信息需求
验收标准（AC）	功能行为是否可测	关注“系统做什么”，CQ 关注“数据能回答什么”
能力问题（CQ）	本体能否回答某个领域问题	直接转化为 SPARQL 验收查询

一句话区分：AC 验证系统行为，CQ 验证模型表达能力。两者可以互相推导，但不能互相替代。

把 CQ 转成验收查询

流程固定为五步：

1. 选一条 CQ，写出输入与期望输出；
2. 在模型中识别所需的概念、属性与路径；
3. 写 SPARQL 查询；
4. 准备覆盖该 CQ 的样例实例数据；
5. 把查询与期望结果放进自动化测试，每次模型变更都回归。

查询写不出来，说明模型缺元素；查询能写但结果为空，说明样例数据没覆盖或映射有问题；查询结果与预期不符，说明模型语义或数据有问题。三类失败分别对应不同的修复动作，比笼统的“模型评审”有效得多。

维护 CQ 清单

CQ 是活的资产，建议用一张表管理，并纳入版本控制：

字段	说明
ID	CQ-01 这样的稳定编号，不复用
问题	一句话，业务语言
来源	业务方、法规、系统需求或数据治理诉求
优先级	必须有 / 应该有 / 可以有
验收查询	SPARQL 文件路径或查询 ID
样例数据	覆盖该 CQ 的数据集标识
状态	待建模 / 已通过 / 阻塞 / 废弃
负责人	模型负责人与数据负责人

指标上可以跟踪“CQ 通过率”：已通过 CQ 数 / 纳入范围的 CQ 数。它比类数量更能说明本体是否真的可用。

常见错误

- **CQ 写成技术查询**：“查询所有 `owl:Class`”不是能力问题，是调试语句。
- **CQ 太笼统**：“支持全渠道订单管理”无法验收，也无法建模。
- **只写正向问题**：忽略空结果、边界值、异常路径，模型会在真实数据上翻车。
- **CQ 与模型脱节**：CQ 写完就存档，从不执行查询，等于没有。
- **CQ 数量失控**：几十上百条不分优先级，项目永远无法收敛。首批控制在 5–10 条，扩展期再逐步增加到 10–30 条，并持续做优先级排序。
- **忽略数据可得性**：模型能回答，但源系统没有数据，等于不可用。

小结

CQ 是本体工程里投入产出比最高的一步：几页问题清单，就能同时解决范围、验收和沟通三个问题。先写 CQ，再建模型；先跑查询，再谈发布。接下来按 建模流程与规范 把 CQ 落成模型。

从零到一本体：建模流程与规范

前两页讲了方法论与 CQ。这一页把流程落到实处：从一张空白画布，到可评审、可发布、可维护的第一版本体。重点是可操作的步骤、IRI 与命名规范、Protégé 操作要点，以及团队协作时的差异。

1. 确定范围与复用策略

先用一段话写清范围，包含四件事：**为谁、回答什么问题、数据从哪来、不做什么。**例如：

> 为订单履约团队回答“订单当前状态、明细构成、发货与异常”相关问题；数据来自订单库与仓储系统；不覆盖支付清结算与售后。

范围确定后立刻做复用调研，不要从零造概念：

策略	适用情况	代价
直接复用	领域通用概念，标准质量高（如 `schema.org` 的商品、组织、时间）	受标准表达方式约束
引入上层本体	需要严格分类与互操作（如 BFO、IAO、SOSA/SSN）	学习成本与推理负担
仅做映射	外部标准与内部模型不一致，但需要交换	需要维护映射与转换
完全自建	业务特有概念，标准未覆盖	需要自己保证质量与文档

复用的原则是“能映射就不引入，能引入就不重造”。引入一个上层本体意味着接受它的公理体系与推理后果，不是加几行 `owl:imports` 那么简单。复用与模块化细节见 模块化与复用。

2. 术语表与命名规范

在写任何 Turtle 之前，先建术语表：中文术语、英文术语、定义、同义词、来源、负责人。它是模型与业务方之间的契约，也是后续多语言标签的来源。

IRI 设计建议遵循以下约定：

- 本体基址用稳定的 HTTPS 域名，例如 `https://example.com/ontology/order#`；
- 单一文档内的词汇用 `#` 片段，跨文档的模块用 `/` 路径，便于按文件拆分；
- 本体自身有 IRI（`owl:Ontology`），每个发布版本有 `owl:versionIRI`，术语 IRI 保持稳定；
- 类用大驼峰单数（`Order`、`OrderLine`），对象属性用小驼峰动词或介词（`placedBy`、`contains`），数据属性用小驼峰（`orderAmount`）；
- 个体用业务标识（`.../order/1001`），不要用自增数据库 ID 或随机串；个体 IRI 与本体 IRI 分属不同命名空间；
- 不要用中文或空格做 IRI，不要频繁改 IRI，弃用用 `owl:deprecated` 标记而不是删除。

turtle

```
@prefix ex:      <https://example.com/ontology/order#> .
@prefix owl:     <http://www.w3.org/2002/07/owl#> .
@prefix rdfs:    <http://www.w3.org/2000/01/rdf-schema#> .
```

```
@prefix dcterms: <http://purl.org/dc/terms/> .

ex: a owl:Ontology ;
    dcterms:title       "订单本体"@zh , "Order Ontology"@en ;
    dcterms:description "订单、明细、履约状态的概念模型。"@zh ;
    owl:versionInfo     "0.3.0" ;
    owl:versionIRI      <https://example.com/ontology/order/0.3.0> .
```

3. 概念草图

草图阶段只做两件事：列出类、列出主要关系。不要过早设计属性类型和基数，更不要直接开 Protégé。白板或绘图工具都可以，但要保证每张图都能标出它服务哪几条 CQ。

草图评审的检查点是：术语是否与业务一致、有没有把角色/状态/事件误画成类、主要关系方向是否清楚。这一步改起来最便宜，到了形式化阶段再改成本就高了。

4. 形式化

形式化按“类 → 对象属性 → 数据属性 → 少量公理 → 形状约束”的顺序推进：

- 先定类层次，只保留有业务意义的父类，避免为了“整齐”造中间层；
- 对象属性必须有 `rdfs:domain` / `rdfs:range`，但要记住它们是推理公理，不是校验；
- 数据属性选定 XSD 类型，金额用 `xsd:decimal`，时间用 `xsd:dateTime` 并统一时区；
- 公理从 OWL profile 的 EL 或 RL 起步，只加有推理价值的少数几条；
- 校验需求另写 SHACL 形状，不要塞进 OWL。

Protégé 的操作要点：新建本体时填好本体 IRI；在 Entities 页建类与属性；对象属性面板设置 domain/range、逆属性与特征；用内置推理器（如 HermiT）启动一致性检查；用 DL Query 页验证类表达式；最后另存为 Turtle 并提交到 Git。WebProtégé 适合多人评审与评论，但不适合复杂编辑。

5. 样例实例

为每条首批 CQ 准备 3–10 个代表性实例，覆盖正常情况、边界情况与“应该查不到”的情况。样例数据单独放一个文件（如 `examples.ttl`），不要与模型混在一起。样例有两个用途：验证模型能否装下真实数据，以及为 CQ 查询提供回归基线。

6. CQ 验证

把 能力问题对应的 SPARQL 查询与期望结果放进自动化测试：

bash

```
# 模型语法检查
riot --validate ontology.ttl

# 数据形状校验
pyshacl -s shapes.ttl -d examples.ttl -f human
```

推理一致性检查可以通过 OWL API + HermiT、Jena 的推理模型或对应语言的推理库完成，不一定有现成命令行。企业环境通常会把语法检查、推理器、SHACL 与 CQ 查询放进同一条 CI 流水线。任何一项失败就阻断合并，这比人工评审可靠。

7. 评审与发布

评审至少覆盖：术语准确性、IRI 与命名、公理必要性、CQ 通过率、文档完整性。发布时打语义化版本号（如 `0.1.0` → `0.2.0`），写变更说明与弃用项，保留 `owl:versionIRI`。版本与兼容性策略见 版本与演进。

8. 文档化

最小文档集包括：README（范围、如何使用）、术语表、CQ 清单、模型图、典型查询、变更日志。文档不需要长篇大论，但必须能让新成员在半小时内知道“这个本体解决什么问题、从哪开始看”。

小团队与大团队的差异

维度	小团队	大团队
流程	一人多角色，1–2 周一轮	分模块并行，双周评审
评审	领域专家 + 一名工程师即可	设立本体委员会与发布门禁
工具	Protégé + Git + 本地校验	WebProtégé、注册表、CI 流水线
复用	优先 schema.org 等轻量标准	引入上层本体与行业标准，建立映射治理
风险	无人维护、文档缺失	模块冲突、版本失控、评审瓶颈

常见风险

- **范围蔓延**：所有需求都往本体里塞，版本永远发不出去。用 CQ 优先级控制。
- **命名漂移**：不同人给同一概念起不同名字，合并时才发现。术语表是唯一裁决。
- **只建模不验证**：没有 CI，缺陷全部留到集成阶段。
- **复用陷阱**：引入庞大上层本体后推理变慢、团队看不懂，最后整体弃用。
- **文档滞后**：模型改了文档没改，新成员按旧文档建模。

把上面每一步的产出物固定下来，流程就自然可重复。下一节从质量角度检查这个流程产出的模型是否靠谱：质量评估与反模式。

质量评估与反模式

判断一个本体好不好，不能只看类图画得整不整齐。质量要能被检查：一致性、可满足性、覆盖度、可理解性、可维护性，每一项都有对应的检查手段。这一页给出评估维度、OntoClean 的用法、评审清单，以及最常见的反模式与修复方式。

质量维度

维度	含义	检查手段
一致性（consistency）	公理之间不矛盾	推理器一致性检查
可满足性（satisfiability）	没有不可满足的类	推理器列出不可满足类
覆盖度（coverage）	能回答目标 CQ	CQ 通过率
可理解性（comprehensibility）	人能看懂命名、定义与结构	评审、术语表、文档
可复用性（reusability）	模块边界清晰、少耦合	模块依赖检查
可维护性（maintainability）	变更影响可控、有版本	变更历史与回归测试
数据契合度	源数据能映射进来	样例数据与映射验证

其中前两项是“硬失败”，必须为零；覆盖度决定本体是否有用；其余决定它能否长期活下去。

OntoClean：用元属性检查 is-a

OntoClean（Guarino 与 Welty 提出）用四个元属性判断分类层次是否合理，是评估 `subClassOf` 最有效的工具：

- **刚性（rigidity）**：一个类是否对其实例必然成立。`Person` 是刚性的（人必然是人），`Student`、`Employee`、`Customer` 是非刚性的（可能只是某个阶段扮演的角色）。
- **同一性（identity）**：类是否提供判定“两个实例是否同一个”的标准。
- **统一性（unity）**：实例是否有明确的整体边界，例如 `Person` 有，`AmountOfWater` 没有。
- **依赖性（dependence）**：实例的存在是否依赖另一个实例，例如 `Husband` 依赖 `Wife`。

最常用的规则是：**非刚性类不能包含刚性类作为子类。**`Person rdfs:subClassOf Student` 是错误建模，因为人有不是学生的时候；反过来 `Student rdfs:subClassOf Person` 是允许的，但要注意角色类会带来“同一实体在不同时间扮演不同角色”的问题。角色、状态这类非刚性概念，更适合用 角色模式 表达，而不是永久改变实例的类型。

同一性冲突也常见：把按 SKU 识别的 `Product` 与按 ISBN 识别的 `Book` 做成父子类，会让子类继承错误的身份标准。正确做法是拆开或用映射表达“一本书也是一种商品”。

推理器一致性检查

推理器能发现两类问题：本体矛盾（inconsistent ontology）与不可满足类（unsatisfiable class）。后者更隐蔽：整个本体一致，但某个类永远不可能有实例。

turtle

```
@prefix ex:  <https://example.com/ontology/hr#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

ex:Employee a owl:Class ;
    rdfs:subClassOf ex:Person ;
    owl:disjointWith ex:Person .
```

这段模型语法正确、能加载，但 `Employee` 被声明为 `Person` 的子类又与 `Person` 不相交，因此它不可能有实例。推理器会报出不可满足类；如果不跑推理，这个错误可能潜伏很久。

实践建议：每次模型变更都在 CI 中跑一次推理一致性检查；用 OWL API、Jena 推理模型或对应语言的推理库都行；对关键公理保留“为什么加”的记录，避免后人不敢删也不敢改。

CQ 通过率

覆盖度用 CQ 通过率量化：

text

```
CQ 通过率 = 已通过的 CQ 数 / 纳入本版本范围的 CQ 数 × 100%
```

“通过”指 CQ 对应的 SPARQL 查询在样例数据上返回预期结果，且查询本身能在模型上执行。建议每轮迭代记录通过率与失败原因，形成趋势而不是一次性数字。发布阈值应由团队结合 CQ 难度与范围自定；例如首版以 80% 通过率作为门槛只是一个常见示例，剩余未通过项进入下一轮。

评审清单

检查项	合格标准
命名	类大驼峰单数、属性小驼峰、IRI 稳定、无中文与空格
定义	每个类与属性有 `rdfs:comment`，术语表有条目
标签	有 `rdfs:label`，多语言场景有语言标签
层次	无循环 `subClassOf`，无万能父类，角色与类型分开
公理	每条公理有明确推理目的，无冗余的 `domain` / `range`
约束	必填、范围、格式由 SHACL 表达，而非 OWL
实例	样例数据覆盖 CQ 的正常与边界情况
验证	推理器无矛盾、无不可满足类；CQ 查询通过
文档	README、术语表、CQ 清单、变更日志齐备
版本	有版本号、`owl:versionIRI` 与弃用说明

常见反模式

类/实例混淆：把“北京”建成类、把“中国”建成类，然后困惑为什么无法查询具体城市。修复：具体对象建个体，只有需要分类的集合才建类。

同义词建类：`Customer` 与 `Client` 两个类，指同一概念。修复：保留一个类，另一个用 `skos:altLabel` 或注释记录，合并 IRI。

过度公理化：追求“语义完备”，加上大量不相交、全称限制、属性链，推理变慢且数据一有噪声就整体不一致。修复：公理按推理价值排序，只保留必要的；约束交给 SHACL。

循环定义：`A subClassOf B` 且 `B subClassOf A`，或 `equivalentClass` 成环。修复：这类循环通常意味着两个概念重复，应合并。

元建模滥用：用 punning（同一个 IRI 既是类又是个体）表达“类型本身也有属性”，结果推理语义混乱。修复：区分元层与对象层，必要时用注释或显式的元类模式。

把属性当类：把“颜色”建成类、把“红色”建成类，再让商品 `rdf:type ex:Red`。修复：颜色作为数据属性或 SKOS 概念，商品指向概念个体；只有当“颜色”本身有属性需要描述时才升级为类。

万能上层类：所有东西都挂在一个 `Thing` 下，层次没有信息量。修复：按业务语义分层，允许存在多个顶层类。

`domain` / `range` 当校验：期待类型推断报错。修复：需要校验就写 SHACL；`domain` / `range` 只用于推理。

模型与数据脱节：模型里有 `ex:warehouse`，但源系统根本没有仓库字段。修复：在建模阶段做数据可行性检查，样例数据必须真实可映射。

无文档、无版本：模型文件即全部。修复：最小文档集 + Git + CI 门禁。

持续评估

把质量检查固化进流水线，而不是依赖评审会：

1. 语法检查：Turtle 可解析、IRI 可解析；
2. 推理检查：无矛盾、无不可满足类；
3. 形状校验：SHACL 在样例与真实数据上通过；
4. CQ 回归：查询结果符合预期；
5. 评审门禁：命名、定义、文档 checklist 通过；
6. 版本记录：变更说明与弃用项完整。

指标上同时跟踪 CQ 通过率、校验失败率、不可满足类数量与查询性能，形成趋势图。质量不是一次评审的结论，而是每次提交都被验证的常态。接下来进入具体建模手法： 本体设计模式。

本体元素总览

一个可用的本体由五类核心元素构成：类、对象属性、数据属性、个体、公理。加上三类配套元素——约束、注释与元数据、模块——就覆盖了从建模到落地的全部构造。理解每个元素“回答什么问题”，比记住它的 OWL 写法更重要。

一张表看清八类元素

元素	回答的问题	主要构造	常见错误	DDD 类比
类 / 概念	领域里有哪些类型?	`owl:Class`、`rdfs:subClassOf`	把同义词建成类、类与个体混淆	实体/值对象的类型
对象属性	对象之间有什么关系?	`owl:ObjectProperty`、`owl:inverseOf`	方向混乱、把外键当语义	关联、聚合引用
数据属性	对象有哪些字面量取值?	`owl:DatatypeProperty`、`rdfs:range`	用字符串存日期/金额	值对象属性
个体	具体有哪些对象?	`owl:NamedIndividual`、类型断言	IRI 不稳定、重复个体	聚合实例、实体实例
公理	概念体系有什么逻辑规律?	等价、不相交、`hasKey`、属性链	过度公理化、用公理做校验	不变量、规则
约束	数据是否完整、是否合规?	SHACL 形状	用 `rdfs:domain` 当校验	聚合不变量、DTO 校验
注释/元数据	给人和机器补充说明与出处?	`rdfs:label`、`skos:altLabel`、`dcterms:source`	只写在文档里、不写进模型	通用语言词汇表
模块	模型如何切分与复用?	命名空间、`owl:imports`	单体本体、循环导入	限界上下文

表中的 DDD 类比只是帮助定位，不是等价关系：本体没有封装，也不承担行为实现，把它当成“另一种类图”会立刻走偏。

五要素与三配套的关系

- **类 + 对象属性 + 数据属性** 构成 TBox 的骨架，决定“能描述什么”。
- **个体** 构成 ABox，决定“实际描述了什么”。
- **公理** 给骨架附加逻辑语义，让推理器能推出隐含知识。
- **约束** 与公理互补：公理管一致性（不矛盾即可），约束管完整性（缺数据就报错）。这一分工是本体工程里最容易踩坑的地方。
- **注释与元数据** 决定模型可理解、可追溯，是长期维护的成本项。
- **模块** 决定模型能否增量演进和被复用。

元素与 OO、ER 的对应与差异

从类图或 ER 图转过来的人，最容易把两套东西按字段一一映射。差异主要在四点：

1. **属性是一等公民。**OWL 属性是独立的 IRI，有自己的 domain、range、逆属性、传递性；类图里的字段只是类的一部分，无法单独被引用和推理。
2. **类没有成员列表。**一个个体属于某类，靠类型断言，而不是“构造函数的参数”。同一个个体可以同时属于多个类。
3. **没有封装与行为。**本体描述“是什么、有什么关系”，不描述“怎么执行”。业务行为仍在代码里，见实现本体总览。
4. **类型可以推理出来。**借助 `rdfs:subClassOf`、`rdfs:domain/range` 等公理，个体可能被推出新的类型；这在类图里不存在。

建模不必一次建全

一个能跑通闭环的最小本体通常只需要：3–5 个类、3–5 个对象属性、2–3 个数据属性、2–3 条公理、1 个 SHACL 形状、10 条左右样例数据。先写这个闭环，再按 CQ 扩展。

推荐的书写顺序是：**类 → 对象属性 → 数据属性 → 公理 → 样例个体 → 约束。**理由很实际：类和关系稳定后才谈公理，有了样板数据才知道约束该写多严。

元素数量不是成熟度指标。一个 20 个类、注释齐全、能力问题全过的本体，比 200 个类、一半没有注释、没人使用的本体有价值得多。

元素归属：哪些进本体，哪些进别处

内容	放在哪	说明
类、属性、公理	本体文件（`.ttl` / `.owl`）	稳定、跨系统共享
SHACL 形状	独立的 shapes 文件	校验规则变化频繁，与模式解耦
常用查询	独立查询文件或用例代码	查询是使用方式，不是模式
IRI 常量、命名空间	代码常量类/模块	避免字符串散落，见实现本体对象
术语说明与变更记录	本体注释 + 版本库	`rdfs:comment`、`owl:versionInfo`

阅读顺序建议

1. 概念与类：先把分类层次建对，这是最影响后续一切的决定。
2. 对象属性与关系：关系决定图的结构，也最容易出现方向错误。
3. 数据属性与取值：字面量、类型与计量建模。
4. 个体与实例数据：什么时候需要实例、IRI 怎么定。
5. 公理：用最少的公理表达必要的逻辑。
6. 约束与业务规则：把完整性交给 SHACL。
7. 模块化与复用：让模型可增长、可复用。

FAQ

本体元素等于类图元素吗?

不等于。类图元素以类型和字段为中心；本体元素以可共享的 IRI、属性和可推理语义为中心。类图可以看作本体的一种“面向对象视图”。

约束算本体元素吗?

在 OWL 本体本身里没有“约束元素”这一说，`rdfs:domain/range` 与基数限制都是推理公理。真正面向校验的约束由 SHACL 提供，属于本体的配套资产，通常单独维护，见约束与业务规则。

注释和元数据重要吗?

重要，而且越大的本体越重要。没有 `rdfs:label` 和 `rdfs:comment`，建模者离开后模型就难以维护；没有来源和版本信息，跨团队复用时无法追溯。成本很低，收益很高。

每个元素都必须写注释吗?

不必逐个写长篇说明，但四类元素必须有注释：类（一句话界定范围）、对外属性（业务含义与单位）、公理（为什么需要它）、公共个体（状态、币种的取值含义）。私有、临时、只在一个查询里用到的元素可以只保留 label。判断标准是“半年后新人能否不问人就看懂”。

要不要把领域里所有个体都建出来?

通常不要。个体一般来自业务数据库、主数据或 ETL，不应手工维护成百上千条。本体先定模式与关键参考数据（状态、单位、类目），批量实例由数据管道生成，见个体与实例数据。

概念与类

类是本体里最基础的元素，代表“一组可能成员的集合”。建类就是做分类：把领域里稳定、可命名、需要被共享引用的类型提取出来。类建错了，后面的属性、公理、约束都会跟着错，所以这是最值得慢下来的一步。

类的语义：集合与分类器

在 OWL 里，类不是“表”也不是“代码里的 class”，而是一个集合。个体通过类型断言成为类的成员：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

shop:Order a owl:Class ;
  rdfs:label "订单"@zh , "Order"@en ;
  rdfs:comment "客户提交的购买请求，一个订单至少包含一个订单行。"@zh ;
  rdfs:subClassOf shop:BusinessDocument .

shop:BusinessDocument a owl:Class ;
  rdfs:label "业务单据"@zh .
```

记住两个后果：其一，`shop:Order` 的成员由类型断言和推理共同决定，不靠“字段列表”；其二，OWL 默认开放世界，推理器不会因为“没见到某个个体”就断定它不属于某个类。这决定了类不能用来表达“必须是这些值”的封闭约束，那属于 SHACL 的职责。

命名与注释

命名规范直接影响本体的可维护性，建议至少遵守四条：

- **IRI 稳定**：类 IRI 一旦发布就不要改，改名用 `owl:deprecated` 过渡，见版本与演进。
- **局部名用英文单数 CamelCase**：`shop:Order`、`shop:OrderLine`，不要用中文或复数。
- **中文靠语言标签**：`rdfs:label "订单"@zh`，不要把它塞进 IRI。
- **写清边界**：`rdfs:comment` 一句话说明“这个类包含什么、不包含什么”，比长篇定义有用。

分类层次与 subClassOf

`rdfs:subClassOf` 表达子集关系，并且是传递的：`shop:RetailOrder rdfs:subClassOf shop:Order`，`shop:Order rdfs:subClassOf shop:BusinessDocument`，则零售订单也是业务单据。这会影响查询（用属性路径可以一次取到所有子类）和推理（引入新的类型断言）。

判断一条 `is-a` 是否成立，用最简单的测试：**“每一个 A 都是 B 吗？”** 零售订单是订单，成立；订单是客户吗？不成立。常见误用是把角色或状态建成子类，例如把“已支付订单”建成 `shop:Order` 的子类——这会让同一个订单在不同时间属于不同类，分类语义被破坏。状态应建成个体加属性，或用角色模式处理，见本体设计模式。

多继承在 OWL 里是允许的，也别害怕用；但层次过深（超过 4–5 层）通常说明划分维度混乱，不如拆成多个扁平维度。层次是否“自然”可以借助 OntoClean 的刚性、身份、统一性等元属性评估，见质量评估与反模式。

类、个体、属性值、角色怎么选

这是建模时问得最多的问题。一个实用判定流程：

1. **它是稳定类型，还是具体对象？** 类型建类（`shop:Customer`），具体对象建个体（`data:customer-1001`）。
2. **它是固定的小集合，且需要被其他对象引用吗？** 是则建成“类 + 个体”的受控取值，例如订单状态、货币、计量单位。只在文本里出现、不需要被引用的取值，用数据属性即可。
3. **它的身份是否随情境变化？** 同一个人在不同场景下是买家、卖家、客服，这是角色而非类型，适合用角色模式或“参与者 + 角色类型”结构。
4. **它是否只是某个类的取值集合？** 可以用 `owl:oneOf` 或 SKOS 受控词表表达，不必建完整类层次。

订单状态就是一个典型例子：它需要被比较、被校验、被查询，所以建成个体而不是字符串：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

shop:OrderStatus a owl:Class ;
  rdfs:label "订单状态"@zh .

shop:StatusDraft     a owl:NamedIndividual , shop:OrderStatus .
shop:StatusPaid      a owl:NamedIndividual , shop:OrderStatus .
shop:StatusShipped   a owl:NamedIndividual , shop:OrderStatus .
shop:StatusCancelled a owl:NamedIndividual , shop:OrderStatus .
```

如果四个状态必须互不相同，用 `owl:AllDifferent` 声明；如果订单只能取其中一个，用函数属性（`owl:FunctionalProperty`）或 `sh:maxCount 1` 限制“至多一个”；如果状态必须来自这四项，用 SHACL 的 `sh:in` 做封闭校验。注意“个体互不相同”与“枚举类（`owl:oneOf`）”是不同层面的语义：前者约束个体，后者定义类，详见公理。

类划分的四条原则

- **按本质属性划分**：按“它是什么”而不是“它当前处于什么状态”“谁在使用它”划分。
- **避免交叉分类**：同一个层次里不要同时按来源和支付方式分，否则组合爆炸，新增维度就要改模型。
- **同义词不建类**：“客户”“顾客”“买方”指向同一概念时用一个类，其余写 `skos:altLabel` 或 `rdfs:label` 的多语言/别名，不要建三个类再对齐。
- **不留兜底类**：`OtherThing`、`Misc` 一类会把分类问题藏起来，宁可暂时不分类。

元类与 punning

元类是“类的类”，例如把“产品类目”本身当作可操作对象。OWL 2 允许 punning：同一个 IRI 既可以出现在类的位置，也可以出现在个体的位置，两者的解释互不干扰。它能解决“类目既是分类维度又是业务对象”的表达问题，但会让模型对非专业读者变难。

多数情况下更简单的做法是二选一：要么类目作为类层次（`shop:ElectronicsProduct rdfs:subClassOf shop:Product`），要么类目作为个体并用 `skos:broader` 组织层级。只有确实需要同时具备两种身份时才用 punning，并在注释里写清意图。

常见错误与修复

错误	症状	修复
把具体对象建成类	`shop:Shanghai` 是类，查询要按城市枚举	改为个体，或建“城市”类并用个体表示
同义词各建一类	客户/顾客/买方三个类，数据分散	合并为一类，别名写注释
状态建成子类	订单随状态改变类型，推理结果混乱	状态建个体，用属性关联
层次过深	五六层 is-a，没人能说清区别	拆成多个扁平维度
类没有注释	新人无法判断该放哪个类	每类至少一句 `rdfs:comment`

两个常见疑问

类必须有实例吗？ 不必。类可以先建、后补数据；空类在开放世界下完全合法。但空类长期存在且无人使用，通常说明模型过度设计。

子类之间必须互斥吗？ 不必，除非业务确实要求。互斥是公理，一旦声明，违反就会导致推理器判定不一致；如果只是“数据里不该同时出现”，用 SHACL 校验更合适，报错也更友好。

类定好之后，下一步是对象属性与关系：类只说明“有哪些类型”，关系才把图连起来。

对象属性与关系

对象属性连接两个个体，是本体的骨架：类只说明“有哪些类型”，关系才把图和语义连起来。建关系时真正要做的决策只有三个——方向怎么定、语义多强、什么时候需要中间个体。

命名与方向

对象属性用一个能读成句子的动词或介词命名，局部名用小写驼峰：`shop:placedBy`（订单由客户下单）、`shop:hasLine`（订单含订单行）、`shop:reportsTo`（员工向经理汇报）。方向选“更自然、更常被查询”的那一侧，另一侧用逆属性补齐。

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

shop:placedBy a owl:ObjectProperty ;
  rdfs:domain shop:Order ;
  rdfs:range shop:Customer ;
  owl:inverseOf shop:placedOrder ;
  rdfs:label "下单客户"@zh .

shop:hasLine a owl:ObjectProperty ;
  rdfs:domain shop:Order ;
  rdfs:range shop:OrderLine ;
  owl:inverseOf shop:lineOfOrder ;
  rdfs:label "包含订单行"@zh .
```

`owl:inverseOf` 不复制数据，它让推理器能从“订单由客户下单”推出“客户下了订单”，因此更适合把两头都定义清楚，而不是让数据里同时写两份方向、结果还可能不一致。

domain 与 range 是推理公理，不是校验约束

这是本体工程中最常见的误解。`rdfs:domain` 与 `rdfs:range` 的作用是**类型推断**：如果某个三元组使用了该属性，推理器就推断主语属于 domain、宾语属于 range。它不会检查、也不会报错。

turtle

```
# 数据方向写反了：客户 placedBy 订单
@prefix data: <https://example.com/shop/data/> .
@prefix shop: <https://example.com/shop/ontology#> .

data:customer-1001 shop:placedBy data:order-1 .
```

配合上面的定义，推理器不会报“类型错误”，而是推出两个结论：`data:customer-1001` 是订单，`data:order-1` 是客户。开放世界假设下，这不一定造成不一致（除非再声明这两个类不相交），于是错误数据被“合法化”了。

所以：**需要校验就写 SHACL。**同一个含义，在 SHACL 里是明确的检查：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix sh: <http://www.w3.org/ns/shacl#> .

shop:OrderShape a sh:NodeShape ;
  sh:targetClass shop:Order ;
  sh:property [
    sh:path shop:placedBy ;
    sh:class shop:Customer ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:message "订单必须且只能有一个下单客户，且客户类型必须为 Customer" ;
  ] .
```

同理，`rdfs:range` 写的 `xsd:decimal` 也不阻止你填入字符串——那是 SHACL 的 `sh:datatype` 的职责。把 domain/range 当校验用，会让团队误以为数据已经受控，问题往往在上线后才暴露。

常用属性特征

特征	语义	典型用途	注意
`owl:inverseOf`	逆关系	双向查询	两个属性语义必须真正互逆
`owl:TransitiveProperty`	传递	上级组织、包含关系、祖先类目	成环会推出自反关系，配合非自反声明才会不一致
`owl:SymmetricProperty`	对称	合作、相邻	不要用于有方向的业务关系
`owl:FunctionalProperty`	至多一个值	身份证明号、主联系方式	缺少值不报错；不能当“必填”
`owl:InverseFunctionalProperty`	值唯一标识主语	邮箱反查账号	与唯一性约束不是一回事
`owl:propertyChainAxiom`	属性链推出新关系	部门负责人 = 部门-上级-负责人	链过深会增加推理成本
`owl:hasKey`	关键值标识个体同一性	员工工号	推出 `sameAs`，不拒绝重复数据

以组织汇报链为例：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

shop:reportsTo a owl:ObjectProperty , owl:TransitiveProperty ;
  rdfs:domain shop:Employee ;
  rdfs:range shop:Employee ;
  rdfs:comment "传递的汇报关系；不要与组织架构中的虚线汇报混用。"@zh .
```

```
shop:departmentHead a owl:ObjectProperty ;
  owl:propertyChainAxiom ( shop:belongsToDepartment shop:departmentManager ) .
```

n 元关系：不要硬塞进二元属性

订单、商品、数量、成交价构成一个典型的四元关系。用二元属性只能表达“订单买了商品”，数量、单价无处安放——挂到订单上会丢失行级信息，挂到商品上则完全错误。正确做法是具体化（qualified relation）：引入“订单行”个体承载关系，每个订单行指向订单、商品，并自带数量与金额。

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

shop:OrderLine a owl:Class ;
  rdfs:label "订单行"@zh ;
  rdfs:comment "订单与商品之间的限定关系，承载数量、成交价等行级属性。"@zh .

shop:lineProduct a owl:ObjectProperty ;
  rdfs:domain shop:OrderLine ;
  rdfs:range shop:Product .

shop:quantity a owl:DatatypeProperty ;
  rdfs:domain shop:OrderLine ;
  rdfs:range <http://www.w3.org/2001/XMLSchema#integer> .
```

这类结构在本体设计模式里叫“限定关系/情形”模式，更多变体见本体设计模式。

关系 vs 外键、关联表

维度	对象属性	外键/关联表
本质	语义关系，可被推理	存储与引用完整性机制
方向	可有逆属性	单向，反向靠索引
校验	需 SHACL/RDFS 配合	数据库约束
跨库	用 IRI 全局指代	依赖本地 ID
多值	天然多值	关联表或 JSON

做语义集成时，外键只说明“物理上关联”，对象属性还要说明“在业务上是什么关系”。两者不是替代关系：关系库里照常用外键，本体层负责跨系统的语义表达。

基数放在哪

- **一致性用 OWL**：`owl:minCardinality`、`owl:maxCardinality`、`owl:exactCardinality` 是公理，配合推理器判断模型是否自相矛盾，但开放世界下“数据暂缺”不会触发错误。
- **业务校验用 SHACL**：`sh:minCount`、`sh:maxCount` 在封闭世界语义下直接报错，适合“订单必须有且仅有一个客户”这类规则。

两者可以并存：OWL 里表达“逻辑上不可能有两个主客户”，SHACL 里表达“发布数据时必须有一个客户”。分工原则见约束与业务规则。

关系建模的四条经验

- **先问“这条关系是否需要独立存在”**：能被其他属性描述、有生命周期、需要被单独引用时，具体化成个体；否则保持二元属性。
- **一个语义只建一个属性**：“购买”“下单”“下单于”不要各建一个，用逆属性与查询方向解决。
- **关系方向服从业务主语**：优先让“主角”当主语，查询时用逆属性或反向模式。
- **命名要能被业务复述**：把属性名读成一句完整的话，业务方听不懂，说明语义还没定义清楚。

什么时候用属性链

属性链适合表达“可由路径推出”的关系，例如“部门负责人 = 部门所属上级部门的负责人”。它能减少冗余数据，但链过深或引入循环会让推理变慢、结果难解释，因此只用在稳定且高频的 CQ 上。若某个关系需要频繁查询却没有稳定推导路径，宁可显式存储。

常见坑

1. **双向属性没有用 `owl:inverseOf` 对齐**，数据写两份，长期不一致。
2. **用 `owl:FunctionalProperty` 当必填**：它只限制“至多一个”，不限制“至少一个”。
3. **把取值放到关系上**：“含税价”是数据属性，不是对象属性；关系只连接个体。
4. **方向随查询方便而改**：方向应服从业务语义，查询方向用逆属性或 SPARQL 反向模式解决。
5. **属性链过深**：三层以上属性链会让推理变慢，也难以向业务解释。

关系建好后，接着处理数据属性与取值：关系连接个体，数据属性给个体挂上具体值。

数据属性与取值

数据属性把个体连接到字面量：价格、日期、编号、名称。它看起来像“建个字段”这么简单，但实际上类型选择、语言标签和计量结构决定了数据能否被正确查询、比较和校验。一个常见的坏味道是：模型结构很漂亮，金额是字符串、日期没有时区、单位散落在注释里。

三种“属性”先分清

类型	连接什么	是否参与推理	典型用途
对象属性	个体与个体	是	订单属于客户、员工属于部门
数据属性	个体与字面量	是	金额、日期、编号
注释属性	个体/类与说明文字	否	`rdfs:label`、`rdfs:comment`、来源出处

判断方法：值是另一个对象就用对象属性；值需要被比较、聚合并可用 SHACL 校验，用数据属性；只是给人看的说明，用注释属性。不要把业务数据塞进注释属性——它不会参与一致性推理，语义上也不对。

选对 XSD 类型

场景	推荐类型	说明
名称、编号	`xsd:string`	编号保留前导零，不要转数字
数量、件数	`xsd:integer` / `xsd:decimal`	计数用整数
金额、价格	`xsd:decimal`	不用 `xsd:double`，避免二进制浮点误差
比例、评分	`xsd:decimal`	需要精度时显式声明
是否	`xsd:boolean`	不要用字符串 `"true"`
日期	`xsd:date`	只有日期无时间
时间点	`xsd:dateTime`	带时区偏移，避免长期歧义
时长	`xsd:duration` / `xsd:dayTimeDuration`	用于计算而非展示
URI	`xsd:anyURI`	链接、外部标识

类型选窄不选宽：能确定是整数就不要用 `xsd:decimal`，能确定是日期就不要用字符串。窄类型让校验和查询都更可靠，也能在数据加载时尽早暴露格式错误。

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .

shop:orderNumber a owl:DatatypeProperty ;
  rdfs:domain shop:Order ;
```

```
  rdfs:range xsd:string ;
  rdfs:label "订单号"@zh ;
  rdfs:comment "对外可见的订单编号，形如 ORD-20250301，保留前导零。"@zh .

shop:placedAt a owl:DatatypeProperty ;
  rdfs:domain shop:Order ;
  rdfs:range xsd:dateTime ;
  rdfs:label "下单时间"@zh .
```

字面量与语言标签

文本值有几种常见写法，注意 RDF 1.1 中 `"foo"` 与 `"foo"^^xsd:string` 是**同一个字面量**，其余写法语义不同：

turtle

```
@prefix data: <https://example.com/shop/data/> .
@prefix shop: <https://example.com/shop/ontology#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

data:product-1 shop:productName "无线键盘" .                  # 纯字符串，无语言
data:product-2 shop:productName "无线键盘"@zh .               # 中文语言标签
data:product-3 shop:productName "Wireless Keyboard"@en .     # 英文语言标签
data:product-4 shop:productName "无线键盘"^^xsd:string .     # 显式 xsd:string，与纯字符串等价，一般不必手写
```

需要多语言展示时一定要用语言标签，并用 `skos:prefLabel` 或 `rdfs:label` 区分展示名；`"foo"` 与 `"foo"@zh` 在 RDF 中是不相等的字面量，混用会导致查询漏数据、SHACL 的 `sh:uniqueLang` 也失去意义。

数值、货币、计量：不要留裸数字

`shop:price "199"` 的问题不是类型，而是“199 什么单位、什么币种”。两种处理方式，按复杂度取舍：

扁平双字段适合单一币种、单一单位系统的小型项目：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

shop:listPrice a owl:DatatypeProperty ;
  rdfs:range xsd:decimal .

shop:priceCurrency a owl:DatatypeProperty ;
  rdfs:range xsd:string .
```

计量对象（Quantity/Money 模式）适合多币种、多单位、需要换算或参与规则计算的场景：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix data: <https://example.com/shop/data/> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

```
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

shop:MonetaryAmount a owl:Class ;
  rdfs:label "金额"@zh .

shop:amountValue a owl:DatatypeProperty ;
  rdfs:domain shop:MonetaryAmount ;
  rdfs:range xsd:decimal .

shop:amountCurrency a owl:ObjectProperty ;
  rdfs:domain shop:MonetaryAmount ;
  rdfs:range shop:Currency .

data:order-total a shop:MonetaryAmount ;
  shop:amountValue "1288.00"^^xsd:decimal ;
  shop:amountCurrency data:CNY .

shop:Currency a owl:Class ;
  rdfs:label "货币"@zh .

data:CNY a shop:Currency .
```

后者多一个中间个体，但换来了可校验（币种必须是受控值）、可换算（汇率规则作用于金额对象）、可扩展（加税、折扣、精度来源）。判断标准很简单：如果未来会有汇率、单位换算或多币种报表，就用计量对象；否则扁平字段更省事。

把校验交给 SHACL

数据属性的 `rdfs:range` 只影响推理，不做检查。完整性与取值范围用 SHACL：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix sh:  <http://www.w3.org/ns/shacl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

shop:ProductShape a sh:NodeShape ;
  sh:targetClass shop:Product ;
  sh:property [
    sh:path shop:listPrice ;
    sh:datatype xsd:decimal ;
    sh:minInclusive 0 ;
    sh:message "商品标价必须是非负 decimal" ;
  ] ;
  sh:property [
    sh:path shop:priceCurrency ;
    sh:in ( "CNY" "USD" "EUR" ) ;
    sh:message "币种必须是受支持的 ISO 货币" ;
  ] .
```

`sh:datatype` 检查字面量类型，`sh:minInclusive` 检查数值下界，`sh:pattern` 检查格式，`sh:languageIn` / `sh:uniqueLang` 检查语言标签。这些在 OWL 里都没有等价物，不要试图用公理模拟。

多值、默认值与更新语义

RDF 里没有“默认值”“空值”和“字段”的概念，只有三元组，这带来四个与关系库不同的约束：

- **多值**：同一主语、同一属性可以出现多个宾语；需要唯一时靠 SHACL 的 `sh:maxCount` 限定。
- **缺失**：不写该三元组就是缺失，不存在 `null`；要区分“不知道”和“没有”，用不同的建模方式（例如用独立状态个体表达“未登记”）。
- **更新**：修改是删除旧三元组、插入新三元组，没有原地赋值，写入方必须保证幂等。
- **默认值**：不要在模型里隐式补默认值，把默认值写进数据或在应用层填充，避免“看起来有、查出来没有”。

注释属性与业务数据的边界

`rdfs:label`、`rdfs:comment`、`dcterms:source` 这类注释属性只用于说明，不参与推理和一致性检查。可以用它们记录术语来源、生效时间和负责人，但不要把状态、金额等业务取值放进去——那样既难以被 SHACL 的 `sh:in` 校验，也无法参与聚合查询。

与 DDD 值对象的关系

值对象是“没有身份、由属性值定义相等性”的概念，映射到本体通常有两种做法：拆成多个数据属性（简单、查询方便），或建一个类加数据属性（可复用、可约束、可与其他概念关联）。`Money`、`Address`、`DateRange`、`Contact` 这类值对象，建议建成类；`color`、`note` 这类单字段值，直接数据属性。

区别在于：代码里的值对象承载行为（格式化、比较），本体里的值只承载语义。行为留在应用层，不要试图在 RDF 里表达“金额相加”的方法。

常见坑

1. **金额用 `xsd:double`**：出现 `199.00000000000003` 这类精度问题，报表对不上。
2. **日期存字符串**：无法做时间区间比较与排序，`FILTER` 里只能做字典序。
3. **单位/币种全局约定**：模型里看不出来，换一个数据源就错。
4. **同一属性多值且无约束**：一个订单挂三个 `totalAmount`，SHACL 没写 `maxCount` 就发现不了。
5. **用空字符串表示“没有值”**：应直接不写该三元组，用 `sh:minCount` 区分“必须填”和“可以不填”。
6. **丢掉时区**：跨时区业务里 `2025-03-01T10:15:00` 与带偏移的时间含义不同。
7. **同一属性混用单位**：有的数据存“厘米”、有的存“米”，没有单位建模就无从发现。

属性与取值定好后，就轮到个体与实例数据：模式只是蓝图，真正被查询的是个体。

个体与实例数据

模式（TBox）回答“领域里有什么”，个体（ABox）回答“具体有哪些”。本体能不能用起来，最终取决于个体数据是否准确、可追溯、可更新。很多项目模式写得很规范，实例数据却靠手工维护、IRI 随意生成，几个月后图里全是重复和悬空引用。

TBox 与 ABox 的边界

- **TBox**：类、对象属性、数据属性、公理，跨系统共享，变化慢。
- **ABox**：个体、类型断言、关系断言、取值断言，来自业务系统，变化快。

边界不是绝对的。订单状态集合既可以是 TBox 里的枚举类，也可以是 ABox 里的参考数据；判断标准是“由谁维护、以什么频率变化”。模式与实例通常放在不同的文件或命名图中，便于独立版本管理。

个体与数据库行的区别

数据库行由表结构定义，靠主键识别，一行的列值是固定的。个体则由 IRI 标识、靠断言描述，可以：

- 同时属于多个类（一个订单既是销售订单又是促销订单）；
- 在任意时刻被追加新的断言（第二个月补上发票信息）；
- 被多个系统用同一个 IRI 引用，无需数据复制；
- 与另一系统的对象建立 `owl:sameAs` 关联。

一个数据库行可以映射成多个个体（订单行、明细、地址），也可能多个行合并成一个个体（一个客户在三个渠道的记录合并）。

个体 IRI 命名策略

策略	形式	优点	风险
业务 ID	`data:order-ORD20250301`	可读、可对账、幂等	ID 规则变更会导致 IRI 变化
代理键	`data:order-8f3c9a2e`	稳定、不暴露业务	可读性差，排查困难
内容哈希	地址、联系人	自然去重	内容一改就是新个体
空白节点	`[ ... ]`	方便表达中间结构	不能跨系统引用，不可追踪

实践建议：**业务实体用稳定业务 ID，纯内部结构用空白节点，绝不拿会变的字段（邮箱、手机号、地址）当 IRI**。发布后的个体 IRI 视为不可变，需要重命名时保留 `owl:sameAs` 或 `dcterms:replaces` 指向，避免引用断裂。

turtle

```
@prefix data: <https://example.com/shop/data/> .
@prefix shop: <https://example.com/shop/ontology#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
```

```
data:order-ORD20250301 a shop:Order ;
  shop:orderNumber "ORD-20250301" ;
  shop:placedAt "2025-03-01T10:15:00+08:00"^^xsd:dateTime ;
  shop:placedBy data:customer-1001 ;
  shop:orderStatus shop:StatusPaid ;
  shop:hasLine data:line-1 , data:line-2 .

data:line-1 a shop:OrderLine ;
  shop:lineOfOrder data:order-ORD20250301 ;
  shop:lineProduct data:product-1 ;
  shop:quantity 2 ;
  shop:lineAmount "398.00"^^xsd:decimal .

data:customer-1001 a shop:Customer ;
  shop:customerId "C-1001" .
```

注意显式写出 `a shop:Order`。没有类型断言的个体仍然存在，但 `sh:targetClass` 找不到它，按类查询也查不到——“数据在，但看不见”是排查成本很高的故障。

什么时候必须个体化

这些对象不适合压成一个属性值，而应建成个体：订单、订单行、地址、事件、评估结果、审批记录，以及任何需要独立时间戳或来源的关系。判断标准是“是否需要被单独引用、追溯或关联”。

反过来，不要把每条业务流水都手写进本体文件。个体应由 ETL、API 同步或数据管道生成，本体只保留关键参考数据（状态、币种、单位、类目）作为受控取值。

sameAs 的风险与实体链接

`owl:sameAs` 是等价公理：声明两个 IRI 指同一个体后，推理器会把两边的所有断言合并。它能解决“同一客户在 CRM 和 ERP 有两个 ID”的问题，但一旦链接错误，错误属性会污染整个图，而且很难定位。

turtle

```
@prefix data: <https://example.com/shop/data/> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .

data:customer-1001 owl:sameAs <https://crm.example.com/customer/C-1001> .
```

使用原则：

1. **只在人工或高置信度规则确认后使用**，批量匹配结果先进“候选表”复核。
2. **不确定的映射用** `skos:exactMatch`、`skos:closeMatch`，它们只是映射标注，不会触发合并推理。
3. **避免链式 sameAs 成环**，必要时做去重与传递闭包检查。
4. 反向操作也要考虑：开放世界、无唯一名称假设意味着两个不同 IRI 完全可能指同一对象，不要默认“不同 IRI 就是不同对象”。

实体链接的工程流程与质量指标见知识图谱构建。

实例数据从哪来

来源	常用方式	注意
关系库	R2RML/RML 映射、SQL 导出后转 Turtle	主键映射为稳定 IRI
业务 API	定时拉取后生成 JSON-LD	分页与增量游标
CSV/Excel	模板映射脚本	编码、日期格式、空值
JSON 文档	JSON-LD `@context` 直接表达	上下文版本要固定

无论哪种方式，都要保证**幂等**：同一份源数据重复导入不产生重复个体；更新用 SPARQL Update 的 `DELETE` / `INSERT` 或整批替换，删除源数据时同步删除断言。否则图会越跑越脏。

另外，导入时保留来源信息（`dcterms:source` 或 `prov:wasDerivedFrom`），出现争议时能追到源记录；批量导入建议用命名图区分批次或来源，便于按批校验与回滚。

实例数据质量

导入后至少跑四类检查：重复个体（同一业务 ID 出现在多个 IRI）、缺失必填断言（SHACL `minCount`）、悬空引用（`shop:placedBy` 指向不存在的客户）、类型冲突（同时是互不相交的两个类）。前两类用 SHACL，后两类用一致性检查加查询。

质量规则与治理流程见约束与业务规则和数据治理与主数据。

常见坑

- 用空白节点表示订单、客户等业务实体，跨系统引用时无法对齐。
- 拿邮箱、手机号做 IRI，用户改一次号码，历史数据就断链。
- 个体没写 `rdf:type`，按类查询和 SHACL 校验双双失效。
- 滥用 `owl:sameAs` 做去重，错误合并后无法回滚。
- 只导入不删除，源系统已作废的数据长期留在图里。
- 用可变字段拼接 IRI，字段一改就断链；应使用稳定业务 ID 或代理键。

公理

公理是本体里“能推出新知识的断言”。它和普通数据断言的区别不在写法，而在后果：普通断言只描述事实，公理会参与推理、改变类型归属、产生隐含关系，甚至让整个模型变得不一致。写得少而准，是公理设计的第一原则。

公理与普通断言

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix data: <https://example.com/shop/data/> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

# 普通断言：客户 C-1001 下了订单
data:order-1 shop:placedBy data:customer-1001 .

# 公理：所有订单都必须有一个下单客户
shop:Order rdfs:subClassOf [
  a owl:Restriction ;
  owl:onProperty shop:placedBy ;
  owl:someValuesFrom shop:Customer
] .
```

前者描述一次具体事实，后者描述一类对象的逻辑规律。推理器加载两者后，会推出 `data:customer-1001` 是 `shop:Customer`（借着 domain/range），也可能为缺失的订单推出匿名客户。**推理不校验数据**，这一点必须反复强调。

常用公理清单

公理	作用	推理效果	主要风险
`rdfs:subClassOf`	分类层级	推出子类个体也属于父类	层次过深、误用 is-a
`owl:equivalentClass`	类等价	双向分类，推出大量新类型	意外归类、推理变慢
`owl:disjointWith` / `owl:AllDisjointClasses`	类互斥	同时属于两类即不一致	数据脏时整库不可用
`owl:equivalentProperty`	属性等价	两边断言合并	映射错误污染图
`owl:inverseOf`	逆关系	推出反向边	语义不真正互逆
属性特征	传递/对称/函数/反函数等	推出新关系或唯一值	函数属性被当必填
`rdfs:domain/range`	属性适用类型	推断主语/宾语类型	被误当校验
`owl:hasKey`	关键值标识个体	相同 key 推出 `sameAs`	被误当唯一约束
`owl:propertyChainAxiom`	属性链	组合路径推出新关系	链过深、循环
`sh:…`（非 OWL）	校验	不推理，只产出报告	与公理混为一谈
基数限制	数量约束	一致性检查	开放世界下不报缺数据

公理强度阶梯

同一个业务事实往往有多种表达，强度从弱到强：

1. **注释说明**：`rdfs:comment "订单必须有客户"`——零推理成本，只给人看。
2. **弱公理**：`rdfs:subClassOf`、`rdfs:domain/range`——帮助分类与推断。
3. **中强公理**：不相交、函数属性、`hasKey`——引入不一致风险。
4. **强公理**：等价类、枚举类、完整基数限制——改变分类结果，推理代价高。

从弱往强选。只有当某条公理能直接支撑一条能力问题（CQ）时才加它，否则写注释就够了。等价类尤其要谨慎：`owl:equivalentClass` 会让推理器按定义重新归类所有个体，一个写宽的定义能把半个图卷进来。

和 SHACL 的分工

维度	OWL 公理	SHACL 约束
目的	保证逻辑一致性	保证数据完整性/合规
语义	开放世界，缺失不等于违反	封闭世界，缺失即违反
失败表现	推理器报 inconsistent，整个图不可用	校验报告列出具体违规节点
典型用途	状态至多一个、类层次、属性特征	必填、范围、格式、条件规则

判断口诀：**“逻辑上不可能”用公理，“数据上不允许”用 SHACL。**例如“一个订单至多只能有一个状态”适合公理；而“订单必须有订单号”适合 SHACL——因为缺号只是数据不完整，不是逻辑矛盾。

三个典型例子

状态至多一个：状态在本指南里建成个体，用函数属性限制“至多一个值”，再用 `owl:AllDifferent` 声明四个状态互不相同：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .

# 一个订单至多有一个状态
shop:orderStatus a owl:ObjectProperty , owl:FunctionalProperty .

# 四个状态个体互不相同
[] a owl:AllDifferent ;
  owl:members ( shop:StatusDraft shop:StatusPaid
                shop:StatusShipped shop:StatusCancelled ) .
```

两条公理合起来才有效果：函数属性给出“至多一个状态”，`AllDifferent` 给出“四个状态互不相同”，于是同一订单若同时断言 `shop:StatusPaid` 与 `shop:StatusCancelled`，推理器会判定不一致。只写函数属性而不声明 `AllDifferent`，开放世界会把两个状态推成 `sameAs` 而不是报错；只写 `owl:AllDisjointClasses` 也不行，它的 `owl:members` 只接受类，而这里的状态是个体。这套组合的价值在于尽早暴露脏数据，代价是“一处脏、全库挂”，所以生产环境通常配合数据修复流程使用。

员工工号唯一：`owl:hasKey` 表达“工号相同的个体是同一员工”：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .

shop:Employee owl:hasKey ( shop:employeeNumber ) .
```

注意它不是数据库的唯一约束：两个员工都填 `E-001` 时，推理器推出他们是同一个人，而不是报错。要拒绝重复，用 SHACL 或数据库约束。这也解释了为什么“用 hasKey 做唯一性校验”是个常见坑。

订单必须包含行：`someValuesFrom` 表达“至少存在一个订单行”：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

shop:Order rdfs:subClassOf [
  a owl:Restriction ;
  owl:onProperty shop:hasLine ;
  owl:someValuesFrom shop:OrderLine
] .
```

在开放世界里，一个没有任何行断言的订单仍然可以“存在某个未记录的行”，所以不会报错。要做“发布前必须至少一行”的检查，仍然需要 `sh:minCount 1`。

基数限制该不该用

`owl:minCardinality`、`owl:maxCardinality`、`owl:exactCardinality` 可以挂在类或属性上，但它们在开放世界下只参与一致性检查：缺少数据不会被判违规，只有“断言了多个值、又声明最多一个”时才触发不一致。因此基数限制适合表达“逻辑上不允许”，例如“一个人最多有一个出生日期”；“录入时必须填出生日期”仍应写成 `sh:minCount 1`。

基数写得越精确，推理代价越高，对数据完整度的要求也越强，通常只在核心概念上使用。多数情况下，用 `sh:minCount`/`sh:maxCount` 表达业务要求、用少量 `owl:maxCardinality` 表达逻辑不可能，是更稳妥的组合。

过度公理化的代价

- **推理变慢**：等价类、属性链、大基数限制显著增加推理时间，尤其是全量物化时。
- **物化膨胀**：每个推理结果都要存储，图规模可能数倍增长，见推理与物化。
- **难以解释**：业务方看到“客户变成了订单”这类推断结果时，信任度会迅速下降。
- **维护困难**：一条公理改动可能影响全图分类，回归测试成本高。

选择推理 profile（EL/QL/RL）并评估推理器能力，是控制成本的工程手段。多数业务场景用 RDFS 加少量 OWL 2 EL 公理就够了。

公理评审清单

- 这条公理对应哪条 CQ 或哪个明确的业务事实？
- 能否用 SHACL 校验代替？如果只是“数据要合规”，答案通常是能。
- 引入后，不一致的风险是什么？数据脏时如何定位与修复？
- 是否使用了等价类？定义是否窄到只覆盖目标类？
- 推理器在目标 profile 下能否处理？预计物化规模多大？
- 公理是否有注释和变更记录，半年后还能解释清楚吗？
- 新增公理后推理耗时与物化规模是否仍在可接受范围？记录基线，避免逐步劣化。

公理定完后，把它与校验规则一起组织：见约束与业务规则。推理引擎与 profile 的工程细节见OWL 2 本体语言。

约束与业务规则

本体只保证“说得通”，不保证“数据齐”。约束要解决的问题是：数据是否完整、是否符合业务规则、是否在发布前被拦截。这件事不能交给 OWL 公理，而应该交给 SHACL、应用层和数据库，各自分担不同层次的规则。

三层分工

维度	OWL 公理	SHACL 约束	应用层/规则引擎
语义	开放世界	封闭世界	命令执行时
目的	逻辑一致性	数据完整性与合规	流程、权限、跨系统
失败表现	推理器报不一致	校验报告列出违规节点	抛出异常、拒绝请求
典型规则	状态至多一个、类层次	必填、范围、格式、条件	审批、扣减库存、调用支付
执行时机	加载/推理	入库、发布、巡检	每次业务操作

判断口诀：**逻辑上不可能 → OWL；数据上不允许 → SHACL；流程上不允许 → 应用层。**把三者混在一起，最常见的结果是规则既没在入口拦住，也无法向业务解释。把三层各自的规则列成清单，比争论“该用 OWL 还是 SHACL”更有价值。

规则分类与实现手段

规则类型	例子	首选实现
必填	订单必须有下单客户	`sh:minCount 1`
唯一	工号不重复	SHACL-SPARQL 或数据库唯一索引
取值范围	金额 ≥ 0	`sh:minInclusive`
数据类型	日期必须是 dateTime	`sh:datatype`
格式	订单号形如 ORD-20250301	`sh:pattern`
枚举	状态只能取四个值之一	`sh:in`
长度	备注不超过 500 字	`sh:minLength` / `sh:maxLength`
跨字段	折后价不高于原价	`sh:lessThan` 或 SHACL-SPARQL
条件	已支付必须有支付时间	SHACL-SPARQL 或 `sh:or`
引用完整性	订单行必须指向存在的商品	`sh:class` / `sh:node`
时序	发货时间晚于支付时间	应用层或查询巡检

SHACL 擅长结构、取值和局部条件；跨记录、跨时间窗口、涉及外部系统的规则，放在应用层更实际。

约束放在哪：三个执行点

1. **数据入口**：ETL 写入、API 提交、主数据发布时先校验，问题数据不落库。成本最低、收益最大。
2. **发布前**：本体新版本、类目、参考数据发布前跑全量校验，避免“模式升级、数据不合规”。
3. **定期巡检**：对历史数据做周期性扫描，发现存量问题并进入修复队列。

约束本身是声明式的，必须有人执行：离线用 pyshacl/Jena 的 SHACL 校验，在线用服务层封装。执行点与架构位置见分层与六边形架构。

严重级别与错误消息

SHACL 允许给形状和约束设置严重级别与消息，这直接决定运维体验：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix sh:   <http://www.w3.org/ns/shacl#> .

shop:OrderShape a sh:NodeShape ;
  sh:targetClass shop:Order ;
  sh:severity sh:Violation ;
  sh:property [
    sh:path shop:orderNumber ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:pattern "^ORD-[0-9]{8}$" ;
    sh:name "订单号" ;
    sh:message "订单号缺失、重复或格式不符（应形如 ORD-20250301）" ;
  ] .
```

`sh:Violation` 表示必须修复，`sh:Warning` 表示可疑但不阻塞，`sh:Info` 仅提示。消息用业务语言写，不要写“constraint violation”；否则业务方看不懂报告，规则就失去了治理价值。

完整示例：订单校验

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix sh:   <http://www.w3.org/ns/shacl#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

shop:OrderShape a sh:NodeShape ;
  sh:targetClass shop:Order ;
  sh:property [
    sh:path shop:placedBy ;
    sh:class shop:Customer ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
  ] ;
  sh:property [
    sh:path shop:totalAmount ;
    sh:datatype xsd:decimal ;
    sh:minInclusive 0 ;
```

```
  ] ;
  sh:property [
    sh:path shop:hasLine ;
    sh:minCount 1 ;
    sh:node shop:OrderLineShape ;
  ] ;
  sh:property [
    sh:path shop:orderStatus ;
    sh:maxCount 1 ;
    sh:in ( shop:StatusDraft shop:StatusPaid
            shop:StatusShipped shop:StatusCancelled ) ;
  ] .

shop:OrderLineShape a sh:NodeShape ;
  sh:targetClass shop:OrderLine ;
  sh:property [
    sh:path shop:quantity ;
    sh:datatype xsd:integer ;
    sh:minInclusive 1 ;
  ] ;
  sh:property [
    sh:path shop:lineProduct ;
    sh:minCount 1 ;
    sh:class shop:Product ;
  ] .
```

条件规则用 SHACL-SPARQL 表达更直接：

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix sh:   <http://www.w3.org/ns/shacl#> .

shop:PaidOrderRule a sh:NodeShape ;
  sh:targetClass shop:Order ;
  sh:sparql [
    a sh:SPARQLConstraint ;
    sh:severity sh:Violation ;
    sh:message "已支付订单必须有支付时间" ;
    sh:select """
      PREFIX shop: <https://example.com/shop/ontology#>
      SELECT $this ?status WHERE {
        $this shop:orderStatus ?status .
        FILTER (?status = shop:StatusPaid)
        FILTER NOT EXISTS { $this shop:paidAt ?t }
      }
    """ ;
  ] .
```

`$this` 是 SHACL 注入的焦点节点变量。SPARQL 约束灵活但可读性差，能用原生约束组合表达时优先用原生约束。

与 DDD 聚合不变量配合

DDD 的不变量在聚合根内由代码保证，事务提交即生效；SHACL 校验的是数据层的“发布口径”。两者不是替代关系：

- **聚合内不变量**（订单总额等于行金额之和）必须在代码里强一致地维护，不能依赖事后校验。
- **聚合间规则**（客户信用额度、库存可用量）通常用最终一致或校验兜底。
- **跨系统共享的合规规则**（字段格式、受控取值）用 SHACL 统一表达，所有数据源共用一套规则。

避免把同一条规则在代码、数据库、SHACL 里各写一遍且口径不同。做法是维护一份规则清单，标注权威实现位置，其余位置只做防御性检查。

从业务规则到 SHACL 的翻译步骤

业务方给出的往往是自然语言规则，直接写形状容易漏项。推荐按四步翻译：

1. **拆对象**：规则约束的是哪类节点？确定 `sh:targetClass` 或 `sh:targetNode`。
2. **定属性**：规则检查哪个属性路径？必要时用 `sh:path` 的多段路径或逆路径。
3. **选组件**：取值类规则优先用原生约束（`sh:in`、`sh:pattern`），跨字段与条件规则用 `sh:sparql`。
4. **配消息与级别**：写业务语言消息、设严重级别，并定义违规后的处理动作（拒收、告警、进修复队列）。

翻译完用样例数据回归：至少一条合规数据、一条违规数据，确保形状真的会报错，而不是永远通过。

规则不是写完就结束

规则会随业务变化，必须有版本与测试：每条规则有编号、负责人和生效时间；修改规则前先在历史数据上跑一遍，评估影响面；废弃规则要保留记录并说明原因。规则清单本身也要有检索入口，方便业务方按“订单”“商品”找到对应规则。没有治理的规则集合，半年后会比没有规则更难维护。

失败模式

1. **把 SHACL 当推理**：以为校验能补出缺失的客户，实际上 SHACL 只报告问题。
2. **只在批处理跑校验**：入口不拦，脏数据先落库，越积越多。
3. **规则写死在代码里**：业务方看不见、改不动，也没有统一报告。
4. **一个形状塞几十条约束**：报告无法定位，修复责任不清。
5. **不分严重级别**：全设成 Violation，告警疲劳后没人再看。
6. **条件规则硬塞进 SHACL**：跨实体、跨时间的复杂规则应上移到应用层或规则引擎。
7. **规则文档与实现脱节**：文档写“金额必须大于 0”，形状里却只校验了类型，上线后才发现。

规则与应用的完整落地方式见业务规则与校验；SHACL 语法细节见SHACL 形状约束。

模块化与复用

一个本体从小到大必然经历“一个文件写不下”的阶段。模块化不是洁癖，而是让本体可持续演进、可分工、可复用的工程手段。与之相伴的是复用决策：哪些概念应该复用标准词表，哪些必须自建。

为什么必须模块化

- **变更隔离**：改订单模块不牵动商品、客户模块，回归范围可控。
- **职责清晰**：每个模块有明确的主题与负责人，评审时知道该找谁。
- **按需加载**：不同应用只需加载自己依赖的模块，减少推理与校验负担。
- **复用与替换**：标准词表可以整体替换或升级，不必动业务模块。
- **发布节奏**：上层本体稳定、领域本体频繁迭代，分开才能各自发版。

反面教材是单体本体：一个上万行的 Turtle 文件，所有人都往里加，没人敢删，推理越来越慢，最后没人能说清某个类为什么存在。

三种切分手段

手段	作用	注意
命名空间/IRI	标识概念归属，避免重名	命名空间一旦发布就要稳定
文件/目录	物理组织，便于评审与版本管理	一个模块一个文件或一个目录
`owl:imports`	逻辑依赖，加载时合并公理	无“部分导入”，导入即全量生效

turtle

```
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

<https://example.com/shop/ontology> a owl:Ontology ;
  owl:imports <https://example.com/commons/party> ;
  owl:imports <https://example.com/commons/product> ;
  owl:versionInfo "1.2.0" ;
  rdfs:label "电商领域本体"@zh .
```

`owl:imports` 的语义是“把被导入本体的所有公理视为本本体的一部分”，没有只导入某个类的机制。这意味着**导入一个重型上层本体，推理器就会加载它的全部公理。**复用前先评估载荷，而不是见到标准就 import。

怎么划分边界

实用的划分依据有三个，通常组合使用：

1. **稳定性**：上层/基础本体（时间、单位、组织、地理）变化最慢，领域本体居中，应用本体变化最快。依赖方向只能从“易变”指向“稳定”，即应用本体 import 领域本体，领域本体 import 基础本体。

2. **主题**：一个模块一个主题，例如商品、订单、客户、履约。不要按“类一个文件、属性一个文件”切，那样所有变更都会扩散。
3. **所有权**：谁维护、谁评审、谁负责发布。模块边界与团队边界大致对齐，协作成本最低。

依赖图必须无环。循环 import 会让推理器加载顺序不可控，也意味着两个模块实际上应该合并或抽取公共部分。

复用标准词表

常见可复用词表及其取舍：

词表	解决的问题	复用方式	注意
schema.org	商品、订单、组织、事件等通用概念	对齐或直接用 IRI	面广但浅，不足以表达业务规则
FOAF	人、组织、社交关系	直接使用	与内部人员模型需要映射
DCAT	数据目录与数据集描述	直接使用	适合数据治理场景
PROV-O	数据来源与处理过程	直接使用	溯源建模的事实标准
SOSA/SSN	传感器、观测	直接使用	物联网场景
SKOS	受控词表、分类法	直接使用	不做逻辑推理，见SKOS 词表与分类法
ORG	组织结构与汇报关系	直接使用	与 HR 系统字段对齐
OWL-Time	时间区间与时刻	直接使用	时间区间推理
QUDT/OM	计量单位与量纲	直接使用	多单位换算场景
BFO/IAO	上层本体	谨慎复用	概念严谨但抽象，学习成本高

复用有三种强度，按成本从低到高：

- **只借用 IRI 词表**：在标签、映射关系里引用标准概念，不 import。
- **映射对齐**：自建概念 `owl:equivalentClass` 或 `rdfs:subClassOf` 到标准概念，保留自己的模型。
- **直接 import**：把标准词表作为依赖，自己的个体直接使用其类。

建议：核心业务概念自建（业务规则、流程、字段都在你自己的模型里），外围通用概念尽量复用标准词表。为了一个“地址”概念而 import 整个 schema.org，是典型的过度复用。

版本与依赖管理

- **版本化命名空间**：稳定 IRI 用于标识，版本 IRI（如 `.../2025-03/`）用于固定依赖；两者并存。
- **锁定依赖**：`owl:imports` 指向具体版本，避免上游静默变更导致推理结果漂移。
- **语义化版本**：加类、加可选属性通常兼容；删类、改语义、收紧公理不兼容。
- **废弃而不删除**：用 `owl:deprecated true` 加 `rdfs:comment` 说明替代方案，给使用方迁移时间。
- **CI 校验**：每次提交跑一致性检查、SHACL 校验和 CQ 回归，防止“改一处、坏一片”。

版本策略的完整做法见版本与演进。

与 DDD 限界上下文的对照

维度	限界上下文	本体模块
边界依据	领域语言一致性与业务自治	概念来源、复用层级与变更频率
自治方式	团队、代码库、发布节奏	文件、命名空间、版本
集成方式	API、事件、防腐层	`owl:imports`、映射对齐
失败模式	模型泥球、边界侵蚀	单体本体、循环依赖
共享方式	上下文映射、共享内核	上层本体、公共词表

两者可以对齐但不应简单等同：一个限界上下文内部的本地术语，不一定适合提升为全局本体概念；跨上下文共享的应该是经过对齐的核心概念，而不是某一方的实现细节。方法上的参照见建模。

实践清单

- 每个模块有明确的 owner、版本号、变更记录和导入清单。
- 模块内 IRI 前缀一致，对外暴露的概念都有 `rdfs:label` 与 `rdfs:comment`。
- 禁止循环导入；依赖方向单向。
- 序列化时可以按模块拆文件，加载时合并；发布时整包打版本。
- 模块变更走评审，至少跑一次 CQ 回归。
- 上层与标准词表依赖固定版本，升级单独排期。

常见坑

1. **循环 import**：A 导入 B、B 又导入 A，推理器行为不可控。
2. **单体本体**：所有概念挤在一个文件，无法分工与复用。
3. **同名不同义**：两个模块都建了 `Customer` 但含义不同，跨模块查询时张冠李戴。正确做法是使用不同 IRI 并显式映射，而不是随手 `sameAs`。
4. **依赖不锁版本**：上游一改，自己的推理结果和查询全部变化。
5. **过度复用**：为了几个类引入庞大的上层本体，推理成本和理解成本都失控。

模块规划完成后，建议先回到端到端案例：电商商品与订单本体看一个完整模型如何组织，或继续阅读本体设计模式了解可复用的结构套路。

本体设计模式

本体建模里有大量重复出现的问题：关系带属性怎么表达、事件怎么建、角色怎么处理、带单位的数量怎么存、历史区间怎么表示。每次从零设计，既慢又容易踩同样的坑。本体设计模式（Ontology Design Pattern，ODP）就是把这些成熟解法沉淀下来复用。

本页给出最常用的九种模式、适用场景与简短 Turtle 示例。原则是：**优先复用模式，其次组合模式，最后才自造。**

什么是本体设计模式

ODP 是针对某类建模问题的可复用方案，通常包含涉及的类、属性、公理与使用说明。社区有公开的模式目录（如 ontologydesignpatterns.org），也有按内容模式、逻辑模式、架构模式分类的整理。

它的价值不在“好看”，而在三点：减少重复错误、统一命名与结构、让评审有共同语言。一个团队长期使用同一套模式，模型的可读性会显著高于各自发挥。

使用模式的前提

模式服务于能力问题（CQ），不是反过来。选模式前先问：这条 CQ 需要什么查询？查询需要什么结构？如果简单结构就能回答，不要为了“标准”引入复杂模式。

另外要区分三种东西：**模式**是可复用方案，**标准**是规范（如 OWL、SHACL），**风格约定**是团队习惯（如命名）。不要把它们混为一谈。

1. n 元关系与限定关系

RDF 的三元组天然是二元的，但“客户在某个时间以某个角色参与订单”是三元甚至更多元的关系。常见解法是把它具体化为一个关系节点：

turtle

```
@prefix ex:  <https://example.com/ontology/order#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:Order-1001 ex:hasParticipation [
    a             ex:CustomerParticipation ;
    ex:participant ex:Customer-C42 ;
    ex:role        ex:Buyer ;
    ex:since       "2024-01-01"^^xsd:date
] .
```

这就是限定关系（qualified relation）的简化形式：关系节点承载限定信息，两端实体通过属性指向它。当关系只有“谁和谁”时用普通对象属性；一旦关系需要时间、角色、金额、来源等属性，就升级为关系节点。

2. 情形与事件

事件模式把“发生了什么”建成个体，而不是属性值。事件节点连接参与者、时间、地点与结果，天然支持多参与者与多时间点：

turtle

```
@prefix ex:  <https://example.com/ontology/order#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:Shipment-9001 a ex:ShipmentEvent ;
    ex:eventFor      ex:Order-1001 ;
    ex:fromWarehouse ex:Warehouse-WH2 ;
    ex:eventTime     "2024-05-01T10:00:00+08:00"^^xsd:dateTime .
```

情形（situation）模式则用一个节点把一组相关事实打包，表达“在某个上下文下成立”，例如某次报价适用的客户等级、渠道与有效期。它适合上下文相关的规则与快照，不适合替代普通关系。

3. 角色

角色是非刚性概念：同一个人可以同时是员工、客户、审批人。直接建 `Employee subClassOf Person` 在简单场景可用，但角色会变，类型不该随之永久改变。角色模式用角色节点表达：

turtle

```
@prefix ex:  <https://example.com/ontology/hr#> .

ex:Person-E1 ex:playsRole [
    a           ex:EmployeeRole ;
    ex:roleAt   ex:Organization-O1 ;
    ex:roleFrom "2023-03-01"^^<http://www.w3.org/2001/XMLSchema#date>
] .
```

角色节点承载“在哪个组织、从何时起、什么职责”。如果角色属性很少且稳定，也可以用子类简化；判断标准是“这个身份会不会随时间变化、是否需要保留历史”。

4. 部分-整体

部分-整体是最常用的结构关系，但“部分”至少有三种：组件（发动机之于汽车）、成员（员工之于部门）、物质（水之于湖）。它们的传递性与约束不同，不要用一个万能 `partOf` 打天下。

turtle

```
@prefix ex:  <https://example.com/ontology/manufacturing#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .

ex:partOf a owl:TransitiveProperty ;
    owl:inverseOf ex:hasPart .

ex:Engine-SN88 ex:partOf ex:Car-VIN001 .
ex:Car-VIN001 ex:partOf ex:Fleet-FL1 .
```

BOM 场景建议同时保留具体关系（`ex:hasEngine`、`ex:hasWheel`）与通用关系（`ex:partOf`），前者便于精确查询与校验，后者便于递归展开。要小心 `partOf` 的传递闭包：跨层级查询很方便，但在大图上代价高。

5. 计量与数量

金额、重量、长度、时长都不应该以裸数字存储，否则单位丢失、比较出错。数量模式把数值与单位绑定：

turtle

```
@prefix ex:  <https://example.com/ontology/order#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:Order-1001 ex:hasWeight [
    a               ex:Quantity ;
    ex:numericValue "12.5"^^xsd:decimal ;
    ex:unit         ex:Kilogram
] .
```

成熟做法是复用 QUDT 或 OM 这类单位本体，而不是自建单位表；至少要把单位建成受控个体，而不是字符串 `"kg"`。货币金额同理：金额 + 币种，或直接用带币种语义的金额类型。

6. 时间区间

只存一个时间点无法表达“价格在某个区间有效”“状态在某段时间内成立”。时间区间模式用区间节点承载起止时间，涉及历史数据时还要区分**有效时间**（业务上何时成立）与**记录时间**（系统何时写入）。

turtle

```
@prefix ex:  <https://example.com/ontology/pricing#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:Price-P1 ex:validDuring [
    a            ex:TimeInterval ;
    ex:startTime "2024-01-01T00:00:00+08:00"^^xsd:dateTime ;
    ex:endTime   "2024-12-31T23:59:59+08:00"^^xsd:dateTime
] .
```

也可以用 OWL-Time 的 `time:Interval` 与 `time:hasBeginning` / `time:hasEnd`。无论用哪种，不要用字符串存时间，也不要只存一个时间点。

7. 来源与溯源

数据来自哪个系统、经过什么加工、由谁负责，属于溯源信息。PROV-O 是 W3C 的溯源本体，核心是 Entity（实体）、Activity（活动）、Agent（代理）三要素：

turtle

```
@prefix ex:   <https://example.com/ontology/order#> .
@prefix prov: <http://www.w3.org/ns/prov#> .

ex:Order-1001 prov:wasGeneratedBy [
    a                    prov:Activity ;
```

```
    prov:wasAssociatedWith ex:SAPSystem ;
    prov:endedAtTime       "2024-04-30T09:00:00+08:00"^^<http://www.w3.org/2001/XMLSchema#dateTime>
] .
```

溯源信息不要等到“上线后再补”。数据治理、审计与问题定位都依赖它，而且后期补录成本极高。轻量做法是至少记录来源系统、导入批次与生成时间。

8. 值划分

值划分（value partition）把取值空间限定为一组互斥且穷尽的取值，既可以用互斥子类表达，也可以把取值建成个体枚举。典型如订单状态、支付方式、风险等级。它适合“取值有限、每个值有明确语义”的场景：

turtle

```
@prefix ex:  <https://example.com/ontology/order#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .

ex:OrderStatus a owl:Class ;
    owl:equivalentClass [ a owl:Class ;
                          owl:oneOf ( ex:Open ex:Paid ex:Shipped ex:Closed ) ] .

ex:Open    a ex:OrderStatus .
ex:Paid    a ex:OrderStatus .
ex:Shipped a ex:OrderStatus .
ex:Closed  a ex:OrderStatus .

ex:Open  owl:differentFrom ex:Paid , ex:Shipped , ex:Closed .
```

如果每个值还需要中文名、定义、同义词，用 SKOS 把每个值建成概念个体更合适；如果只需要表达“取值集合”，`owl:oneOf` 足够。值划分的校验（必须在集合内）交给 SHACL 的 `sh:in`，不要指望 OWL 报错。

9. 分类与本体

“分类”和“类型”经常被混用。类型是本体意义上的类，回答“它本质上是什么”；分类是给对象贴标签，回答“它属于哪个类目”。商品可以同时属于多个类目，类目体系会随运营调整，因此更适合用 SKOS 词表：

turtle

```
@prefix ex:   <https://example.com/ontology/product#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .

ex:Product-P7 ex:inCategory ex:Category-Electronics .

ex:Category-Electronics a skos:Concept ;
    skos:prefLabel "消费电子"@zh ;
    skos:broader   ex:Category-All .
```

如果每个类目都建成 OWL 类，会得到成百上千个类，推理变慢且没有语义收益。判断标准：类目需要逻辑定义与推理时用类，只是组织与检索时用 SKOS。

模式选用表

需求	推荐模式	关键结构
关系本身有属性	限定关系 / n 元关系	关系节点 + 参与方
记录发生了什么	事件	事件个体 + 参与者 + 时间
身份随时间变化	角色	角色节点 + 组织/时间
层次结构、BOM	部分-整体	`partOf` + 具体关系
金额、重量、时长	计量/数量	数量节点 + 单位
有效期、历史快照	时间区间	区间节点 + 起止
数据来自哪里	溯源	活动节点 + 来源系统
有限取值集合	值划分	枚举类或 SKOS
运营类目、标签	分类	SKOS 概念 + 归类关系

什么时候自造模式

现有模式目录不可能覆盖所有业务。出现以下情况可以自造：已有模式语义不匹配、组合后结构过于笨重、领域问题确实独特。自造时必须做三件事：写清适用场景与不适用场景、给出正反例、在团队内评审并纳入内部模式库。否则半年后没人知道这个结构为什么长这样。

与 DDD 模式的对照

熟悉 DDD 建模 的读者可以参考下表，但不要机械对齐：本体模式描述“数据与概念怎么表达”，DDD 模式描述“系统怎么组织与行为”。

DDD 概念	本体对应	注意
值对象	数量、时间区间、地址等结构化字面量或节点	本体里通常表示成有 IRI 或空白节点的结构
领域事件	事件模式	事件要不要持久化为个体，取决于是否需要历史查询
聚合	部分-整体 + 约束	聚合边界不等于本体层次，不要强行对齐
限界上下文	模块与命名空间	见 模块化与复用

常见坑

- **为模式而模式**：简单关系硬套限定关系，查询复杂度翻倍。
- **模式节点命名不统一**：`Participation`、`Relation`、`RoleLink` 混用，等于没有模式。
- **传递闭包滥用**：`partOf+` 在大图上很贵，要限定起点与深度。
- **单位/时间用字符串**：失去可比较、可推理的能力。
- **溯源后补**：上线后才发现无法回答数据来源问题。
- **把 ODP 当标准**：模式是经验总结，不是规范，仍需文档与评审。

模式的下一步是把它放进真实系统：先看 实现总览，再看 架构模式。

实现本体

建模阶段产出的是一份“约定”，实现阶段产出的才是“能力”。本页说明实现本体到底要交付什么、为什么建议给它单独一层、以及它和 DDD 的实现怎样配合。架构层面的选择见架构模式总览，工具与平台细节见技术栈与平台总览，模型元素怎么落到代码里见实现本体对象。

实现本体要交付哪些东西

很多团队说的“完成了本体”，其实就是几份 Protégé 里画出来的类图导出成 PDF。这只是建模阶段的部分产物。真正让本体在系统里发挥作用，至少要凑齐下面六件东西，缺哪一件，本体就退化成一样没人消费的静态资产。

交付物	典型形态	主要维护者	缺失后的后果
术语层（TBox）	Turtle/OWL 文件、IRI 命名规范、版本信息	知识工程师 + 领域专家	没有共享语义，各系统各说各话
实例数据（ABox）	导入管道、实例 IRI 策略、来源标注	数据工程师	本体只有骨架，查不出任何业务事实
查询层	命名好的 SPARQL 查询集、按用例封装的语义服务 API	后端工程师	模型无法被业务功能使用
校验层	SHACL 形状、发布前校验任务	数据治理 + 开发	数据质量靠人工看，错误数据入库
推理配置	选定的 profile 或规则集、物化策略与增量方案	架构师	推理由不同人各自为政，结果不可复现
治理机制	提案-评审-发布流程、CI 校验、变更记录	本体 Owner	模型一年后无人敢改

前两项属于“模型与数据”，后四项属于“工程与治理”。实践中失败的项目大多是只交付了第一项，第二到第六项从未规划。

判断项目是否真的实现了本体

不用看文档篇幅，看三个判据：

1. **有显式的概念体系**：类和关系的含义写在模型里，而不是散落在代码命名和口头共识中。
2. **有形式化表达**：机器可解析、可校验、可推理，不是纯文字说明。
3. **被系统使用**：生产路径上有查询、校验或推理在消费它。

三者缺一，就还停留在“写了个模型”。第三条最容易落空，也最值得在立项时就设计出来——至少要先有一个真实用例（一次检索、一次对账、一次发布前校验）跑通端到端。

为什么建议给语义留出独立的一层

把本体相关代码散落在业务模块里，短期看不出问题，长期有三个代价：

- **多消费方共享不了**：语义检索、规则校验、数据集成、报表口径往往需要同一份概念模型，散落各处就会出现四套略有差异的“事实标准”。
- **演化节奏冲突**：业务代码按周迭代，本体语义应保持稳定、按季度评审，混在一起会互相拖拽。
- **技能栈不同**：写 OWL/SHACL 与写领域代码是两类工作，混在同一个模块会让评审标准失效。

所以主流做法是设立一个**语义层**（也可叫知识层、语义服务层）：向上提供按用例命名的接口，向下屏蔽三元组库、推理器、外部数据源。DDD 的端口适配器同样适用于这一层，具体分层见分层与六边形架构。

实现与建模是双环，不是瀑布

实现过程中一定会发现模型问题，这属于正常反馈，而不是返工。有效的做法是让两个环快速转起来：建模出一个最小版本，立刻配样例数据和一个查询跑通，再回头改模型。

下面的最小闭环通常半天就能搭起来，比评审五十页文档更能暴露问题。

turtle

```
@prefix : <https://example.com/onto/order#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

:Order a owl:Class .
:DraftOrder rdfs:subClassOf :Order .

:order-1001 a :DraftOrder ;
    :placedBy :customer-42 .
```

sparql

```
PREFIX : <https://example.com/onto/order#>
SELECT ?order WHERE { ?order a :Order . }
```

如果第二条查询返回的 `:order-1001` 是靠 `DraftOrder ⊑ Order` 推出来的，说明类层次、实例数据、查询链路都已打通；如果返回空，问题可能出在数据没加载、命名空间写错或推理没开，而这三类问题都必须在实现早期暴露。接着补 SHACL 校验和一条“能不能查出待处理订单”的能力问题（CQ），闭环就完整了。CQ 的写法见能力问题。

技术选型的原则

选型不是选“最好的产品”，而是选“最不坏的一组约束”。按优先级：

1. **由问题倒推能力。**需要跨系统语义互操作、标准查询、可推理，才需要 RDF 栈；只是想要邻居遍历，属性图可能更省事；只是想要一张术语表，SKOS 加文档就够了。
2. **最小可运行优先。**先用嵌入式或单机三元组库（如 Jena TDB2、Oxigraph、RDF4J 内存库）把链路跑通，再决定要不要上集群。
3. **标准优先于产品。**Turtle、OWL、SHACL、SPARQL 是可迁移资产，专有查询语言和专用存储格式会形成锁定。
4. **把运维成本算进去。**图数据库的备份、升级、权限、监控都需要人力，小团队常低估这一块。

5. **与团队能力匹配。**团队里没人能读懂 OWL 公理时，复杂推理不要上生产。

存储的具体取舍见存储选型，推理能力见推理与物化。

与 DDD 的实现如何配合

本体和 DDD 不冲突，它们是两种不同中心：DDD 以**行为与一致性边界**为中心，本体以**共享语义与跨系统数据**为中心。落到实现上大致是这样分工的：

DDD 概念	本体侧的对应物	注意
聚合 / 聚合根	通常对应本体中的核心类与个体	聚合内不变量用代码或 SHACL 表达，不要用 OWL
仓储	语义层的查询/更新入口	仓储返回领域对象，语义层返回视图或查询结果
领域服务	语义服务（查询、推理、校验）	语义服务不是业务行为的替代品
领域事件	知识库变更的来源之一	异步投递即可，避免与业务库强一致双写
限界上下文	本体模块与命名空间划分	见模块化与复用

三条边界原则值得写进团队规范：

- **聚合不变量不要用 OWL 表达。**OWL 是开放世界假设，缺数据不会报错，只会推出更多结论；要“必须存在”就用 SHACL 或领域代码。这条与约束与业务规则一致。
- **不要在业务事务里双写。**写业务库成功、写知识库失败会造成长期不一致；用事件投递、批量同步或定期重建。
- **不要让业务代码直接拼 SPARQL。**查询应封装在语义层里按用例命名，便于优化、缓存和权限控制。

常见问题

本体是不是就是文档？ 不是，但现实中很容易变成文档。判据见上文“三个判据”：能被系统消费才算实现。

实现本体必须上图数据库吗？ 不必须。数据量小、查询简单时，内存三元组库或“关系库 + RDF 视图”完全够用；是否上图库取决于查询模式、推理需求和运维预算，见存储选型。

一定要有推理吗？ 不一定。推理解决的是“隐含结论能否自动得出”，很多场景只靠显式数据和 SPARQL 就够。是否开启推理见推理与物化。

和 DDD 项目二选一吗？ 不是。常见做法是业务系统继续用 DDD 管行为与一致性，另设语义层承载共享概念与跨系统查询，两者通过事件和查询接口松耦合。

实现本体的架构模式

本体落进系统后，会带来四个新问题：它在架构里放在哪、数据存到哪、结论怎么算出来、模型怎么演进。这四个问题各自沉淀出一些做法，本页把它们列在一起，给出“什么时候必须、什么时候可选”的判断依据，细节分别见四篇子页。需要先说明：这些模式不是必选项，也**不存在“上了本体就必须上图数据库”**这样的默认结论。

四个关注点与对应模式

关注点	模式	解决的核心问题	详见
边界与依赖	分层 + 端口适配器（六边形）	语义能力独立演化，业务代码不耦合具体图库	分层与六边形架构
数据落点	三元组库 / 属性图 / 关系库 / 混合	选择承载 TBox 与 ABox 的存储，并处理与业务库的关系	存储选型
结论计算	物化 vs 查询时推理、规则集	隐含结论由谁算、何时算、算多少	推理与物化
长期演进	版本策略、兼容规则、迁移	IRI 与语义稳定，数据与代码能平滑升级	版本与演进

什么时候必须、什么时候可选

不同模式的收益差距很大，判断时要回到具体问题，而不是架构风格偏好。

模式	必须的典型条件	可以省略的典型条件
独立语义层	两个以上系统/团队消费同一份模型；模型有独立评审与发布节奏	单应用内部的小词表，只有一处消费
端口适配器	需要替换存储或推理器；需要在 CI 中用内存库跑测试	一次性分析脚本，逻辑不长期维护
RDF 三元组库	需要标准 SPARQL、跨源合并、可推理、对外互操作	只做单表查询、无推理需求
独立推理服务	需要 DL 级推理（如用 ELK/HermiT 做可满足性检查）	只需 RDFS 级子类闭包，存储自带即可
物化	查询延迟要求高、推理结果稳定、写少读多	数据高频写入、结论变化快
版本与注册表	模型被多方依赖，跨系统共享	内部原型，一周内会推翻重做

判断口诀是：**看消费方数量、看是否需要标准与推理、看模型的生命周期长度。**三者都小的时候，最简方案通常是对的。

与 DDD 架构模式的对照

本体架构模式借用了 DDD 的很多思路，但两者的中心不同，直接照搬会出问题。

DDD 架构模式	本体侧的相似物	关键差异
四层架构	接口 / 应用 / 语义 / 适配器	语义层不承载业务行为，只承载概念、查询与校验
端口适配器	语义服务为端口，三元组库、推理器、外部 API 为适配器	端口粒度是“用例查询”而不是“聚合读写”
CQRS	查询走 SPARQL、写入走数据管道或 SPARQL Update	读模型常是物化推理图，写模型是原始事实，两者不必对称
事件溯源	本体版本历史、知识库增量更新	事件溯源以聚合为单位保序；本体演进的单位是图或模块，通常不做事件回放
仓储	语义层 Repository	仓储强调聚合一致性；语义查询往往是跨类、跨模块的图遍历
限界上下文	本体模块 + 命名空间	上下文边界靠 IRI 与 `owl:imports` 隔离，缺少事务边界

最重要的差异有两条：

- **一致性强度不同。**聚合不变量要求强一致、立即校验；本体更关注跨系统的语义一致，通常允许最终一致。把本体当“全局聚合”来保证强一致，会付出极高代价。
- **失败方式不同。**OWL 层面写错约束通常不会报错，只会推出奇怪的结论；所以本体侧的重点从“防御非法状态”转向“用 SHACL 显式校验 + 定期一致性检查”。

不推荐的做法

以下四类做法在项目里反复出现，代价高、收益低：

1. **把本体直接塞进业务聚合。**在实体里挂 RDF Model、在聚合方法里调 SPARQL，结果是领域模型被图库 API 污染，测试和重构都变难。
2. **无治理的全量推理。**上来就开 OWL 2 RL 或自定义规则全量物化，数据一更新就重算整图，延迟和存储迅速失控。
3. **业务库与知识库双写。**同一个事务里写两处，任何一边失败都会留下长期不一致，且没有补偿机制。
4. **把图数据库当万能库。**用图库承载高频事务写入、做报表聚合、当主数据主库，通常既慢又难运维。

一个够用的小型架构

如果消费方就一两个、数据量在单机可承受范围内，可以直接采用最简形态，等出现真实压力再拆：

text

```
[接口层 API]
     |
[语义服务]  ← 按用例命名的查询/校验接口
     |
[三元组库（嵌入式或单机）+ 可选 RDFS 推理]
     |
[Turtle / JSON-LD 文件 + 定时导入]
```

等价实现可以是：Jena TDB2 + Fuseki，或 RDF4J 内存库 + Spring 服务，或 Oxigraph + 一个 HTTP 接口。中间不需要消息队列、不需要独立推理服务、不需要图库集群。等出现下列信号再升级：查询变慢且无法靠索引解决、需要多副本高可用、需要复杂规则增量推理、消费方数量增长到需要权限隔离。

起步建议

1. 先确定**第一个真实用例**（一次检索、一次校验、一次对账），据此决定要不要三元组库。
2. 用最小组件把端到端链路跑通，验证模型能支撑该用例。
3. 再补版本与评审机制，而不是一开始就设计完备的架构。
4. 每引入一个模式都问一句：它替我们挡住了哪个具体风险？答不上来就暂时不加。

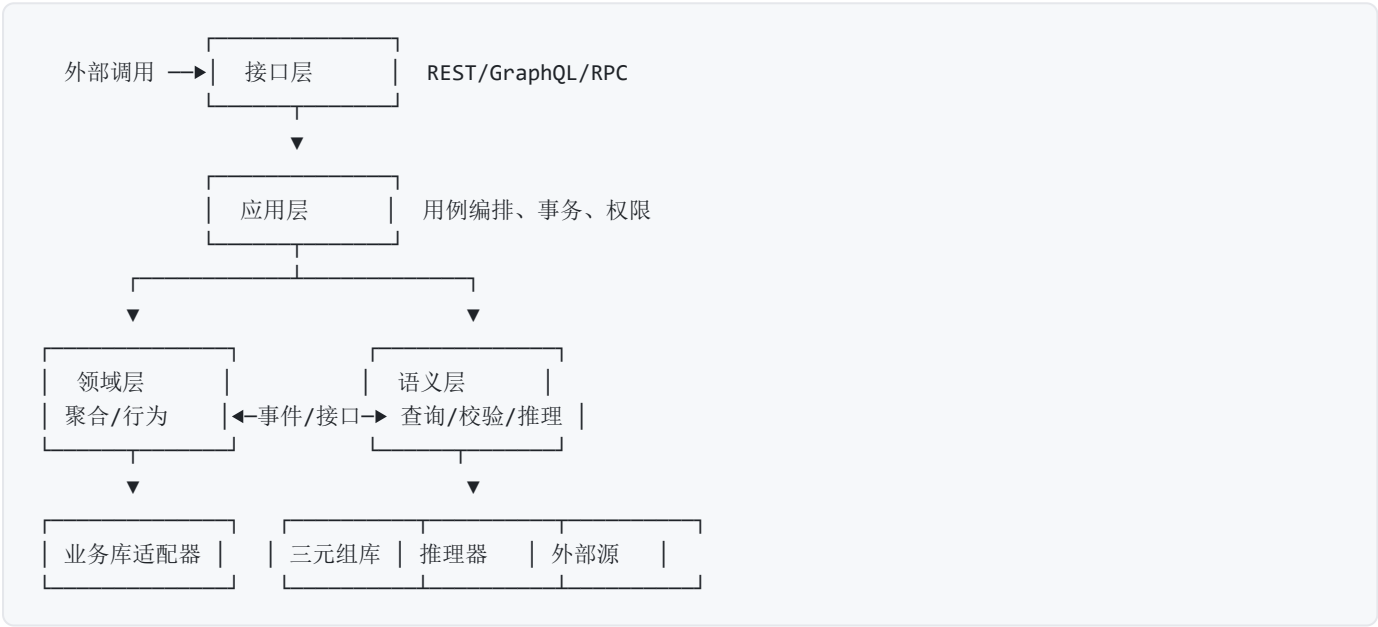
平台与库的选择见技术栈与平台总览，实现细节见实现本体对象。

分层与六边形架构

结论先给：把语义能力当成系统里独立的一层，上层通过**端口**使用它，三元组库、推理器、外部数据源都放到**适配器**里。这样本体可以独立演进，业务代码也不必知道底下是 Jena 还是 GraphDB。这套思路与 DDD 的端口适配器同源，只是端口的内容从“聚合读写”换成了“语义查询与校验”。存储怎么选见存储选型，推理怎么开见推理与物化。

语义层在系统中的位置

text



横向上，语义层和领域层是**并列的两个能力中心**，不是上下级。领域层回答“业务该怎么变”，语义层回答“这些名词共同指什么、跨系统怎么对上、隐含结论是什么”。把语义层塞进领域层内部，会让领域模型被 RDF API 与 SPARQL 字符串污染。

各层职责与禁止事项

层	职责	不应该做
接口层	协议转换、鉴权、参数校验	拼 SPARQL、直接访问三元组库
应用层	用例编排、调用语义与领域能力、事务与补偿	把模型语义判断写成 if-else
语义层	暴露按用例命名的查询/校验接口、屏蔽推理与存储细节	承载业务行为、直接返回裸 `Model`
适配器	三元组库读写、推理器调用、外部源导入、缓存	包含业务规则

六边形：端口和适配器怎么切

端口是**以问题语言命名**的接口，而不是对底层 API 的包装。下面这个端口只描述“系统需要知道什么”，不暴露任何 RDF 概念：

java

```
public interface OrderSemanticPort {
    List<OrderSummary> findOrdersByCustomer(CustomerId customerId);

    ValidationResult validateOrder(OrderId orderId);
}
```

适配器负责把它翻译成具体实现。同一个端口可以有两个适配器，测试时用内存实现，生产时用三元组库：

java

```
public final class SparqlOrderAdapter implements OrderSemanticPort {
    private final QueryExecutor queries;   // 封装 SPARQL 执行与超时
    private final ShapeValidator validator;

    @Override
    public List<OrderSummary> findOrdersByCustomer(CustomerId id) {
        var bindings = queries.select("order/by-customer.rq", Map.of("customer", id.iri()));
        return bindings.stream().map(OrderSummary::from).toList();
    }

    @Override
    public ValidationResult validateOrder(OrderId orderId) {
        return validator.validate(orderId.iri(), "order-shapes.ttl");
    }
}
```

`QueryExecutor` 这类东西本身也是端口，它的适配器可以是 Fuseki HTTP 客户端、Jena 本地 `QueryExecution`、RDF4J `RepositoryConnection`。查询文本放在 `queries/*.rq` 资源文件里，而不是散落在 Java 字符串里——这一点对可测试性和优化都很关键。

依赖方向的四条规则

1. 上层依赖语义层的接口，语义层不依赖上层。
2. 语义层不 `import` 任何具体图库或 HTTP 客户端（Jena、RDF4J、rdflib 都只出现在适配器里）。
3. 适配器之间互不依赖，交换数据走端口或显式 DTO。
4. 语义层返回的值对象是**为消费方设计的**，不是 RDF 的 `Model`、`Resource`、`Statement`。

第 4 条常被忽略。直接把 `Model` 抛给上层，等于让调用方自己写 SPARQL，过半年就没人说得清系统里到底有多少条查询、哪些是热点、哪些已经失效。

语义服务 API 怎么设计

按用例封装，一条用例对应一个方法或一个命名查询，避免“通用 SPARQL 端点”直接暴露给内部调用方。

- **方法名用业务语言**：`findOverdueOrders`，而不是 `runQuery("q-017")`。
- **输入是业务标识**，不是 IRI 字符串拼接；IRI 生成集中在 IRI 常量类里，见实现本体对象。
- **输出是视图对象**，字段明确、可序列化，顺带完成从 IRI 到业务 ID 的映射。
- **校验类接口返回结构化结果**（是否通过、违规项、严重级别、涉及节点），而不是一句布尔值。

- **读接口可以缓存**，写接口一律走数据管道，不要让业务代码直接发 SPARQL Update 改知识库。

如果消费方只有 BI 或数据科学团队，也可以保留一个受控的 SPARQL 端点，但要限制超时、并发、结果行数和可用图，必要时做查询白名单。开放端点的风险见SPARQL 查询与更新。

与 DDD 四层架构的对照

DDD 四层	本体侧对应	差异
用户接口层	不变	语义层通常只被内部调用
应用层	不变，负责编排两侧	一个用例可能既改聚合又查语义
领域层	领域层保持原样，另设语义层	语义层不是领域层的子集，两者并列
基础设施层	三元组库、推理器、导入任务作为适配器	语义层自身不含基础设施代码

六边形架构里的“核心”在 DDD 是领域模型，在本体语境里要拆成两个：**领域模型**（行为）和**本体**（语义）。二者都需要被保护，但保护方式不同——领域模型靠不依赖外部技术，本体靠版本化、可校验、可推理。

小型项目简化版

只有一个应用、数据量不大时，不必真的画出两个中心：

- 语义层做成一个包或一个模块，对外只有 3–5 个方法；
- 直接使用嵌入式三元组库（Jena TDB2、Oxigraph），省掉独立服务；
- SHACL 校验放在导入任务末尾，而不是每次查询都跑；
- 仍然坚持“查询文本外置 + 不返回裸 Model”这两条，它们是低成本高回报的纪律。

等到第二个消费方出现、或需要独立发布节奏时，再把模块提成服务，改动量并不大——前提是端口划分一开始就按用例做对了。

常见坑

- **语义层变成 SPARQL 转发器。**所有方法都只是 `select(sparql)`，等于没有抽象，只是多了一层文件。
- **把推理放进查询路径。**每条查询都触发推理，延迟不可控；应当把推理结果物化到独立命名图，见推理与物化。
- **事务边界混乱。**业务库与知识库各自开事务，缺补偿方案；应当明确知识库更新是最终一致。
- **适配器里写业务规则。**“这个客户算不算 VIP”出现在 SPARQL 里，等于法律条款写进了 SQL。

存储选型

存储选型是本体项目里代价最高、最难回退的决定。结论先给：**先判断是否需要 RDF 的语义互操作与标准查询，再决定产品**；不需要时，属性图或“关系库 + 只读语义视图”往往更省事。推理带来的存储放大见推理与物化，与业务库的协同方式见分层与六边形架构。

先回答三个问题

1. 数据是否需要与外部系统、行业标准共享语义，或者需要标准查询语言？需要，则 RDF 栈优先。
2. 是否需要推理和 SHACL 校验？需要，则 RDF 生态的工具链最完整。
3. 是否已有承载核心交易的关系库或图库？有，则多半应该做混合，而不是迁移。

三个都是“否”时，不要因为“本体”这个词就上图数据库。一份 Turtle 文件加一个嵌入式解析库，可能已经是正确方案。

四类候选

类别	代表	强项	弱项
RDF 三元组库	Apache Jena TDB2/Fuseki、RDF4J、GraphDB、Stardog、RDFox、Virtuoso、AllegroGraph、Amazon Neptune、Oxigraph	标准 SPARQL、命名图、推理与 SHACL 生态、跨源合并	学习曲线、运维成本、部分产品商业授权
属性图（LPG）	Neo4j 等	边带属性、路径查询直观、生态与人才多、事务成熟	无标准 schema/推理，跨系统语义互操作弱
关系库	PostgreSQL 等，部分商业库带 RDF 扩展	事务与运维成熟、团队熟悉	深层多跳查询别扭，语义表达靠表结构，易失真
混合	RDF 语义层 + 业务库 + 检索	各取所长，渐进落地	同步与一致性复杂，需要治理

可决策的对比

维度	RDF 三元组库	属性图	关系库	混合
数据模型	三元组 / 命名图，全局 IRI	节点与边，边可带属性	表与连接	各模型分工
查询语言	SPARQL 1.1（标准）	Cypher 等（非标准，生态各异）	SQL	SPARQL + SQL
模式与约束	OWL + SHACL（标准）	约束有限，schema 靠约定	DDL 约束完整	各自负责
推理	原生或插件支持（视产品）	基本靠应用层	基本靠应用层	语义层承担
深层多跳	强	强	弱（递归 CTE 可解但难维护）	强
事务	视产品，通常弱于 OLTP 库	较强	强	写路径靠业务库
运维门槛	中到高	中	低到中	高
锁定风险	低（标准可迁移）	中	低	中

几个产品特征值得知道（具体能力以官方文档与自测为准）：Jena 的 TDB2 适合单机/嵌入式持久化，配 Fuseki 提供端点；RDF4J 是 Java 侧另一套成熟框架与存储；GraphDB 提供规则集与推理；Stardog 支持虚拟图，可把关系库映射成 RDF 视图；RDFox 以内存 Datalog 推理和增量物化为特点；Virtuoso、AllegroGraph 常见于大规模部署；Amazon Neptune 同时提供 RDF 与属性图两种模型；Oxigraph 可作为嵌入式 Rust 库使用。Blazegraph 的社区维护状态需要自行确认后再上生产。

RDF 还是属性图

判定可以压缩成一句话：**要把“含义”共享出去，就用 RDF；只在自己系统里做关系分析，属性图通常更快落地。**

选 RDF 的典型信号：

- 需要与行业标准或第三方词表对齐（schema.org、DCAT、PROV-O 等）；
- 需要 SPARQL 联邦查询或跨机构数据交换；
- 需要 OWL 推理或 SHACL 校验；
- 数据模型的演进需要被显式记录和复用。

选属性图的典型信号：

- 查询以路径遍历和推荐为主，不需要推理；
- 边上有大量属性（权重、时间、来源），LPG 表达更自然；
- 团队已有 Neo4j 运维经验，数据不对外交换。

注意“图数据库 = 本体”是常见误解：属性图能存图，但不等于有形式化语义。Neo4j 的 n10s 可以导入 RDF，属于桥接手段，而非把 LPG 变成 RDF。两种范式的详细对比见RDF 数据模型与序列化。

混合架构怎么落地

text

```
业务库（关系库/LPG，强事务）
        │  变更事件 / 批量抽取
        ▼
语义落图（ETL 或虚拟图）
        │
        ▼
RDF 存储（TBox + ABox + 推理图）
        │
        ├─▶ SPARQL 语义服务
        ├─▶ SHACL 校验
        └─▶ 检索/向量索引（可选）
```

三条纪律决定混合架构能否维护：

- **单一写入口**：业务数据只在业务库被写，知识库由管道同步；不要在业务事务里双写。
- **可重建**：知识库必须能由源数据 + 映射规则全量重建，否则不敢重跑。
- **虚拟图优先于复制**：数据量不大或时效要求高时，用虚拟图/OBDA 方式（如 R2RML 映射、Stardog 虚拟图、Ontop 一类工具）按需查询关系库，能省掉一套同步链路。

数据加载与导出

常见做法是格式转换、批量导入、增量更新分开做：

bash

```
# 校验语法与 IRI，确认文件可被解析
riot --validate orders.ttl

# JSON-LD 转 Turtle，便于 diff 与评审
riot --output=TURTLE orders.jsonld > orders.ttl

# 离线全量导入 TDB2，适合初始化或重建
tdb2.tdbloader --loc=DB2 ontologies/orders.ttl data/orders.ttl

# 通过 SPARQL Update 做增量写入
curl -sS -X POST "http://localhost:3030/ds/update" \
  -H "Content-Type: application/sparql-update" \
  --data-binary @incremental.ru
```

增量脚本本身只是若干条更新语句：

sparql

```
PREFIX : <https://example.com/onto/order#>
INSERT DATA {
  :order-1002 a :Order ;
              :status :Paid ;
              :placedBy :customer-42 .
}
```

导出时优先用命名图分批导出，避免一次性 dump 影响服务：

bash

```
tdb2.tdbdump --loc=DB2 > backup.nq
```

CSV/JSON 源系统通常走 R2RML 或 RML 映射生成 RDF，再用上面的流程加载。映射规则要进版本库，和数据一起评审。

容量与运维要点

- **估算三元组量级**：行数 × 每行属性数，再乘以推理放大倍数（见推理页）得到物化后规模。
- **按命名图分区**：来源图、推理图、版本图分开，重算时只替换推理图，避免读到半成品。
- **备份策略**：原始图必须可恢复；推理结果可从原始图重算，不必单独备份，但要保留规则版本。
- **读写分离**：查询压力大时加只读副本（视产品支持），写路径保持单一。
- **监控指标**：查询 P95、慢查询 top N、导入失败率、推理重算耗时、图大小增长趋势。

常见坑

- **把图库当 OLTP 主库**：高频小事务写入、强事务保证，通常不是图库的强项。
- **业务库与知识库双写**：任何一侧失败都会留下长期不一致，且没人发现。
- **用空白节点标识业务实体**：空白节点无法被外部引用、无法跨图合并，业务实体必须用稳定 IRI。
- **命名图随意使用**：没有约定时，联邦查询和增量替换都会失控。
- **只选型不做回滚方案**：至少准备“导出为 N-Triples/Turtle 后迁到另一存储”的路径，标准格式是唯一的退路。

选型清单

1. 第一个用例需要的查询能否用 SQL/属性图更简单地满足？能，就先别上图库。
2. 需要推理吗？需要哪一级（RDFS/EL/RL/自定义规则）？产品是否原生支持？
3. 需要 SHACL 校验吗？在库内校验，还是应用侧用 pyshacl 一类库在入库前校验？
4. 数据量与增长预期？单机能否覆盖未来 12 个月？
5. 高可用、备份、权限、审计的要求是什么？运维团队能否承接？
6. 授权成本与锁定风险是否可接受？标准格式导出是否可行？
7. 与业务库的同步方式：事件、批量、还是虚拟图？

推理与物化

结论先给：**推理是可选能力，不是本体的必备件。**先确定业务是否真的需要“未显式写出的结论”，再决定用哪一级推理、在什么时机算。绝大多数项目只需要 RDFS 级闭包加少量规则，并且只把关键结论物化；一上来就开全量 OWL DL 推理，是失败率最高的做法之一。存储里怎么隔离推理结果见存储选型，公理怎么写得克制见公理。

推理能推出什么

推理的价值是把模型里的公理“展开”成显式事实，让查询和校验都能直接使用。常见能力如下：

能力	输入	推出的结论	来源
子类闭包	`:DraftOrder ⊑ :Order`、`:order-1 a :DraftOrder`	`:order-1 a :Order`	RDFS
类型推断	`:placedBy domain :Order`、`:order-1 :placedBy :c1`	`:order-1 a :Order`	RDFS
逆属性/对称	`:parentOf owl:inverseOf :childOf`	反向三元组	OWL
传递闭包	`:partOf a owl:TransitiveProperty`	多跳 `:partOf`	OWL
等价与同一性	`:Customer owl:equivalentClass :Buyer`	类型互换	OWL
不一致检测	不相交类同时被断言	本体的不可满足性	OWL DL
业务结论	金额、状态、计数等条件	风险等级、归类标记	自定义规则

注意 RDFS 的两条最容易踩：`rdfs:domain/range` 是**推理公理**，写错只会把实例推成错误的类型，不会报错；SHACL 才是校验。这是 OWL 与 SHACL 分工的核心，详见SHACL 形状约束。

先判断要不要推理

三个判据：

1. 结论数量有限、类型稳定、可以被枚举 → 直接物化成显式三元组，甚至不必引入推理引擎。
2. 需要检查本体自身是否自洽（类不可满足、个体矛盾）→ 用 OWL DL 推理器离线跑，建模期与 CI 里跑，不进在线查询路径。
3. 只是要保证“数据符合要求”→ 用 SHACL，不需要推理。校验和推理解决的是不同问题。

物化还是查询时推理

维度	物化（写时算）	查询时算
查询延迟	低，等价于普通三元组查询	高，随推理复杂度增长
存储成本	放大，闭包越大越明显	几乎为零
写入成本	高，每次变更都要更新闭包	低
增量难度	删除尤其难	无需维护
可解释性	可保留推导来源，便于审计	结论临时生成，难追溯
实现复杂度	需要重建、切换、一致性方案	依赖引擎与查询改写
适用场景	读多写少、结论稳定	数据高频变化、结论少

常见折中是**分级处理**：RDFS 闭包（子类、类型）物化，写少读多；DL 级一致性检查放到离线任务；易变结论用查询时规则或干脆用业务代码算。查询时推理在 SPARQL 里也能做一部分，例如用属性路径手工展开子类闭包：

sparql

```
PREFIX : <https://example.com/onto/order#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?order WHERE {
  ?order a ?type .
  ?type rdfs:subClassOf* :Order .
}
```

这条查询不依赖推理器，代价是每次都要遍历类层次，且无法覆盖 domain/range 之类的推断。

物化会膨胀多少

膨胀来自四个方向：每个个体对每个祖先类各加一条类型三元组；domain/range 推断出的类型；逆属性导致的边翻倍；传递闭包在长链上的组合爆炸（单链长度 n 的传递闭包是 $O(n^2)$ 条）。

估算方式很简单：先数原始三元组，再乘以“平均祖先深度”和“逆属性条数”，得到量级。类层次深、个体多的本体，物化后放大数倍到十几倍都属正常。控制手段：

- **限定 profile**：只开业务真正需要的规则子集，不要“全开以防万一”。
- **只物化关键谓词**：例如只对 `:Order` 相关类展开，其余按需查询。
- **隔离推理图**：推理结果写入独立命名图，查询默认合并，重算时只替换该图。
- **接受集合语义**：RDF 图本身去重，重复推导不会产生重复三元组，膨胀只来自真正新增的结论。

增量更新与一致性

写入不难，**删除才难**：删掉一条原始事实后，依赖它的推导结论需要撤销，而大多数引擎不会自动维护完整的支持关系。工程上的三种做法：

1. **标脏 + 定期重算**：变更记录到脏标记，按小时或按天从原始图重建推理图，重建期间旧图继续服务，完成后原子切换命名图。
2. **依赖增量推理的产品能力**：部分存储（如以增量物化为特点的 RDFox、GraphDB 的规则机制）能降低重算成本，但需要实测其删除语义与延迟。
3. **不物化易变结论**：把易变部分排除在闭包之外，从源头回避撤销问题。

无论哪种，都要记住：推理图必须**可由原始图 + 规则版本重算**。做不到这点，就等于埋了一个不敢碰的黑盒。规则文件进版本库，重建脚本进 CI。

引擎与各自边界

引擎	覆盖范围	运行方式	适用
Jena reasoner	RDFS、OWL 子集、自定义规则	Java 库内	应用内中小规模推理，规则灵活
ELK	OWL 2 EL	Java 库	大规模类层次的快速分类，以 TBox 分类为主
HermiT / Openllet	OWL 2 DL	Java 库	建模期一致性检查、类可满足性，ABox 大时慢
GraphDB 规则集	RDFS/OWL-Horst/OWL 2 QL/RL 及自定义规则	库内	生产查询可直接使用推导结果
RDFox	Datalog 规则、内存增量物化	独立服务/库	规则复杂、需要增量更新；内存预算与授权条款需先确认
Stardog	推理 + 虚拟图	商业平台	需要统一推理与数据虚拟化
owlready2	通过 HermiT 做推理	Python 库（需 Java）	数据科学侧探索、离线校验
SHACL-AF	规则与校验扩展	各实现不一	需要“校验 + 推导”混合时评估

两个必须说清的边界：ELK 不是通用 DL 推理器，含析取、复杂基数的本体它处理不了；owlready2 的 `sync_reasoner()` 默认调用 HermiT，需要本机 Java 环境，不能当作纯 Python 方案。SHACL-AF 是 W3C 工作组注记而非正式推荐标准，各产品支持程度不同，跨产品移植前必须验证。

规则设计原则

- **少而稳**：规则数量控制在能被团队讲清楚的规模。每条规则应能对应一句业务语言。
- **可命名、可版本化**：规则文件放版本库，命名体现意图（`high-value-order.rules`），而不是 `rule-07`。
- **可测试**：用 CONSTRUCT 查询或规则的单测断言期望输出，形成回归集。改规则前先跑通旧用例。
- **可解释**：物化结论最好能回答“为什么它是高风险”。做法是保留推导日志或来源标注（例如记录规则 ID 与输入三元组的哈希），必要时用 reification 或 RDF-star 表达来源。
- **不写易变策略**：促销门槛、审批阈值这类每月变的东西，不要固化进本体规则。

Jena 的自定义规则写起来很轻：

text

```
# rules/risk.rules
@prefix : <https://example.com/onto/order#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

[highValue:
  (?o rdf:type :Order), (?o :totalAmount ?a), greaterThan(?a, 10000)
  -> (?o :riskLevel :High)]
```

java

```
Model data = RDFDataMgr.loadModel("data/orders.ttl");
Reasoner reasoner = new GenericRuleReasoner(Rule.rulesFromURL("rules/risk.rules"));
InfModel inf = ModelFactory.createInfModel(reasoner, data);
```

GraphDB 一侧通常在仓库配置里选择规则集（如 `rdfs-plus`、`owl2-rl`），自定义规则以规则文件挂载；字段名与可用规则集请以所用版本文档为准，不要照抄博客里的旧配置。

推理与业务规则引擎怎么分工

维度	本体推理	业务规则引擎
表达对象	稳定的语义闭包（分类、传递、等价）	易变的业务策略（审批、风控、计费）
变更频率	低，走评审	高，业务方自行调整
可解释性	强，基于公理与规则	取决于引擎，需显式设计
实时性	通常离线/准实时	常在请求路径上
权威性	语义事实的唯一来源	决策策略的唯一来源

判定原则：**“这个结论是否无论业务策略怎么变都成立”** —— 是，放本体；否，放规则引擎或代码。两者同时使用时，务必明确谁是权威，避免同一结论两处计算、两处不一致。

性能与一致性注意

- 不要把推理塞进在线请求路径。请求内推理的延迟不可预测，尤其涉及传递闭包时。
- 物化更新与原始数据更新通常不在同一事务里，要接受“短暂读到旧闭包”，并用命名图切换减少窗口。
- 读写一致性按产品能力设计：有的存储推理在查询时生效，有的需要显式维护推理图，混用时会出现“我本地能查到、生产查不到”。
- 定期跑一致性检查（本体不可满足、SHACL 违规、推理图与原始图行数偏差），把它们做成监控指标。

版本与演进

结论先给：**本体是长期资产，IRI 的稳定性比“当前设计是否优雅”重要得多。**代码可以随时重构、重新部署，本体不能——它的 IRI 一旦被别处引用、被历史数据固化，改动的代价就会随时间指数增长。本页给出可执行的版本策略、兼容性判定表、弃用与迁移流程。评审与治理的组织视角见运营与生命周期，模块划分见模块化与复用。

为什么本体版本比代码版本难

- **IRI 是全局标识。**别处的数据、查询、映射规则都以它为锚点；改一次，所有引用方一起断。
- **数据是持久的。**代码升级失败可以回滚，历史三元组却会长期带着旧语义。
- **没有编译器帮你发现错误。**改一个 `rdfs:range` 不会让构建失败，只会让推理结果悄悄变错。
- **多处解释同一模型。**同一份本体可能同时被检索、校验、报表、对外交换使用，兼容性要按最保守的消费方判断。

因此版本管理的目标不是“记录版本号”，而是三件事：**语义可追溯、兼容可判定、迁移可回滚。**

命名空间与版本 IRI

有两种常见策略，选择时最重要的是别让**数据**随版本整体重写。

策略	形式	优点	代价
哈希命名空间（推荐默认）	`https://example.com/onto/order#Order`	类 IRI 长期稳定，升级不影响存量数据	版本信息只能靠元数据表达
路径版本	`https://example.com/onto/order/2.0/Order`	多版本可并行在线	每次大版本都要重写全部数据与查询
混合	稳定命名空间 + `owl:versionIRI` 指向版本 IRI	兼顾稳定与可追踪	需要纪律：发布时必须正确设置 versionIRI

推荐默认走混合策略：类与属性的 IRI 保持在稳定的哈希命名空间里，版本信息放在本体元数据上。

turtle

```
@prefix : <https://example.com/onto/order#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix dcterms: <http://purl.org/dc/terms/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

<https://example.com/onto/order> a owl:Ontology ;
    owl:versionIRI <https://example.com/onto/order/2.3.0> ;
    owl:versionInfo "2.3.0" ;
    dcterms:issued "2025-03-01"^^xsd:date ;
    dcterms:modified "2025-06-10"^^xsd:date ;
    rdfs:comment "订单领域本体" .
```

注意 OWL 2 要求 `owl:versionIRI` 与本体 IRI 不同；同一本体 IRI 下不同 versionIRI 表示同一术语体系的多个版本，这是标准里少见的、但很好用的一个区分。早期 OWL 1 的 `owl:priorVersion`、`owl:backwardCompatibleWith`、`owl:incompatibleWith` 等词汇在 OWL 2 中仍保留，但已标记为弃用（deprecated），新模型建议用 `owl:versionIRI` 表达版本关系，记录历史版本时用注释属性（如 `dcterms:replaces`、`prov:wasRevisionOf`）更稳妥。

语义化版本怎么套

用 `MAJOR.MINOR.PATCH` 是可行的，但要知道“兼容”在本体里有四层含义：逻辑兼容（新增公理不使旧结论失效）、数据兼容（旧实例仍可用）、查询兼容（旧 SPARQL 仍返回合理结果）、代码兼容（IRI 常量与生成的类不变）。判定兼容性必须按最弱的一层算。

变更	版本位	影响	处理动作
新增类、新增可选属性	MINOR	旧数据与查询不受影响	直接发布，检查与既有词是否语义重叠
新增注释、标签、`rdfs:isDefinedBy`	PATCH	无语义影响	直接发布
新增 SHACL 必填/范围约束	MAJOR	旧数据可能立即违规	先以警告级别上线，给修复期再提为违规
放宽约束、加可选的 `rdfs:subClassOf`	MINOR	结论变多，通常可接受	评估推理结果变化
新增等价类、属性链、传递属性	MAJOR 或至少大 MINOR	推理结果显著变化，可能引入不一致	需回归测试与一致性检查
修改 `rdfs:domain/range`	MAJOR	类型推断结果改变，可能推出错误类型	视为语义变更，写迁移说明
删除类或属性	MAJOR	旧数据成为“孤儿”，查询返回空	先弃用至少一个大版本，再删
收窄取值范围、改属性基数	MAJOR	旧数据可能不满足	同约束收紧
用同一 IRI 改语义	**禁止**	新旧含义混在同一标识上，无法区分	必须新建 IRI，旧 IRI 弃用

最后一条是红线。把 `:status` 从“订单状态”改成“支付状态”，即使拼写没变，也是新概念，必须换 IRI。这类事故往往在半年后才被发现，届时已经无法区分哪些数据属于哪种含义。

弃用而不是删除

弃用要给消费方留出迁移窗口，通常至少跨一个 MAJOR。

turtle

```
:customerName a owl:DatatypeProperty ;
    owl:deprecated true ;
    rdfs:comment "已弃用：请改用 :legalName；计划在 3.0 移除。" ;
```

```
    rdfs:isDefinedBy <https://example.com/onto/order> .

:legalName a owl:DatatypeProperty .
```

过渡期用派生保证旧查询还能跑：新属性为权威，旧属性由规则或一次性的 SPARQL Update 派生。

sparql

```
PREFIX : <https://example.com/onto/order#>
INSERT { ?s :customerName ?n }
WHERE  { ?s :legalName ?n .
         FILTER NOT EXISTS { ?s :customerName ?n } }
```

这条更新可以重复执行，结果一致（幂等），便于失败重试。派生期间要监控两边数量差异，差异归零后才可进入下一步。

数据迁移

推荐顺序是“先兼容、再切换、后清理”，而不是原地大改：

1. **兼容期**：新词上线、旧词弃用、双向派生或双写；查询仍读旧词。
2. **切换期**：代码与查询改为读新词，观察一个发布周期，确认无回归。
3. **清理期**：删掉旧词与派生逻辑，跑一次全量重建，回收空间。

批量改写 IRI 的更新语句要小步执行并保留快照：

sparql

```
PREFIX : <https://example.com/onto/order#>

DELETE { ?s :oldStatus ?v }
INSERT { ?s :status ?v }
WHERE  { ?s :oldStatus ?v }
```

几条纪律：

- **迁移前导出快照**（N-Triples/Turtle 均可），它是唯一可靠的退路。
- **迁移脚本必须幂等**，能被安全重跑；否则中途失败就只能人工对账。
- **优先全量重建**：如果知识库可由源数据 + 映射重建，就重建到新版本命名图，验证通过后切换，比原地改写安全得多。
- **按命名图分区版本**，可以并行保留新旧两个图，出问题一键切回。

本体注册表与评审

规模一旦超过一个团队，就需要轻量的注册表机制（可以就是一个 Git 仓库加目录约定）：

- **提案模板**：动机、影响的能力问题（CQ）、兼容性判定、迁移方案、回滚方式。CQ 的写法见能力问题。
- **CI 校验**：语法解析、推理器一致性检查、CQ 回归查询、示例数据的 SHACL 校验、与上一版的结构 diff。
- **发布记录**：versionIRI、变更摘要、弃用清单、依赖的本体版本。
- **消费方登记**：谁在用这份本体、能否接受 MAJOR 升级。没有登记机制就无法评估兼容性。

与代码和 API 版本的关系建议是“解耦但可映射”：API 走自己的版本号，本体走自己的版本号，在文档里明确对应关系（例如 API v3 依赖 ontology 2.x）。不要用本体 MAJOR 强行绑架 API 版本，也不要在代码里散落版本号字符串。

模块化降低影响面

把本体按模块拆开，每个模块独立命名空间与版本，只升级受影响的部分。`owl:imports` 应**固定到版本 IRI**，而不是指向“最新版”——否则上游一次发布就可能让下游全部失效。这条与模块化与复用里的实践清单一致。

变更清单

发布前逐条确认：

1. 是否新增或修改了公理？是否评估过推理结果变化？
2. 是否删除了被引用的类或属性？删除前是否已弃用一个版本？
3. 是否改了 `domain/range` 或约束强度？有没有通知下游？
4. `owl:versionIRI`、`owl:versionInfo`、日期是否已更新？
5. 迁移脚本是否幂等？是否有快照与回滚步骤？
6. CI 是否通过：解析、一致性、CQ、SHACL、diff？
7. 发布记录是否包含弃用清单与消费方影响？

常见坑

- **复用 IRI 改语义**：最严重，且最难事后修复。
- **只改模型不改数据**：模型升级了，历史三元组还带着旧词汇，查询开始返回空集。
- **命名空间里带版本**：每次升级都要重写全部数据，最终没人敢升级。
- `owl:imports` **指向 latest**：上游变更直接击穿下游，且无法复现历史行为。
- **没有 versionIRI 与变更记录**：出问题时无法回答“哪个版本引入了这条错误推理”。

实现本体对象

建模交付的是文件，实现交付的是代码。本体进入系统之后，就不再是 Protégé 里的一张图，而是一组需要在代码中被引用、加载、查询、校验和推理的资产。与 DDD 不同，本体没有“聚合、实体、值对象”式的代码范式可以直接照搬：我们不需要为每个 OWL 类生成一个业务类，也不应该把业务规则塞进 SPARQL。

本页的结论有三条。第一，TBox 与 ABox 必须在代码和部署上分开：本体文件当源码管理，实例数据当数据处理。第二，所有 IRI 必须集中管理，不允许字符串散落在业务代码里。第三，把查询和校验封装成用例级接口，SPARQL 与 Shape 作为受版本控制的资源文件，而不是内联字符串。

本体对象由哪几类代码承载

本体元素在代码里的形态，与它们在模型里的位置有关，而不是与面向对象的类层次一一对应。常见分工如下。

建模产物	代码或文件形态	归属	说明
类、属性、命名空间	IRI 常量类、枚举、命名空间对象	语义层	全项目唯一的 IRI 事实来源
TBox	`ontologies/**/*.ttl`	语义层	随应用发布，走代码评审
ABox	三元组库、数据文件、导入任务	数据层	不进应用包，按数据治理流程走
SHACL Shape	`shapes/**/*.ttl`	语义层	校验规则的唯一来源
能力问题查询	`queries/**/*.rq` 加参数化封装	语义层	查询是资产，不是临时字符串
推理配置	profile、规则集、物化任务	基础设施	决定推理由谁在何时执行
映射代码	Translator、Assembler、DAO	应用层/适配器	负责 RDF 与领域对象互转

其中“类与属性”这一类最容易被写坏。很多项目在业务代码里到处出现 `"https://example.com/onto/commerce#Order"` 这样的字面量，改命名空间时只能全局搜索替换，拼写错误又只有运行期才暴露。

目录组织：把 TBox 当源码管

推荐把本体文件放在源码仓库里，与 Java、Python 代码一起评审、一起发布。一个可维护的组织方式如下：

text

```
src/main/resources/
  ontologies/
    commerce/
      core.ttl          # 复用与对齐：schema.org、上层本体
      order.ttl         # 订单相关类与属性
      party.ttl         # 客户、组织
  shapes/
    order-shape.ttl
  queries/
    find-orders-by-customer.rq
```

这样做的直接收益是：本体的变更可以走代码评审与 CI（解析是否通过、是否一致、CQ 查询是否仍能返回结果），而不是靠人记得更新某台服务器上的文件。实例数据则相反，它由导入管道或图数据库持有，应用启动时只加载 TBox 和必要的参考数据。两者的生命周期不同：TBox 按季度演化，ABox 按小时或按天更新，混在一起发布会让每次数据变更都触发应用重新部署。

模块边界建议与限界上下文对齐：一个模块一个命名空间、一个目录、一组 `owl:imports`。边界划分与复用策略见模块化与复用。

IRI 常量：把字符串关进类里

集中管理 IRI 是所有后续工作的前提。Java 里可以这样写：

java

```java
import org.apache.jena.rdf.model.Property;
import org.apache.jena.rdf.model.Resource;
import org.apache.jena.rdf.model.ResourceFactory;

public final class CommerceVocab {
    public static final String NS = "https://example.com/onto/commerce#";

    public static final Resource ORDER = ResourceFactory.createResource(NS + "Order");
    public static final Property CUSTOMER = ResourceFactory.createProperty(NS + "customer");
    public static final Property AMOUNT = ResourceFactory.createProperty(NS + "amount");

    private CommerceVocab() {
    }
}
```

`Resource` 与 `Property` 是 Jena 的全局对象，不绑定具体模型，可以安全地作为常量复用。Python 的写法更轻：

python

```python
from rdflib import Namespace

COMMERCE = Namespace("https://example.com/onto/commerce#")
ORDER = COMMERCE.Order
AMOUNT = COMMERCE.amount
```

`COMMERCE.Order` 返回的是 `URIRef`，拼接、比较、作为查询参数都合法，比手写字符串安全得多。若由本体文件生成本地代码（codegen），请把生成物标记为“勿手改”，并在 CI 中校验生成物与本体文件一致；否则生成代码会慢慢偏离模型，成为第二套事实标准。

本体类不等于业务实体类

这是实现阶段最容易走偏的地方。OWL 类表达的是共享概念、分类与推理约束；业务实体类表达的是行为、生命周期和聚合不变量。DDD 侧可对照领域对象实现，两者的关注点不同，可以对应，但不必一一对应。

常见的两种极端都要避免。其一是“为本体每个类生成一个实体类”：生成代码没有行为，字段跟着模型变，业务逻辑无处安放。其二是“把本体当唯一模型”：用 OWL 表达订单状态机和聚合不变量，结果既难写又难测，还会被开放世界假设坑到。

推荐的做法是让领域模型与本体并行存在，中间加一层显式映射。本体承担“对外语义视图”和“跨系统契约”的角色，领域模型承担“系统内行为模型”的角色。映射关系大致如下。

本体元素	代码对应物	注意
类	值对象类型、枚举、实体类型标记	一个类可能对应多个实体类型
个体	聚合根或实体实例	身份由业务 ID 生成 IRI 承载
对象属性	关联与引用	不要退化成外键字段
数据属性	值对象字段	取值校验交给 SHACL
公理	不变量的一部分	一致性交推理器，完整性交 SHACL
Shape	服务层的校验器	输出结构化错误报告
模块	包或限界上下文	与命名空间一一对应

一个最小映射器长这样：

java

```
import org.apache.jena.rdf.model.Model;
import org.apache.jena.rdf.model.Resource;
import org.apache.jena.rdf.model.ResourceFactory;
import org.apache.jena.vocabulary.RDF;

public final class OrderTranslator {

    public static Resource toRdf(Order order, Model model) {
        Resource r = model.createResource(
                "https://example.com/data/order/" + order.id());
        r.addProperty(RDF.type, CommerceVocab.ORDER);
        r.addProperty(CommerceVocab.CUSTOMER,
                ResourceFactory.createResource(
                        "https://example.com/data/customer/" + order.customerId()));
        r.addProperty(CommerceVocab.AMOUNT,
                ResourceFactory.createTypedLiteral(order.amount()));
        return r;
    }
}
```

注意两点。第一，业务 ID 拼进 IRI 前要做转义或编码，否则含空格、斜杠、中文的 ID 会生成非法 IRI。第二，`order.amount()` 应当是 `BigDecimal` 而不是 `double`，否则金额的十进制语义会在序列化时丢失。

服务封装：仓储、查询服务、校验服务

上层代码不应该直接接触 `Model`、`Graph`、`QueryExecution`，更不应该自己拼 SPARQL。按用例封装成窄接口，是让本体可维护的关键一步。

java

```
public interface OrderSemanticRepository {
    void save(Order order);
    Optional<Order> findById(String orderId);
}

public interface OrderCompetencyQuery {
    List<OrderSummary> findOrdersByCustomer(String customerId, int limit);
}

public interface ValidationService {
    ValidationReport validate(Resource resource);
}
```

`OrderCompetencyQuery` 对应一条能力问题，而不是一张表。它的实现读取 `queries/find-orders-by-customer.rq`，通过参数绑定传入客户 IRI，再把结果集映射成 DTO 返回。

sparql

```
PREFIX c: <https://example.com/onto/commerce#>

SELECT ?order ?amount
WHERE {
  ?order a c:Order ;
         c:customer ?customer ;
         c:amount ?amount .
  FILTER (?customer = ?customerIri)
}
LIMIT 50
```

java

```
import java.nio.file.Files;
import java.nio.file.Path;
import org.apache.jena.query.*;

String queryText = Files.readString(
        Path.of("queries/find-orders-by-customer.rq"));

QuerySolutionMap binding = new QuerySolutionMap();
binding.add("customerIri", ResourceFactory.createResource(customerIri));

try (QueryExecution qe = QueryExecutionFactory.create(
        QueryFactory.create(queryText), model, binding)) {
    ResultSet rs = qe.execSelect();
    while (rs.hasNext()) {
        QuerySolution row = rs.next();
        // 映射为 OrderSummary，而不是把 ResultSet 抛给上层
    }
}
```

参数绑定除了可读性，更重要的是防注入：不要把用户输入拼进查询字符串。`LIMIT` 这类整型参数不能用变量占位，需要先校验为合法整数再拼入，或固定上限后由应用层截断。

Python 侧的查询服务同理，用 `initBindings` 传参：

python

```python
from rdflib import Graph, URIRef

graph = Graph().parse("data/commerce.ttl", format="turtle")
rows = graph.query(query_text, initBindings={
    "customerIri": URIRef(f"https://example.com/data/customer/{customer_id}")
})
```

推理与校验的代码位置

推理不要在每次请求里全量执行。常见做法是启动时加载 TBox 并预热，或在批处理任务中物化推断结果；请求路径只做查询。推理器与规则集的选择见推理与物化。

校验则相反，应该出现在写入口和发布流程里：数据导入前、本体发布前、关键写入前各跑一次，结果要落到可追溯的报告中（哪个 Shape、哪个节点、什么严重级别）。OWL 管一致性、SHACL 管完整性的分工见SHACL 形状约束与约束与业务规则。

反模式清单

- **IRI 字符串满天飞**：没有常量类，改命名空间时全局替换，错误只在运行期暴露。
- **业务逻辑写进 SPARQL**：查询会变成不可测、不可复用、无法评审的“第二套代码”；规则应放在 SHACL 或领域服务里。
- **把 `Model` 传遍全系统**：上层被迫理解 RDF 细节，换存储或换语言时全部重写。
- **每次请求重新解析 TTL**：本体文件读盘加解析的开销被忽略，延迟随文件变大而上升。
- **用 `rdfs:domain/range` 当校验**：开放世界假设下缺类型只会推出更多结论，不会报错。
- **图库与关系库双写**：没有补偿或事件机制时必然长期不一致。
- **把 TBox 塞进 ABox 图**：版本回滚与数据重建互相牵连，备份策略也无法分开。

落地顺序

1. 定命名空间与 IRI 规范，落成常量类或命名空间对象。
2. 建立 `ontologies/`、`shapes/`、`queries/` 三个受版本控制的目录。
3. 先封装一个查询服务和一次 SHACL 校验，跑通端到端。
4. 再加映射器与仓储，把 RDF 与领域对象隔开。
5. 最后接推理与治理流程，见分层与六边形架构。

技术栈具体到 JVM、Python、JavaScript 的库与依赖，见JVM 技术栈、Python 技术栈、JavaScript/TypeScript 技术栈。

技术栈与平台总览

本体落地没有“一站式框架”。一个能跑的系统通常由六类组件拼起来：建模编辑、存储与查询、推理、校验、应用接入、可视化。任何平台都只覆盖其中一部分，剩下的靠组合。因此选型的第一步不是比较产品，而是弄清我们的问题落在哪几类组件上。

三条结论先放在前面。第一，标准比产品重要：Turtle、OWL、SHACL、SPARQL 是可迁移资产，专有查询语言和私有存储格式会形成锁定。第二，先定推理与校验的位置，再选存储；很多项目反过来做，最后发现图库的推理能力用不上，校验还得自己补。第三，单机能解决的不要上集群，用一周以内的 PoC 拿真实查询与真实数据做判断。

一套本体系统由哪些组件组成

层	职责	代表工具
建模与编辑	编写类、属性、公理，跑推理测试	Protégé、WebProtégé、TopBraid EDG
存储与查询	持久化三元组，执行 SPARQL	Jena TDB2/Fuseki、RDF4J、GraphDB、Stardog、RDFox、Virtuoso、AllegroGraph、Amazon Neptune、Oxigraph
推理	RDFS、OWL profile、规则、物化	Jena 推理器、ELK、HermiT、GraphDB 规则、RDFox、Stardog
校验	SHACL 形状校验与报告	Jena SHACL、RDF4J SHACL、pyshacl、rdf-validate-shacl
应用接入	在业务代码里读写、查询	Apache Jena、RDF4J、rdflib、pyoxigraph、N3.js、rdflib.js
可视化	子图展示与探索	Cytoscape.js、vis-network、AntV G6

组件之间是标准接口，通常可以替换。真正难换的是“推理放在哪”：放在存储层（如 GraphDB 规则、RDFox）意味着性能好但受产品约束；放在应用层（Jena 推理器、OWL API）意味着灵活但要自己管物化与增量。

按语言与运行时选库

运行时	解析与模型	查询	校验	推理	适合
JVM	Apache Jena、RDF4J、OWL API	ARQ、RDF4J SPARQL	jena-shacl、rdf4j-shacl	HermiT、ELK、Jena 规则推理	服务端主力，生态最完整
Python	rdflib、pyoxigraph	rdflib SPARQL、SPARQLWrapper	pyshacl	owlready2（依赖 Java）、owlrl	ETL、数据科学、LLM 原型
JavaScript/TypeScript	N3.js、rdflib.js、oxigraph（WASM）	SPARQL 端点、Oxigraph	rdf-validate-shacl 等	浏览器内不做 DL 推理	前端展示、BFF、离线工具

JVM 与 Python 是两套主流选择：需要成熟服务端、完整推理器和企业级运维，选 JVM；需要快速试错、和数据管道及 LLM 生态贴合，选 Python。两者可以共存，用 SPARQL 或文件交换数据。细节见JVM 技术栈与Python 技术栈。

平台与三元组库对比

产品	形态	推理	校验	许可与部署	适合
Apache Jena + Fuseki + TDB2	开源库与服务	规则型 RDFS/OWL 子集，非完整 DL	Jena SHACL 库	Apache-2.0，自建自运维	通用服务端，成本可控
Eclipse RDF4J	开源 Java 框架	RDF4J 推理器与自定义规则	rdf4j-shacl	Eclipse 许可，可嵌入	已有 Java 应用内的语义模块
GraphDB	商业图数据库（有免费版）	规则推理能力较强	支持	商业许可	企业知识图谱、规则密集场景
Stardog	商业图数据库	推理与数据虚拟化	支持	商业许可	多源数据虚拟化加知识图谱
RDFox	商业内存图数据库	增量规则推理	支持	商业许可	高推理吞吐、规则实时更新
TopBraid EDG	商业建模治理平台	依赖其规则与推理组件	SHACL 支持较完整	商业许可	业务用户参与的本体治理
Virtuoso	开源版加商业版	以查询与规模见长，推理需搭配	需按版本确认	开源加商业	大规模 SPARQL 端点
AllegroGraph	商业图数据库	规则推理与图分析	需按版本确认	商业许可	图分析与语义应用
Amazon Neptune	托管图数据库	不做通用 OWL 推理	需外部组件	按云服务计费	已在使用 AWS 的托管场景
Oxigraph	开源 Rust 实现	无 OWL 推理	需外部组件	开源，可嵌入	边缘、测试、轻量服务
Neo4j + neosemantics	属性图数据库加 RDF 插件	无 OWL 推理，走属性图模型	需外部组件	社区版与商业版	已有属性图，只需 RDF 互操作

这张表只用于缩小范围。产品能力与版本强相关，最终以官方文档为准；尤其“校验”一列，商业产品的 SHACL 支持范围差异很大，PoC 时必须用真实的形状跑一遍。

编辑与治理工具相对独立：Protégé 适合单人建模型、跑推理检查；WebProtégé 适合多人协作评审；TopBraid EDG 这类平台把建模、校验、发布流程打包，代价是许可成本与学习曲线。小团队用“Protégé + Git + CI”就能获得大部分收益。

评估维度

维度	要回答的问题	判断提示
推理需求	只要 RDFS？还是 OWL EL、DL、规则？	决定 profile 与引擎，见推理与物化
校验需求	SHACL 是核心需求还是附带功能？	原生支持与外部库的运维成本不同
数据规模	多少三元组、每天多少更新、查询并发多大？	先测真实数据，不用官方基准
事务与一致性	写入是否需要 ACID、是否需要版本回滚？	图数据库的事务语义差异大
高可用	是否需要集群、备份、故障切换？	单机加备份常已够用
许可与成本	生产环境条款、节点/核数限制是什么？	把许可审计和续费算进总成本
生态与人才	客户端库、文档、社区、招聘难度如何？	冷门产品的长期维护风险高
运维	部署、监控、升级、备份是否有人负责？	图库不是装上就不用管
集成	与现有数据栈、身份认证、CI 如何对接？	认证与权限常是卡点
可迁移性	能否完整导出 RDF 与本体文件？	导出不全是隐性锁定

PoC 的做法：准备 5 条能力问题、2 个 SHACL 形状、一份真实规模的样例数据，在候选平台上各跑一遍，记录导入耗时、查询延迟、校验报告质量、推理结果和运维步骤。不要拿厂商基准数字做决策。

部署形态：嵌入式、独立服务与托管

同一个库往往支持多种部署形态，选错形态的代价比选错库更大。

嵌入式：把图存储放进应用进程，例如 RDF4J 的内存或原生存储、Jena 的 Model、Oxigraph 库。延迟最低，部署最简单，但容量受单机内存与磁盘限制，也无法让多个应用共享同一份图。适合测试、边缘设备、单应用的中小规模场景。

独立服务：Fuseki、GraphDB、Stardog 等以独立进程运行，通过 HTTP 暴露 SPARQL 端点。多个消费方共享一份数据，权限、配额、监控都有统一入口；代价是多一跳网络、多一套需要运维的服务，以及端点暴露带来的安全面。多数生产系统落在这个形态。

托管服务：如 Amazon Neptune 等云上产品，省去安装与备份，但受云平台、区域、计费方式和能力边界约束，推理与校验往往需要外挂组件。

判断方法很简单：只有一个应用独占数据、规模不大，嵌入即可；有多个消费方或需要统一权限，上独立服务；没有运维人力且已经在云上，考虑托管。无论哪种形态，都要保留把数据完整导出为 RDF、把本体文件留在自己仓库的能力。

常见组合

- **小团队与试点**：Protégé 建模，Jena + TDB2 + Fuseki 提供端点，pyshacl 或 Jena SHACL 做校验，应用通过 SPARQL 访问。成本最低，扩展路径清晰。
- **企业知识图谱**：GraphDB 或 Stardog 承担存储与推理，SHACL 做数据质量门禁，WebProtégé 协作评审，应用侧只面向语义服务接口。

- **高推理吞吐**：RDFox 这类内存增量推理引擎，适合规则多、更新频繁、查询要求低延迟的场景，代价是内存与许可成本。
- **数据科学与 LLM**：Python 侧用 rdflib 或 pyoxigraph 做 ETL 与查询，pyshacl 做校验，FastAPI 暴露查询服务；LLM 只负责生成候选查询或自然语言答案，不直接操作生产图。
- **前端展示**：N3.js、rdflib.js 解析与渲染子图，SPARQL 查询走自家 BFF；离线演示可用 Oxigraph 的 WASM 版本。

选型建议

先写清楚三件事：要回答哪些问题（能力问题）、数据从哪来、谁维护。三者清楚了，平台范围就缩小到两三个。

不要在没跑通一条端到端链路前采购商业许可。商业产品买的是支持、治理能力和规模，不是“更懂本体”。

推理与校验分开选。需要 DL 推理时，存储层的规则引擎往往不够；需要业务校验时，推理器也替代不了 SHACL。两者的边界见SHACL 形状约束。

把本体文件留在自己手里，把数据用标准格式导出。这样即使换平台，迁移成本也可控。版本与兼容策略见版本与演进。

常见问题

必须上图数据库吗？ 不必须。几千到几十万三元组、查询不复杂时，内存模型加文件即可；数据量、并发或治理要求上来后再换。存储取舍见存储选型。

开源够用吗？ 多数场景够用。Jena、RDF4J、Oxigraph、pyshacl 覆盖了服务端的主要能力。需要原生高可用、企业支持、图形化治理时再考虑商业产品。

关系数据库能存 RDF 吗？ 能，三元组表或 JSON 列都可以，但语义查询与推理要自己实现，容易变成重复造轮子。混合架构（关系库存业务数据、三元组库存语义视图）是常见折中。

团队只会 JavaScript，能落地吗？ 可以，但建议把重推理与全量数据放在服务端，前端只做子图展示与轻量校验，见JavaScript/TypeScript 技术栈。

怎么避免过度设计？ 从一条能力问题、一个形状、一份样例数据开始，跑通再扩。路线图见落地路线图。

JVM 技术栈

JVM 是本体工程中生态最完整的技术栈之一。解析、SPARQL、推理、SHACL 校验都有可直接用于生产的库，商业图数据库也大多以 Java 客户端为一等公民。代价是组件多、职责容易重叠，我们需要先分清“哪个库解决哪一段”，否则会出现 Jena 与 RDF4J 混用、两套模型互相转换的维护负担。

一条实用结论：需要做服务端、需要完整推理器、需要企业级运维时选 JVM；具体到库，通用场景优先 Apache Jena，需要 OWL 2 公理级操作与 DL 推理时用 OWL API 加 HermiT 或 ELK，已有 RDF4J 仓储抽象习惯的团队继续用 RDF4J 即可，不必迁移。

生态分工

库	定位	关键 API	典型用途
Apache Jena	RDF 模型、解析、SPARQL、存储、服务	`Model`、`RDFDataMgr`、`QueryExecution`、`TDB2Factory`、`Fuseki`	应用内建模与服务端主力
Eclipse RDF4J	仓储抽象、SPARQL、Sail 存储	`Repository`、`RepositoryConnection`、`SailRepository`	已有 Java 仓储风格的项目
OWL API	OWL 2 对象模型与公理操作	`OWLOntologyManager`、`OWLOntology`、`OWLReasoner`	程序化修改本体、DL 推理
ELK	OWL 2 EL 推理器	`ElkReasonerFactory`	大规模 EL profile 本体分类
HermiT	OWL 2 DL 推理器	OWL API 的 `OWLReasonerFactory` 实现	完整 DL 一致性检查
Jena SHACL	SHACL 校验	`ShaclValidator`、`Shapes`、`ValidationReport`	数据与本体发布前校验
RDF4J SHACL	事务内 SHACL 校验	`ShaclSail`	写入即校验

要特别注意：Jena 自带的 OWL 推理器是规则型实现，覆盖 RDFS 与 OWL 的一部分语义，不等价于完整的 OWL 2 DL 推理。需要严格的 DL 推理时，用 OWL API 加 HermiT 或 ELK，而不是指望 Jena 的 `OntModel`。

Maven 依赖

版本统一由属性或 BOM 管理，不要在业务模块里散落具体版本号。

xml

```
<!-- ${jena.version} 等版本属性由父 POM 的 properties 统一管理 -->
<dependency>
  <groupId>org.apache.jena</groupId>
  <artifactId>apache-jena-libs</artifactId>
  <version>${jena.version}</version>
  <type>pom</type>
```

```
</dependency>

<dependency>
  <groupId>org.apache.jena</groupId>
  <artifactId>jena-shacl</artifactId>
  <version>${jena.version}</version>
</dependency>
```

`apache-jena-libs` 是聚合 POM，一次引入 Jena 的主要模块。RDF4J 用 BOM 管理版本：

xml

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.eclipse.rdf4j</groupId>
      <artifactId>rdf4j-bom</artifactId>
      <version>${rdf4j.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

OWL API 的坐标为 `net.sourceforge.owlapi:owlapi-distribution`；ELK、HermiT 按所选推理器引入对应的 OWL API 绑定，具体 artifact 以各自项目文档为准。

读 Turtle、建模型、跑查询

java

```
import org.apache.jena.query.*;
import org.apache.jena.rdf.model.*;
import org.apache.jena.riot.RDFDataMgr;

Model model = ModelFactory.createDefaultModel();
RDFDataMgr.read(model, "ontologies/commerce-core.ttl");
RDFDataMgr.read(model, "data/orders.ttl");

String cq = """
        PREFIX c: <https://example.com/onto/commerce#>
        SELECT ?order ?amount
        WHERE {
          ?order a c:Order ;
                 c:amount ?amount ;
                 c:customer ?customer .
          FILTER (?customer = ?customerIri)
        }
        LIMIT 50
        """;

QuerySolutionMap binding = new QuerySolutionMap();
binding.add("customerIri",
        ResourceFactory.createResource("https://example.com/data/customer/42"));
```

```
try (QueryExecution qe = QueryExecutionFactory.create(
        QueryFactory.create(cq), model, binding)) {
    ResultSet rs = qe.execSelect();
    while (rs.hasNext()) {
        QuerySolution row = rs.next();
        System.out.println(row.getResource("order").getURI()
                + " " + row.getLiteral("amount").getLexicalForm());
    }
}
```

三个细节。`QueryExecution` 必须关闭，否则游标与连接会泄漏，用 try-with-resources。参数用 `QuerySolutionMap` 绑定，避免把外部输入拼进查询造成注入。文本块需要 Java 15 及以上，低版本改用普通字符串拼接。

TDB2 与事务

生产环境不要把大图全量读进 `Model`。TDB2 提供事务与磁盘存储：

java

```
import org.apache.jena.query.Dataset;
import org.apache.jena.query.ReadWrite;
import org.apache.jena.riot.RDFDataMgr;
import org.apache.jena.tdb2.TDB2Factory;

Dataset dataset = TDB2Factory.connectDataset("target/tdb2");

dataset.begin(ReadWrite.WRITE);
try {
    RDFDataMgr.read(dataset.getDefaultModel(), "data/orders.ttl");
    dataset.commit();
} catch (RuntimeException e) {
    dataset.abort();
    throw e;
} finally {
    dataset.end();
}
```

读事务同理，用 `ReadWrite.READ`。TDB2 的数据目录是运行时状态，不要提交到仓库；本体的 TBox 更新应当作为一次受控迁移执行，而不是每次启动都覆盖写入。

推理：先选对引擎

只需要 RDFS 级别的类型与子属性推断，Jena 的规则推理器足够：

java

```
import org.apache.jena.reasoner.Reasoner;
import org.apache.jena.reasoner.ReasonerRegistry;

Reasoner reasoner = ReasonerRegistry.getOWLReasoner();
InfModel inf = ModelFactory.createInfModel(reasoner, model);
```

这类推理是规则匹配，遇到复杂类表达式或大图时可能变慢甚至不终止，必须做规模测试。需要 OWL DL 时用 OWL API：

java

```java
import java.io.File;
import org.semanticweb.elk.owlapi.ElkReasonerFactory;
import org.semanticweb.owlapi.apibinding.OWLManager;
import org.semanticweb.owlapi.model.OWLOntology;
import org.semanticweb.owlapi.model.OWLOntologyManager;
import org.semanticweb.owlapi.reasoner.InferenceType;
import org.semanticweb.owlapi.reasoner.OWLReasoner;

OWLOntologyManager manager = OWLManager.createOWLOntologyManager();
OWLOntology ontology = manager.loadOntologyFromOntologyDocument(
        new File("ontologies/commerce-core.owl"));

OWLReasoner reasoner = new ElkReasonerFactory().createReasoner(ontology);
boolean consistent = reasoner.isConsistent();
reasoner.precomputeInferences(InferenceType.CLASS_HIERARCHY);
reasoner.dispose();
```

ELK 面向 OWL 2 EL profile，超出 profile 的公理不在其保证范围内；需要完整 DL 时换成 HermiT。两者都适合“发布前一致性检查 + 分类结果物化”，不适合放在每个 HTTP 请求里实时跑。推理时机与物化策略见推理与物化。

SHACL 校验

Jena 的 SHACL 模块用法直接：

java

```java
import org.apache.jena.graph.Graph;
import org.apache.jena.riot.RDFDataMgr;
import org.apache.jena.shacl.ShaclValidator;
import org.apache.jena.shacl.Shapes;
import org.apache.jena.shacl.ValidationReport;

Graph shapesGraph = RDFDataMgr.loadGraph("shapes/order-shape.ttl");
Graph dataGraph = RDFDataMgr.loadGraph("data/orders.ttl");

Shapes shapes = Shapes.parse(shapesGraph);
ValidationReport report = ShaclValidator.get().validate(shapes, dataGraph);

if (!report.conforms()) {
    report.getEntries().forEach(entry ->
            System.err.println(entry.focusNode() + " " + entry.resultMessage()));
}
```

如果需要“写入即校验”，RDF4J 的 `ShaclSail` 把校验放进事务：不合规的提交直接失败，省掉应用层的事后检查。代价是每次写入都付出校验开销，形状复杂时要评估吞吐。

java

```
import org.eclipse.rdf4j.repository.Repository;
import org.eclipse.rdf4j.repository.sail.SailRepository;
import org.eclipse.rdf4j.sail.memory.MemoryStore;
import org.eclipse.rdf4j.sail.shacl.ShaclSail;

ShaclSail shaclSail = new ShaclSail(new MemoryStore());
Repository repository = new SailRepository(shaclSail);
repository.init();
```

Fuseki 的边界

Fuseki 是快速暴露 SPARQL 端点的方式，但默认不带用户认证，SPARQL Update 端点一旦开放，任何能访问端口的人都能改数据。生产部署至少要做四件事：启用认证与按数据集的权限、只读查询与更新分离、配置查询超时与结果集上限、放在内网或网关之后。用 Fuseki 做公网开放端点，需要额外的限流与审计。

常见问题

要不要用 `OntModel`？ `OntModel` 在 `Model` 之上加了本体词汇与可选的推理封装，读写 `owl:Class`、`rdfs:subClassOf` 时更方便。代价是隐式推理开销与额外的类型转换，规模变大后不易控制。简单场景可以用，复杂链路建议直接用 `Model` 加显式推理器，把推理时机握在自己手里。

为什么推理出来的三元组查不到？ 推理结果存在于 `InfModel` 或 `OntModel` 这个视图上，不会自动写回底层 `Model`。查询必须对推理视图执行；如果希望持久化，需要把推断结果物化写回存储，并接受随之而来的膨胀与增量更新问题。

Jena 与 RDF4J 能只选一个吗？ 能，而且推荐。两者都提供模型、解析、SPARQL 与存储，能力高度重叠，混用只会增加依赖与转换成本。已有 RDF4J 仓储体系就继续用 RDF4J，新项目从 Jena 起步更常见。

实践注意

- **`Model` 可变且非线程安全。**每个请求或线程使用独立模型；共享数据用 TDB2 数据集的事务，不要在多线程间传同一个 `Model`。
- **推理器与 `InfModel` 非线程安全。**需要并发时每线程实例化，或把推理结果物化后只共享只读结果。
- **别把大图读进内存。** `RDFDataMgr.loadGraph` 适合样例与小文件；生产用 TDB2 或远端端点，并给查询设上限。
- **Jena 与 RDF4J 不要混用模型类型。**两套 `Model` / `Graph` 抽象不同，同一模块里混用会带来无谓的转换代码；选一套作为主模型，边界处显式转换。
- **序列化用标准格式。** `RDFDataMgr.write(out, model, RDFFormat.TURTLE)` 便于人工审阅与跨语言交换；N-Triples 更适合流式与差分。
- **依赖冲突要检查。**Jena、RDF4J、OWL API 都可能传递引入不同版本的 RDF 与 JSON 库，使用 `mvn dependency:tree` 定期排查。

与其他语言的协作

JVM 服务不必独占整个系统。常见形态是 JVM 侧持有图存储与推理，Python 侧做数据管道与 LLM 集成，前端通过 BFF 取子图。跨语言边界用 SPARQL 端点、RDF 文件或 JSON-LD 消息，不要在进程间共享 `Model` 对象。这样团队可以按技能分工，代价是多一层网络与序列化开销，需要在接口粒度上控制。

选型建议

先跑通“读 Turtle、一条 CQ 查询、一次 SHACL 校验”，再决定要不要推理以及用哪个引擎。通用服务端用 Jena；需要 OWL 公理操作与 DL 推理加 OWL API；已有 RDF4J 体系则保持不动。存储与端点的取舍见存储选型，与其他语言的协作见Python 技术栈。

Python 技术栈

Python 在本体工作里的优势很明确：解析、校验、数据管道、LLM 生态都在同一门语言里，试错成本低，几百行代码就能把“解析 → 查询 → 校验 → 服务”的闭环跑起来。短板同样明确：内存图不适合大规模数据，并发服务需要额外设计，DL 推理依赖本机 Java。

结论是分工使用：数据工程、原型验证、离线校验、语义检索与 LLM 集成放在 Python；生产级大图、高并发查询、重推理放到 JVM 或专用三元组库。两边用文件、SPARQL 端点或消息交换，不要把整个知识图谱塞进一个 Python 进程。

库的分工

库	定位	关键 API	注意
rdflib	纯 Python 的 RDF 模型、解析与 SPARQL	`Graph`、`Namespace`、`parse`、`query`、`update`	内存图，规模上千万三元组后吃力
pyoxigraph	Rust 内核的图存储与 SPARQL	`Store`、`load`、`query`	性能好于纯内存图，生态较新
pyshacl	SHACL 校验	`validate()`	支持先跑 RDFS/OWL RL 推理再校验
owlready2	OWL 2 本体操作与 DL 推理	`get_ontology`、`sync_reasoner`	推理依赖本机 Java
owlrl	RDFS 与 OWL RL 规则推理	`DeductiveClosure`	物化后数据会膨胀
SPARQLWrapper	调用远程 SPARQL 端点	`SPARQLWrapper`、`query`	注意超时、认证与查询注入

解析与 SPARQL

python

```python
from rdflib import Graph, Namespace, URIRef

COMMERCE = Namespace("https://example.com/onto/commerce#")

graph = Graph()
graph.parse("ontologies/commerce-core.ttl", format="turtle")
graph.parse("data/orders.ttl", format="turtle")

rows = graph.query(
    """
    PREFIX c: <https://example.com/onto/commerce#>
    SELECT ?order ?amount
    WHERE {
      ?order a c:Order ;
             c:amount ?amount ;
             c:customer ?customer .
      FILTER (?customer = ?customerIri)
    }
```

```
    """,
    initBindings={
        "customerIri": URIRef("https://example.com/data/customer/42")
    },
)

for row in rows:
    print(row.order, row.amount)
```

`initBindings` 是 rdflib 的参数化机制，等价于把变量预先绑定到 IRI 或字面量，比字符串拼接安全。查询结果按变量名取列，取到的仍是 RDF 术语对象，需要 `str()` 转成业务类型。

`Graph.parse` 按扩展名或 `format` 参数选择解析器；一次加载多个文件时注意基础 IRI，否则相对 IRI 会按错误基准解析。序列化用 `graph.serialize(format="turtle")`，输出前建议统一排序，便于版本对比。

SHACL 校验

pyshacl 的调用很直接，返回三元组：是否合规、结果图、文本报告。

python

```
from pyshacl import validate

conforms, report_graph, report_text = validate(
    graph,
    shacl_graph="shapes/order-shape.ttl",
    inference="rdfs",
    abort_on_first=False,
)

if not conforms:
    raise ValueError(report_text)
```

`inference` 参数决定校验前是否先做推理：`"rdfs"` 补子类与子属性，`"owlrl"` 跑 OWL RL 规则，`"both"` 两者都跑。很多“形状明明写了却不生效”的问题，根源是数据里只有子类实例、没有显式类型，而校验前没开推理。反过来，推理也会让本来因缺类型而不合规的数据变得合规，是否开启取决于业务语义，需要写进校验规范。

`report_graph` 是标准 RDF 结果图，可以入库、做看板或与 CI 集成；`report_text` 适合直接进日志。生产校验建议输出结构化的字段（严重级别、焦点节点、约束、路径），而不是只抛一个布尔值。

pyoxigraph：比 rdflib 更快的内存与磁盘图

python

```
from pyoxigraph import RdfFormat, Store

store = Store("target/oxigraph")
store.load(path="ontologies/commerce-core.ttl", format=RdfFormat.TURTLE)
store.load(path="data/orders.ttl", format=RdfFormat.TURTLE)

for solution in store.query(
    """
```

```
    PREFIX c: <https://example.com/onto/commerce#>
    SELECT ?order ?amount WHERE {
      ?order a c:Order ; c:amount ?amount .
    }
    LIMIT 50
    """
):
    print(solution["order"], solution["amount"])
```

pyoxigraph 的核心由 Rust 实现，适合需要比 rdflib 更快加载与查询、又不想引入 JVM 的场景。它的 API 面积比 rdflib 小，缺少部分 Python 侧的便利工具，团队要接受这一点。

推理：owlready2 与它的前提

python

```
from owlready2 import get_ontology, sync_reasoner

onto = get_ontology("file://ontologies/commerce-core.owl").load()
sync_reasoner()  # 推理器由 Java 实现，需要本机安装 JRE
```

owlready2 提供接近面向对象的本体操作方式，用 Python 类映射 OWL 类，适合做本体探索与规则验证。它的推理走 Java 的 HermiT 或 Pellet，部署时必须装 JRE，容器镜像要相应变大；没有 Java 时推理会直接失败。

如果只需要 RDFS 或 OWL RL 级别的规则推断，`owlrl` 加 `DeductiveClosure` 可以纯 Python 完成，代价是物化结果与原图混在一起，查询时需要区分“显式”与“推断”数据。OWL DL 的完整推理不在纯 Python 能力范围内，见JVM 技术栈。

用 FastAPI 暴露查询服务

python

```
from fastapi import FastAPI, Query
from rdflib import Graph, URIRef

app = FastAPI()

graph = Graph()
graph.parse("data/commerce.ttl", format="turtle")

QUERY = """
PREFIX c: <https://example.com/onto/commerce#>
SELECT ?order ?amount
WHERE {
  ?order a c:Order ;
         c:customer ?customer ;
         c:amount ?amount .
  FILTER (?customer = ?customerIri)
}
"""


@app.get("/customers/{customer_id}/orders")
```

```
def list_orders(customer_id: str, limit: int = Query(default=20, le=100)):
    rows = graph.query(
        QUERY,
        initBindings={
            "customerIri": URIRef(
                f"https://example.com/data/customer/{customer_id}"
            )
        },
    )
    return [
        {"order": str(row.order), "amount": str(row.amount)}
        for row in rows
    ][:limit]
```

这是一个最小可用的语义查询接口，生产化还要补四件事：图加载放到启动阶段并做只读保护；多进程部署时每个 worker 各自持有图，更新要用版本化文件或外部存储；查询加超时与结果上限；把 SPARQL 文本放在资源文件里而不是函数内。

FastAPI 的同步端点会在线程池里执行，rdflib 的并发读取通常没问题，但并发写同一个 `Graph` 不安全。需要更新时就换成“重新加载并原子替换”，或把写入交给独立服务。

pandas 与批量导入

python

```
import pandas as pd
from rdflib import Graph, Literal, Namespace, URIRef
from rdflib.namespace import RDF, XSD

df = pd.read_csv("data/orders.csv")
graph = Graph()
COMMERCE = Namespace("https://example.com/onto/commerce#")

for row in df.itertuples(index=False):
    order = URIRef(f"https://example.com/data/order/{row.order_id}")
    graph.add((order, RDF.type, COMMERCE.Order))
    graph.add((order, COMMERCE.amount,
               Literal(str(row.amount), datatype=XSD.decimal)))
```

两个坑。第一，数值不要直接用浮点数构造字面量，`str(row.amount)` 或 `Decimal` 才能保住十进制语义。第二，业务 ID 进 IRI 前要校验字符集，含空格或斜杠的 ID 必须编码，否则生成非法 IRI。

大批量导入时逐条 `graph.add` 很慢，可以先构建 N-Triples 流再一次性 `parse`，或用 pyoxigraph 的批量加载接口。

与 LLM 集成：本体站在哪一层

本体在检索增强里通常承担三件事：术语与实体的规范化、结构化召回、答案约束。一个务实的做法是把“检索单元”从纯文本块换成“本体术语加子图”：

python

```
def retrieve_context(question: str, graph, top_k: int = 5):
    terms = search_ontology_terms(question, top_k)  # 向量或关键词检索
    triples = []
    for term in terms:
        iri = term["iri"]
        for predicate, obj in graph.predicate_objects(iri):
            triples.append((iri, predicate, obj))
    return terms, triples
```

其中 `search_ontology_terms` 可以用你现有的向量库实现，索引对象是类名、标签、注释、同义词（`skos:altLabel`），而不是整篇文档。取回结果后，把“命中术语 + 邻居三元组”作为上下文交给模型，并要求答案引用 IRI，便于审计。

接 LangChain 或 LlamaIndex 时，实现一个自定义 retriever 即可：把上面的函数包装成 `BaseRetriever` 的 `_get_relevant_documents`，返回 `Document(page_content=..., metadata={"iri": ...})`。框架 API 会变，关键是不要把“检索到一段文本”当成目标，而是“检索到本体里的概念与关系”。

LLM 生成 SPARQL 要极其谨慎：只允许在只读端点、限定前缀、限制超时与结果条数，生成的查询先做语法校验，再由模板兜底。不要让模型直接生成 `INSERT` 或 `DELETE`。架构讨论见语义检索与 RAG。

实践注意

- **规模**：rdflib 适合几十万到百万级三元组；再往上优先 pyoxigraph 或远程端点，别硬扛。
- **并发**：`Graph` 的写操作非线程安全；多 worker 部署要明确“每个进程一份只读图 + 版本化更新”的模型。
- **Java 依赖**：owlready2 推理需要 JRE，CI 与容器镜像要显式安装，否则只在生产环境暴露问题。
- **推理边界**：owlrl 与 pyshacl 的预推理都是规则子集，不等价于 OWL DL；不要用它们判断本体的一致性。
- **SPARQL 支持度**：rdflib 的查询引擎覆盖常用语法，但与三元组库相比在联邦查询、部分函数和扩展上有限，复杂查询放端点执行。
- **安全**：所有对外查询走参数绑定，端点加认证与限流；LLM 生成的查询必须经过白名单与语法检查。

选型建议

做数据管道、离线校验、RAG 原型，用 rdflib 加 pyshacl 起步，需要更快时换 pyoxigraph；本体需要 DL 推理时，接受 Java 依赖或用远程 JVM 服务。对外服务不要直接用 FastAPI 包一个内存图面向公网，至少加一层缓存与查询约束。服务端选型的整体比较见技术栈与平台总览。

JavaScript/TypeScript 技术栈

JS/TS 在本体系统里的定位是“接入与呈现”，不是“存储与推理”。前端和 Node 能解析 RDF、调用 SPARQL、渲染子图、做轻量校验，但全量知识图谱和 OWL DL 推理不应放在浏览器里。选型时先把这条边界划清楚，后面的库选择会简单很多。

一个务实的分工：SPARQL 查询与推理留在服务端，前端只请求按用例裁剪过的子图；需要离线或边缘能力时，用 Oxigraph 的 WASM 版本在本地装一小份图；Node 侧适合做语义 BFF，把多个端点和多个数据源聚合成前端友好的 JSON。

生态分工

库/工具	运行环境	能做什么	不适合
N3.js	Node 与浏览器	Turtle/TriG/N-Triples 解析、RDF/JS 数据集、序列化	不提供 SPARQL 引擎
rdflib.js	浏览器为主	前端图存储、解析、增量同步、SPARQL 客户端	大数据量与复杂查询
oxigraph（WASM）	浏览器与 Node	嵌入式 SPARQL 1.1 查询与存储	全量图、高并发
rdf-validate-shacl 等	Node 与浏览器	基于 RDF/JS 的 SHACL 校验	缺少预推理时语义不完整
jsonld	Node 与浏览器	JSON-LD 的 expand、compact、转 RDF	不做语义校验
Cytoscape.js、vis-network、AntV G6	浏览器	图可视化、布局、交互	不做数据建模

N3.js：解析、模式查询与序列化

N3.js 是 RDF/JS 规范的常用实现，适合在 Node 或浏览器里做解析与轻量数据处理。

ts

```ts
import { DataFactory, Parser, Store, Writer } from 'n3';

const turtle = `
@prefix c: <https://example.com/onto/commerce#> .
<https://example.com/data/order/1001> a c:Order ;
    c:amount "99.00"^^<http://www.w3.org/2001/XMLSchema#decimal> .
`;

const parser = new Parser({ format: 'text/turtle' });
const quads = parser.parse(turtle);

const store = new Store(quads);
```

```
const orderType = DataFactory.namedNode(
  'https://example.com/onto/commerce#Order',
);
const rdfType = DataFactory.namedNode(
  'http://www.w3.org/1999/02/22-rdf-syntax-ns#type',
);

const orders = store.getQuads(null, rdfType, orderType, null);
console.log(`订单数：${orders.length}`);

const writer = new Writer({ format: 'text/turtle' });
writer.addQuads(store.getQuads(null, null, null, null));
writer.end((error, result) => {
  if (error) throw error;
  console.log(result);
});
```

`Store.getQuads` 按四个位置做模式匹配，`null` 表示通配，这是前端做局部查询最常用的方式。注意 N3.js 做的是“按模式找三元组”，不是 SPARQL；需要联表、聚合、属性路径时，要么写多步遍历，要么把查询交给端点。

解析大文件时不要在主线程一次性处理，用 `Parser` 的流式写法配 Web Worker，避免页面卡顿。

调用 SPARQL 端点

最常见的接入方式是通过 HTTP 调服务端端点，拿 SPARQL 结果 JSON 再渲染。

ts

```
const endpoint = 'https://example.com/sparql';

async function findOrders(customerIri: string) {
  const query = `
    PREFIX c: <https://example.com/onto/commerce#>
    SELECT ?order ?amount
    WHERE {
      ?order a c:Order ;
             c:customer ?customer ;
             c:amount ?amount .
      FILTER (?customer = <${customerIri}>)
    }
    LIMIT 50
  `;

  const response = await fetch(endpoint, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/sparql-query',
      Accept: 'application/sparql-results+json',
    },
    body: query,
  });

  if (!response.ok) {
    throw new Error(`SPARQL 请求失败：${response.status}`);
  }
```

```
  const { results } = await response.json();
  return results.bindings.map((row: any) => ({
    order: row.order.value,
    amount: row.amount.value,
  }));
}
```

两个安全点。第一，不要把 IRI 直接拼进查询字符串：这里用模板插值只为示例，生产代码应当由后端接收业务 ID 并完成绑定，前端只传 ID，不传任意 IRI。第二，浏览器不应直连生产端点：端点需要处理 CORS 与认证，通常是前端调自家 BFF，由 BFF 携带凭据访问端点并限制查询模板。

浏览器内的图存储：Oxigraph WASM 与 rdflib.js

Oxigraph 的 JS/WASM 包把 Rust 实现的 SPARQL 引擎带进浏览器或 Node，可以在本地装载术语表或子图并执行查询。它适合离线演示、文档站内嵌查询、以及“数据量不大但要真 SPARQL”的场景。代价是 WASM 包体积与初始化时间，需要按需加载，且不要指望它承载全量知识图谱。

rdflib.js 面向浏览器场景，提供前端图存储、解析、与远端文档的增量同步，以及 SPARQL 客户端能力。它适合“页面里维护一小份 RDF 状态”的应用，例如表单草稿、实体卡片、离线编辑。选它之前先确认版本与 API，前端库的迭代速度通常快于 JVM 生态。

无论用哪个，原则一致：浏览器只持有当前用例需要的子图。全量图放在服务端，按需拉取邻居，用缓存与分页控制内存。

SHACL 与 JSON-LD

SHACL 校验在前端通常只做即时反馈：用户填表时对已加载的形状做局部校验，提示缺字段或格式错误。但局部数据通过不代表全局合规，真正的校验必须由服务端在完整数据上执行。形状文件由服务端统一发布，前端不要各自维护一份，否则容易出现两套规则。SHACL 的语义见SHACL 形状约束。

JSON-LD 是前端最友好的 RDF 序列化形式。用 `@context` 把 IRI 映射成短名，接口返回的 JSON 既可读又能直接转成三元组。常见做法是：后端用 JSON-LD 输出，前端用 `jsonld` 库做 `expand` / `compact`，需要三元组时转成 N-Quads 交给 N3.js。注意 `@context` 本身也要版本化，改了上下文就等于改了数据契约。

可视化：从三元组到图视图

可视化库只负责画图，不负责理解 RDF。渲染前要做一次“视图模型转换”：把三元组聚合成节点与边，隐藏 `rdf:type` 这类结构性谓词，把类型映射成节点形状或颜色，把对象属性映射成边，数据属性放进节点详情面板。

- Cytoscape.js：图数据结构与布局算法完整，交互与扩展生态成熟，适合中等规模的业务图谱。
- vis-network：上手快、配置简单，适合内部工具与演示。
- AntV G6：布局、交互与组件体系较完整，适合产品化的图谱前端。

三条实践原则。按能力问题取子图，一次渲染几十到几百个节点，而不是全量。节点用 IRI 做稳定 key，避免每次刷新后状态丢失。点击节点再按需拉邻居，做渐进展开；布局计算放到 Worker 里，避免大图阻塞主线程。

边界与性能

- **内存**：浏览器里几十万条三元组就开始吃紧，WASM 图建议只装子图或术语表。
- **主线程**：解析大文件、跑布局都会阻塞 UI，用 Web Worker 分担。
- **查询**：前端只发模板化查询，不接收用户自由输入的 SPARQL；结果做字段投影与分页。
- **推理**：不在浏览器跑 DL 推理；即使是 RDFS 传递闭包，也要先估算数据量。
- **认证与 CORS**：直连端点需要 CORS 配置与令牌方案，更常见的做法是经自家 BFF 代理。
- **缓存**：按版本或 ETag 缓存图数据与形状文件，避免每次进入页面重新下载。

适用场景

适合：知识图谱浏览与探索、语义搜索的实体卡片与关系路径展示、录入表单的即时校验反馈、静态文档站内嵌可查询的小型本体、Node 侧聚合多个语义端点的 BFF。

不适合：全量知识图谱存储、企业级推理、多用户并发治理、需要复杂 SPARQL 的分析任务。这些交给JVM 技术栈或Python 技术栈侧的服务。

推荐组合

- **N3.js 加 fetch 加 Cytoscape.js**：解析、查询、渲染三件事各自用最合适的库，是前端知识图谱最常见的组合。
- **rdflib.js 加自家 BFF**：前端维护局部 RDF 状态，BFF 负责认证、模板查询与结果裁剪。
- **Oxigraph WASM 加静态数据**：离线演示与文档站内嵌查询，数据量控制在子图规模。

架构上如何把前端与语义服务隔开，见分层与六边形架构；知识图谱的整体构建流程见知识图谱构建。

应用总览

本体的价值不在建模本身，而在模型被系统稳定使用。本页给出本体在工程中的常见应用地图、判断“值不值得做”的方法、与 DDD 的分工，以及如何选出第一个落点场景。每个场景的展开见各子页。

本体在系统里解决什么问题

把本体放进系统，通常是为了解决下面四类问题中的一类或几类。判断一个需求是否真需要本体，先看它命中了哪一类。

问题类型	典型症状	本体的作用	无本体的替代方案
语义歧义	同一个词在不同系统里含义不同，对账总对不上	用显式类和属性固定含义，靠 IRI 全局标识	人工维护字典、逐系统硬编码映射
数据异构	同一实体有几十种字段命名和结构	统一到共享模式，映射一次、复用多次	点对点 ETL，系统数为 n 时映射数为 n^2
需要推理	结论由多条事实组合得出，规则分散在代码里	用公理和规则推导隐含事实，可解释	在业务代码里写分支判断
需要互操作	与外部伙伴、行业标准、开放数据交换	复用标准词表（schema.org、FHIR、DCAT 等）	自定义 JSON 契约，逐对谈判

这四类问题有一个共同点：**语义是跨边界的。**如果语义只在一个限界上下文内部使用，用 DDD 的领域模型和类图就够了，不需要本体。这是把本体与 DDD 区分开的关键判据。

应用地图

场景	核心能力	主要收益	展开阅读
知识图谱构建	抽取、对齐、映射、加载、推理、服务	把分散数据合成可查询的实体网络	知识图谱构建
语义检索与 RAG	查询理解与扩展、结构化召回、答案校验	降低歧义、减少幻觉、答案可追溯	语义检索与 RAG
数据治理与主数据	共享术语、数据目录、血缘、质量规则	口径统一、责任清晰、质量可测量	数据治理与主数据
业务规则与校验	一致性推理、SHACL 校验、规则治理	规则显式化、可测试、可版本化	业务规则与校验
语义集成与互操作	映射、联邦查询、语义化 API	降低集成成本，对接行业标准	语义集成与互操作
运营与生命周期	治理流程、CI 校验、指标监控	让本体成为可持续资产	运营与生命周期

此外还有两类常被提及的场景：**决策支持**（把规则与事实结合，做合规判定、风险筛查、推荐解释）与 **AI 可信**（为模型输出提供事实依据与来源追溯）。它们多数是上述场景的组合：决策支持 ≈ 规则校验 + 推理；AI 可信 ≈ 语义检索 + 来源建模（provenance）。不建议单独为它们立项，而应落到具体链路上。

什么时候值得做

满足下列条件中的多数，才值得投入本体：

- **有多个系统或组织需要共享同一套语义**，且用点对点映射已经明显吃力；
- **语义本身是争点**，例如“客户”到底指签约主体还是使用者，靠讨论难以对齐；
- **需要推理或可解释性**，结论必须能说明依据；
- **有合规、审计、互操作的外部压力**，标准词表能直接复用；
- **有长期维护的预期**，至少两三年内语义资产不会被整体废弃。

反过来，下面这些情况通常不该上本体：

- **单系统、单团队、边界清晰**：DDD 的领域模型加数据库约束即可；
- **只是想做可视化大屏**：图数据库或前端图可视化组件就够，不需要形式化语义；
- **数据规模很大但语义简单**：先解决数据质量问题，本体救不了脏数据；
- **没有明确的消费方**：建完没人查、没人验，就会变成一次性项目；
- **团队没有能力维护 OWL/SHACL**：那就从 SKOS 词表起步，不要直接上重公理。

反例：这些做法注定失败

1. **先建大本体，再找应用。**典型的“企业级本体”项目，半年产出一堆类和属性，却没有一个系统读取它。正确顺序是：找到消费方，从能力问题反推模型。
2. **把本体当文档交付。**文档不参与运行，就不会被校验，也就不会保持准确。至少要有一条自动校验链路（SHACL 或 CQ 查询）挂在 CI 上。
3. **试图让本体承担全部业务逻辑。**本体的强项是描述语义和推导事实，不是编排流程、处理事务。业务行为仍归应用层。
4. **过度公理化。**为了“更智能”加满 OWL 公理，结果推理变慢、模型难改、业务方看不懂，收益却很小。

这些反例的共同特征是：**没有可验收的消费场景。**立项时就要写清楚“谁在什么流程里读这个模型、读不到会怎样”。

ROI 怎么估算

本体的收益通常是“避免损失”，比“创造收入”更难量化，但并非不能算。按三栏对照：

- **成本**：建模人力、映射与管道开发、存储与工具（开源方案的主要成本往往在运维人力）、长期的维护与评审投入；
- **收益**：减少的返工工时、对账差异减少、查询与取数等待缩短、合规与互操作风险敞口下降、新系统接入周期缩短；
- **风险**：模型停滞、数据失真、消费方流失。没有治理机制时，收益会归零，这部分要计入风险成本。

估算方法是**先立基线再谈收益**：统计当前每月因语义不一致产生的返工工时、对账差异笔数、跨系统取数耗时，折算成可比较的投入；再看试点期内这些数字是否改善。做不出基线的场景，先别立项。

与 DDD 的分工

本体与 DDD 是互补关系，不是替代关系。一句话区分：**DDD 管业务边界与行为，本体管共享语义与数据互操作。**

维度	DDD	本体
关注点	限界上下文内的业务规则与行为	跨上下文的共享概念与事实
主要产物	聚合、实体、值对象、领域事件、领域服务	类、对象属性、数据属性、公理、个体
边界处理	上下文映射（防腐层、共享内核）	模块化、`owl:imports`、IRI 命名空间
一致性	聚合不变量、事务边界	推理一致性（OWL）、完整性校验（SHACL）
变化节奏	随业务迭代频繁调整	相对稳定，按版本演进

实践中的组合方式有两种：

- **DDD 为主、本体为辅**：业务系统按 DDD 建，跨系统的语义与主数据用本体表达。这是大多数团队的现实路径，见 分层与六边形架构。
- **本体为主、DDD 管行为**：以数据/知识为核心的系统（知识图谱平台、数据治理平台），语义层由本体驱动，行为逻辑仍用 DDD 组织。

不要生搬类比。聚合不变量与 OWL 公理形式相似，但前者约束一个一致性边界内的事务，后者是开放世界下的逻辑断言，混用会出问题。详见 约束与业务规则。

如何选第一个应用场景

用下面四个问题筛选，全部通过才进入试点：

1. **痛点是否可量化**：现在每天花多少人力对账、查数、清洗？有多大概率出错？没有基线就无法验收。
2. **范围是否收敛**：能否限定在 1–2 个业务域、3–5 个源系统、10 条以内的能力问题？范围越大，越容易拖成无底洞。
3. **数据是否可得**：源系统能否提供稳定接口或导出？有没有可控的更新频率？
4. **消费方是否明确**：谁会用？是查询接口、检索服务，还是校验流水线？没有消费方的场景不要选。

常见的第一场景排序（按落地难度从低到高）：

- **语义检索/问答**：见效快、可演示，适合争取支持，但对语义严谨度要求不高；
- **数据治理术语与目录**：组织阻力小，是后续所有场景的基础；
- **规则校验**：边界清晰、价值直接，适合合规驱动的组织；
- **知识图谱平台**：投入最大，建议在其他场景验证过语义模型后再做。

应用工程的通用链路

尽管场景不同，落地链路大体一致，后续各页都会回到这条主线：

text

```
能力问题（CQ）
  → 本体模型（TBox：类/属性/公理/约束）
```

```
→ 实例数据（ABox：映射、清洗、实体对齐）
  → 存储（三元组库或混合架构）
    → 服务（查询/校验/推理/检索接口）
      → 消费方（应用、报表、模型、外部伙伴）
        → 反馈（CQ 通过率、数据质量、使用量）回到模型迭代
```

链路上每一环都可以先做最简版本：TBox 用轻量词表起步，ABox 先导入一个源系统，存储先用单机三元组库，服务先暴露一个 SPARQL 查询接口。不要等所有环节都设计完美再开始。

接下来读什么

- 想了解建模方法：从 本体工程方法论 开始。
- 想了解技术实现：看 实现本体总览 与 存储选型。
- 想知道完整案例：读 端到端案例：电商商品与订单本体。
- 想把本体引入组织：读 落地路线图。

知识图谱构建

知识图谱（Knowledge Graph）是由本体定义模式、由实例数据填充的图结构数据资产。本页讲清楚从源系统到可服务图谱的完整流水线、每个阶段的具体做法、质量指标，以及最容易被低估的坑。

知识图谱与本体是什么关系

一个常见误解是把图数据库里的数据都叫“知识图谱”。严格说：

- **知识图谱 = 本体（模式）+ 实例数据（+ 推理结果）**。没有模式的图只是数据，语义靠字段名和口头约定，换个团队就读不懂；
- **本体 ≠ 知识图谱**。本体是类和公理，通常几百到几千个声明；图谱是 TBox 加 ABox 的整体。同一个本体可以支撑多个图谱；
- **图数据库 ≠ 知识图谱**。图数据库是存储引擎，属性图模型没有内建语义推理，RDF 三元组库也不强制你写本体。

所以“建知识图谱”的真实工作量，大部分不在存储上，而在**语义建模、实体对齐和数据质量**上。经验上，模型设计、映射开发和质量治理占大头，加载与查询只占小头。规划时按这个比例分配人力。

流水线总览

text

```
源系统（数据库/API/文件/文档）
  → 抽取（增量/全量、脱敏、格式统一）
    → 清洗与规范化（单位、编码、时间、去重）
      → 实体识别与消歧（把“同一个人/公司/商品”合并）
        → 本体映射（映射到类与属性，生成 RDF）
          → 加载与索引（三元组库/图库）
            → 推理与校验（隐含事实、SHACL 一致性）
              → 质量评估（覆盖率、准确率、连通性）
                → 服务（查询 API、检索、图谱可视化）
                  → 反馈迭代（补映射、修数据、改模型）
```

下面逐段说明。每段都给出最简可上线版本和后续加强方式。

阶段一：界定范围与能力问题

先写 5–10 条能力问题（CQ），再决定抽什么数据。做法见 能力问题（CQ）。

范围要同时限定三个维度：**业务域**（如“售后商品”而不是“全部商品”）、**源系统**（3–5 个）、**时间范围**（首批只导入近一年数据）。三个维度都不限，项目必然延期。

同时明确“图谱不需要什么”。例如不需要全量订单明细、不需要用户行为日志，这些放在数据仓库更合适。

阶段二：抽取与清洗

抽取方式按源系统类型选择：

源类型	抽取方式	注意点
关系数据库	批量 SQL 导出、CDC 增量	主键变更会破坏实体 IRI 稳定性
REST/GraphQL API	定时拉取、分页游标	速率限制与字段含义文档缺失是主要风险
文件（CSV/Excel）	批处理导入	编码、单位、空值表示要统一
文档（PDF/HTML）	文本抽取 + 规则/模型抽取	抽取结果必须带来源与置信度
行业交换格式	复用标准映射（如 FHIR、IFC、GS1）	先核对版本差异

清洗阶段的核心是**把值域收敛**：国家代码统一 ISO 3166、货币统一 ISO 4217、单位统一 SI、时间统一 UTC 或明确时区。做不到统一，就先建映射表，别急着上图谱。

阶段三：实体识别、消歧与链接

这是知识图谱质量的分水岭。同一条真实记录在不同系统里有不同标识，必须合并，否则图谱只是“把多张表堆在一起”。

常用策略按可靠性排序：

1. **权威 ID 直连**：源系统共享同一个业务编码（如商品 SKU、统一社会信用代码），直接复用为实体 IRI。最可靠，优先做。
2. **确定性规则匹配**：规范化后的邮箱、手机号、证件号完全相等即视为同一实体。
3. **模糊匹配 + 人工确认**：名称相似度、地址相似度组合打分，高置信度自动合并，中置信度进人工队列。
4. `owl:sameAs` **声明**：合并结果用 `sameAs` 表达，但**不要滥用。** `sameAs` 是等价公理，推理器会把两个个体的所有属性合并，错一条会污染一大片。更稳妥的做法是保留各自的“系统内 ID”，另用专门的实体表记录对齐关系，需要时再投影成 `sameAs`。

消歧必须可审计：每条合并记录保存依据（哪个字段、什么规则、置信度）与操作人。之后出现错并，才能拆分修复。

阶段四：本体映射

把清洗后的结构化数据映射到本体。三种主流方式，按复杂度选择：

- **R2RML**：W3C 推荐标准，用映射文档把关系表映射为三元组，适合“表结构稳定、映射规则清晰”的场景；
- **RML**：在 R2RML 思想上扩展，支持多格式源（CSV、JSON、XML），目前是社区规范；
- **SPARQL CONSTRUCT / 程序映射**：用 `CONSTRUCT` 或代码逐行生成三元组，灵活度最高，适合复杂转换和需要调用外部服务的场景。

一段最小的 `CONSTRUCT` 映射示例（把源表的商品行映射为本体实例）：

sparql

```
PREFIX ex: <https://example.com/onto#>
PREFIX prop: <https://example.com/onto/prop/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

CONSTRUCT {
  ?product a ex:Product ;
           ex:productName ?name ;
           ex:hasCategory ?category ;
           ex:listPrice ?price .
  ?category a ex:Category .
}
WHERE {
  ?row prop:sku ?sku ;
       prop:name ?name ;
       prop:categoryCode ?categoryCode ;
       prop:price ?price .
  BIND(IRI(CONCAT("https://example.com/product/", ENCODE_FOR_URI(?sku))) AS ?product)
  BIND(IRI(CONCAT("https://example.com/category/", ENCODE_FOR_URI(?categoryCode))) AS ?category)
  FILTER(DATATYPE(?price) = xsd:decimal)
}
```

映射开发的三条纪律：

- **IRI 生成规则唯一且稳定。**用业务主键派生，不要用自增行号或时间戳，否则重跑会造出重复实体；
- **映射即代码**，进版本库、有测试、可重跑。手工在 Protégé 里点出来的数据无法维护；
- **记录来源。**每个实体保留 `prov:wasDerivedFrom` 或自定义来源属性，指向源系统与记录 ID，用于追溯和重新抽取。

阶段五：加载、推理与校验

加载前先定存储。选型见 存储选型。要点：

- **批量导入用专用加载器**，不要用逐条 SPARQL Update 灌几百万条数据；
- **推理物化与查询时推理二选一。**全量物化简单但占空间、更新慢；查询时推理省空间但延迟不可控。混合方案通常是“RDFS 子集物化 + 规则按需”。详见 推理与物化；
- **加载后跑 SHACL。**数据不通过形状校验就发布，等于把问题推给消费方。校验规则见 SHACL 形状约束。

阶段六：质量指标

不要用“数据量”衡量图谱。用下面四类指标，并且**给出基线值和目标值**（具体数值由项目根据历史数据确定，不要抄别人的百分比）：

指标	定义	测量方式
覆盖率	目标实体中有多少已入图、必填属性填充率	抽样比对源系统计数
准确率	抽样实体中属性值与实体对齐正确的比例	人工抽样评审，记录样本量与置信区间
连通性	孤立实体比例、平均度数、关键关系缺失率	图查询统计
一致性	OWL 推理一致性、SHACL 违规数、重复实体数	自动校验流水线

指标要能追溯到具体规则。例如“订单-商品关系缺失率”对应一条 SHACL `minCount` 约束，这样质量下降时能定位到原因，而不是只知道“图变差了”。

阶段七：服务与运营

图谱要有人用，必须有服务层：

- **查询 API**：把常用 CQ 封装为参数化接口，不让前端直接拼 SPARQL（注入与性能都不可控）；
- **检索融合**：图查询与关键词/向量检索组合，见 语义检索与 RAG；
- **可视化**：按业务问题定制视图，不做通用“全图展示”，节点超过几百个就没有可读性。

运营上，把回调链路建起来：消费方反馈的错例 → 映射或模型修正 → 重跑受影响分区 → 回归校验。没有这条链路的图谱，半年后就会和源系统脱节。

与数据仓库、数据湖的关系

三者不冲突，分工不同：

- **数仓**擅长聚合统计与历史快照，结构化查询性能好；
- **湖**擅长存原始多态数据；
- **图谱**擅长关系遍历、多跳查询与跨源实体整合。

现实做法通常是混合：数仓做指标计算，图谱负责实体关系与语义服务，通过 ID 对齐，不追求把数仓全量搬进图里。把所有明细都灌进三元组库，往往换来查询变慢和运维复杂，收益有限。

常见坑

1. **先建图后想用途**：图建好了没人查。先定 CQ 和消费方。
2. **追求大而全的本体**：一个域一个模块，逐步扩展，见 模块化与复用。
3. `sameAs` **滥合并**：错并难拆，且推理会放大污染。保留来源 ID 与对齐表。
4. **IRI 不稳定**：重跑生成新实体，图谱越来越脏。
5. **忽略增量更新**：只做一次性全量导入，三个月后数据过期。
6. **无校验直接上线**：脏数据进入检索和决策链路，比没有图谱更糟。
7. **推理开满**：全量物化导致加载以小时计，调参无门。按需限制到 RDFS 子集或目标 profile。

语义检索与 RAG

把本体用在检索与 RAG 上，目标不是“让搜索更智能”，而是**让检索命中正确的实体、让生成有据可依、让答案可追溯。**本页给出三条检索路线的取舍、本体在链路上的六个插入点、参考架构与评估指标，并对 GraphRAG 给出审慎判断。

三种检索范式

范式	原理	强项	弱项	适用
关键词检索（BM25 等）	词项匹配与统计相关度	精确词、编号、专名命中最准；可解释	同义词、跨语言、语义漂移失效	有明确术语的垂直检索
向量检索	语义嵌入 + 近邻搜索	语义相近、问法与文档措辞不同也能召回	精确匹配弱、易召回“意思像但不对”的内容、难解释	问答、长文本召回
图 + 向量混合	向量召回实体/块，再沿本体关系扩展或过滤	结构化约束、多跳关系、可追溯	实现复杂，依赖图谱质量	关系密集、需要多跳的领域

结论：**关键词检索不会消失，向量检索不是银弹，混合路线是当前主流。**向量召回解决“找不到”，图结构解决“找得乱”。如果只有向量库、没有本体，就无法把“这款产品的替代型号”这类关系型问题答对。

本体在链路中的六个位置

多数团队只用本体做一件小事，其实它有六处可用：

1. **查询理解**：把自然语言问题解析为实体、属性、约束。识别出“去年 Q3”“华东区”“毛利率”分别对应本体的时间区间、区域、指标定义。
2. **查询扩展**：用 `skos:altLabel`、上下位关系、等价类补全同义词与更宽/更窄的检索词。这是 SWRL 之外最便宜、收益最稳的一招。
3. **实体链接**：把提问中的名称链接到图谱中的实体 IRI，消解同名与别名问题。链接准确率直接决定后续结构化召回的质量。
4. **结构化召回**：用 SPARQL 精确筛选（如“属于某品类且价格区间内且已停产”），再把结果作为上下文。比纯向量召回可控得多。
5. **约束生成与校验**：把本体中的取值域、基数、单位转成生成阶段的约束，输出后按 SHACL/本体一致性做检查，拦截不合法答案。
6. **答案可解释**：返回答案时同时返回依据的三元组或路径，用户能核对，也能据此判断该信任多少。

前三点投入小、见效快，建议先做；后三点依赖图谱质量与工程投入，按需推进。

参考架构

text

```
用户问题
  → 查询理解（分词/意图/实体识别 + 本体词典）
```

```
    → 实体链接（别名 → 实体 IRI，取候选打分）
      → 三路召回并行：
          a) 关键词检索（BM25）
          b) 向量检索（chunk embedding）
          c) 图查询（SPARQL 结构化召回 + 邻域扩展）
        → 融合去重（按实体/文档聚合打分）
          → 重排（rerank 模型或规则）
            → 生成（LLM，提示中注入结构化上下文）
              → 校验（SHACL/一致性/事实核对）
                → 返回答案 + 依据
```

工程上要注意几点：

- **分块策略与本体对齐。**chunk 的元数据里带上实体 IRI、文档来源、时间，检索时才能按本体约束过滤，而不是把全文丢给向量库；
- **向量库与三元组库分离**，通过实体 IRI 关联。不要试图用三元组库替代向量检索；
- **上下文预算。**结构化上下文要精简到“回答问题所必需的子图”，把整张子图塞进提示会挤掉其他证据。

查询理解与扩展示例

用 `skos:altLabel` 与上下位关系做扩展，是最容易落地的一段：

sparql

```
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX ex: <https://example.com/onto#>

# 输入：用户提到的词 ?term；返回同义词 + 下位词作为扩展检索词
SELECT DISTINCT ?expanded WHERE {
  ?concept skos:prefLabel ?term ;
           ?link ?expanded .
  VALUES ?link { skos:altLabel skos:narrower }
}
```

注意 `narrower` 只在确实需要“更具体也算命中”时展开；否则会召回大量噪声。扩展策略应当按查询类型分别配置，而不是全局开关。

GraphRAG：新兴实践，别急着上

GraphRAG 指用 LLM 从语料中抽取实体与关系，构建图，再基于图与社区摘要增强检索与生成。微软研究院在 2024 年提出了这一方向的代表性工作，此后社区工具快速跟进。需要明确：

- 它**仍在演进**：抽取质量依赖模型与提示，评测方法尚未统一，不同实现的结论差异很大；
- 建图成本高：需要 LLM 逐块抽取实体关系，语料规模、更新频率都会放大成本；
- 它解决的是“全局性问题”（跨文档归纳、主题聚类），对“精确事实检索”未必优于混合检索；
- 如果你的领域已有稳定本体，**不必让 LLM 从零建图**，直接映射到已有模式质量更高、可复现性更好。

建议：先做“本体 + 混合检索”，把实体链接与结构化召回做扎实；确有跨文档归纳需求，再评估 GraphRAG，并且用小样本做 A/B 验证，而不是全量上线。

评估指标

检索与 RAG 的评估必须拆开看，否则无法定位问题：

环节	指标	测量方式
检索召回	Recall@k、实体链接准确率	人工标注问题集，统计命中
检索排序	NDCG@k、MRR	标注相关度分级后计算
生成质量	答案正确率、依据支持率（是否被检索证据支撑）、幻觉率	抽样人工评审 + 自动核对三元组
稳定性	同一问题的答案一致性	重复提问对比
性能	P50/P95 延迟、单次查询成本	埋点统计

样本集要覆盖四类问题：**事实型**（单一实体属性）、**比较型**（多实体对比）、**关系型**（多跳关系）、**聚合型**（跨实体的统计或归纳）。只测事实型会导致指标虚高。

成本与落地建议

- **先建词典，再建向量。**术语表与别名词典（可用 SKOS 维护）成本最低，实体链接准确率提升最直接；
- **混合检索优先于纯向量。**有明确术语的行业场景，BM25 + 实体过滤往往就够；
- **生成阶段强制引用。**提示中要求给出依据 IRI 或文档编号，没有依据就拒答。这比事后校验便宜；
- **控制领域范围。**检索范围越小，召回干扰越少。先做一个业务域，再横向扩展；
- **别用 RAG 掩盖数据问题。**源数据有错，检索只会稳定地返回错误答案。

常见坑

1. **只做向量检索**：精确编号、型号、法规条款查询效果差，而这类查询在行业场景里占比很高。
2. **本体只用于同义词扩展**：浪费了结构化召回与约束校验的价值；反过来，也不必强求全部六点都上。
3. **实体链接不做消歧**：同名实体混在一起，答案张冠李戴，且很难被发现。
4. **评测算平均值**：聚合指标掩盖长尾失败。按问题类型分组看指标。
5. **chunk 不带元数据**：无法按本体过滤，检索退化为纯文本相似度。
6. **让 LLM 自由总结图谱**：生成阶段脱离检索证据就会编造，必须限制在给定子图内。

延伸阅读：知识图谱构建（图谱质量是检索上限）、SPARQL 查询与更新（结构化召回写法）、Python 技术栈（rdflib 与检索服务集成）。

数据治理与主数据

数据治理最常见的失败不是缺工具，而是**缺一套共享语义**：业务说“客户”、报表说“会员”、风控说“主体”，三份口径互不一致，对账永远对不完。本体在这里的角色是**把术语、结构、规则和来源变成机器可读的资产**，让治理措施能被自动执行。

本体作为共享语义层

治理工作中的四个落点，都可以由本体承载：

落点	传统做法	本体做法
业务术语表	Excel / Wiki 词条	SKOS 概念 + 定义 + 同义词 + 责任方，可查询
数据字典	表结构文档	数据属性 + 取值域 + 与业务概念的映射
数据目录	治理平台元数据	DCAT 描述数据集/服务 + 本体标注
血缘与来源	平台自动采集	PROV-O 表达派生关系，跨平台可合并

关键在于**机器可读**：术语表写在 Wiki 里，只能靠人查；写成 SKOS 后，检索系统、校验流水线、BI 工具都能读。这就是本体与文档的本质差别。

业务术语表：从 SKOS 起步

不需要一上来就用 OWL。术语表阶段用 SKOS 就够：

turtle

```
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix ex:   <https://example.com/glossary/> .

ex:Customer a skos:Concept ;
    skos:prefLabel "客户"@zh , "Customer"@en ;
    skos:altLabel  "用户"@zh , "会员"@zh , "客户主体"@zh ;
    skos:definition "与本公司签署销售合同并承担付款义务的法人或自然人。"@zh ;
    skos:broader ex:Party ;
    skos:related ex:Contact ;
    skos:notation "CUST" .
```

三条纪律：

- **定义要能判定**：写“签署销售合同并承担付款义务”，而不是“购买产品或服务的人”。前者能判断真假，后者不能；
- **同义词必须收全**：`altLabel` 直接决定检索与实体链接的召回率，是治理工作中投入产出最高的部分；
- **标注责任方**：用 `dcterms:owner` 或自定义属性记录术语的负责人与评审状态，否则术语会随人员流动而失管。

当术语需要参与推理（如“客户”是“交易方”的子类，且与“供应商”不相交），再升级为 OWL 类。升级路径见 SKOS 词表与分类法 与 本体元素总览。

数据目录：用 DCAT 描述资产

数据目录要回答“有什么数据、谁负责、怎么取、多久更新”。DCAT 是 W3C 的目录词汇标准，用 `dcat:Dataset`、`dcat:Distribution`、`dcat:DataService` 等描述数据集及其分发形式，可以直接复用，不必自造。

落地要点：

- **数据集与本体概念对齐。**每个数据集标注它覆盖的业务概念（`dcterms:subject` 指向 SKOS 概念），这样业务方能从“客户”反查有哪些数据源；
- **目录自动采集优先。**手填目录必然过期。能从元数据系统、数据库、对象存储采集的字段就自动采；
- **目录要能被查询。**用 SPARQL 端点提供“某个概念对应哪些数据集”的查询，比让人翻网页有效。

血缘与来源：PROV-O

数据血缘用 PROV-O（W3C 推荐）表达：`prov:Entity`（数据）、`prov:Activity`（加工过程）、`prov:Agent`（系统或人）、`prov:wasDerivedFrom` / `prov:wasGeneratedBy`。

治理价值在于**跨平台合并**：ETL 工具、调度系统、报表工具各记一套血缘，只有用统一模型才能串成完整链路。实践建议：

- 血缘粒度先到“表/数据集”级，不要一开始追求字段级；
- 血缘要能回答“这个指标出错，影响哪些报表和下游系统”，这是最常被问的问题；
- 血缘数据本身也要有质量：加工任务重跑、改名、废弃都会让血缘失真，需要定期校准。

质量规则：SHACL 承载可执行约束

治理规则最怕“写了不执行”。把规则写成 SHACL 形状，就能挂到 CI 或调度任务里自动跑，输出可定位的违规报告。示例见 SHACL 形状约束。

规则分层放置：

- **结构/取值规则**（必填、类型、范围、格式）→ SHACL；
- **跨实体一致性**（订单金额 = 明细之和）→ SHACL-SPARQL 或独立校验任务；
- **业务不变量**（一个订单只能属于一个客户）→ 部分可由 OWL 基数公理或 key 表达，但要注意开放世界的语义差异，见 公理；
- **统计型监控**（空值率、分布漂移）→ 数据质量平台，不必塞进 SHACL。

严重级别（`sh:severity`）要区分：`sh:Violation` 阻断发布，`sh:Warning` 记录待修，`sh:Info` 仅提示。全部设为 Violation 会导致没人看报告。

主数据建模

主数据（客户、产品、供应商、组织、物料）是治理的核心对象，也是本体最容易见效的地方：

1. **确定主数据实体的唯一标识**：优先复用行业编码（如商品条码、统一社会信用代码），没有就设计稳定内部编码；

2. **划分来源优先级**：每个属性指定权威源系统（source of truth），冲突时按优先级取值，不要把合并逻辑散落在代码里；
3. **跨系统映射独立维护**：源系统 ID 与主数据 ID 的映射表是长期资产，变更要留版本；
4. **属性分级**：核心属性（名称、编码、状态）严格校验；描述属性宽松处理，避免因细节阻塞主数据发布。

主数据建模与 DDD 的对应关系：主数据实体大致对应跨上下文的共享内核，而各业务上下文仍保留自己的聚合。不要试图用一份全局模型替代所有上下文内部模型。

指标口径统一

指标口径争议通常源于定义中的隐含维度。用本体把指标拆成可计算的结构：

turtle

```
@prefix ex: <https://example.com/metric#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .

ex:GrossMarginRate a ex:Metric ;
    skos:prefLabel "毛利率"@zh ;
    ex:formula "(营业收入 - 营业成本) / 营业收入" ;
    ex:hasDimension ex:Region , ex:ProductCategory , ex:Period ;
    ex:numeratorMetric ex:OperatingRevenue ;
    ex:denominatorMetric ex:OperatingCost .
```

口径统一的关键动作是**把分子分母、维度、时间窗口、排除项写清楚**，并与物理实现绑定（哪张表、哪个字段）。指标定义进版本库，变更走评审。这样“为什么两个报表的毛利率不同”就能被直接查出来，而不是开会吵。

与湖仓、数据中台的关系

- **湖仓/数仓**负责存储与计算，指标在数仓实现；
- **治理平台**负责人、流程、审批；
- **本体/知识图谱**负责语义：术语、映射、血缘、质量规则。

三者通过 ID 与元数据对接。不要指望用本体替代数仓的聚合计算，也不要指望治理平台自己长出语义。常见集成方式：数仓表 → 映射为本体实例（R2RML/程序映射）→ 暴露语义查询；治理平台的元数据 → 导出为 DCAT + PROV-O。

组织与角色

角色	职责	常见误区
数据管家（Steward）	维护术语、审核变更、处理质量问题	被当成兼职行政工作，没有时间投入
领域专家	定义业务概念与规则	只在启动会露面，之后不再参与
数据工程师	映射开发、质量规则实现、流水线	只关心管道跑通，不关心语义正确
语义/知识工程师	本体设计、推理与校验、模型评审	过度追求形式完美，脱离业务

治理要能持续，必须给管家**明确的时间预算与考核项**，否则术语表三个月后就没人更新。这一点比工具选型重要得多。

落地步骤与指标

建议按季度推进：

1. **第 1 季度**：选一个域，建术语表（SKOS），覆盖核心 50–100 个概念，明确责任方；
2. **第 2 季度**：把该域核心数据集纳入目录（DCAT），标注概念映射；上线 5–10 条 SHACL 质量规则；
3. **第 3 季度**：建主数据实体与跨系统映射，接入血缘（PROV-O），打通“概念 → 数据集 → 指标”的查询；
4. **第 4 季度**：扩展第二个域，建立变更评审流程与 CI 校验，把指标口径纳入版本管理。

指标建议：术语覆盖率（核心概念中有定义且有责任方的比例）、映射覆盖率、质量规则自动执行率、问题平均修复时长、目录查询使用量。数值目标按自身基线设定，不要照搬外部基准。

常见坑

1. **术语表只写不读**：没有检索或校验消费，等于文档。至少接一个查询入口。
2. **一上来做全企业本体**：范围失控。按域推进。
3. **定义写成同义反复**：“客户就是客户信息”，无法判定。
4. **质量规则没有责任人**：违规报告没人处理，三个月后自动失效。
5. **主数据合并逻辑藏在 ETL 里**：无法审计、无法回滚。映射与优先级要显式建模。
6. **把治理当纯 IT 项目**：没有业务方参与，术语一定失真。

业务规则与校验

业务规则散落在代码、存储过程、Excel 和人的记忆里，是系统最脆弱的部分。本页讲如何把规则拆成三类载体——**OWL 一致性、SHACL 校验、规则引擎**——各自承担什么、怎么治理，以及和 DDD 聚合不变量如何配合。

先分清三类规则载体

载体	语义	适合的规则	结果
OWL 公理	开放世界，声明必然为真	类不相交、基数、属性链、key	推理出一致性冲突，或推出隐含事实
SHACL	封闭世界，校验数据是否满足	必填、类型、范围、格式、跨字段、计数	输出违规报告，指出哪个节点哪个属性
规则引擎	任意前件后件，过程化	需要执行动作、跨系统调用、复杂计算	触发动作、变更状态

最常见的错误是把 `rdfs:domain` / `rdfs:range` 当校验规则。它们只产生类型推断：写 `ex:orderDate rdfs:range xsd:dateTime`，数据里放字符串"2024-13-01"不会报错，反而会推出“该值是 xsd:dateTime”的结论。要校验就用 SHACL。

另一条原则：**能用 SHACL 就不用规则引擎。**SHACL 声明式、可测试、可版本化；规则引擎强在“做什么”，但规则一多就变成不可维护的逻辑泥潭。

规则分层：从数据入口到业务动作

按执行时机分三层，避免所有规则堆在一个点：

text

```
入口层（写入/导入）
  → SHACL：结构、类型、必填、格式  ← 快速失败，成本最低
    → 发布层（数据合并/主数据发布）
      → SHACL-SPARQL + SPARQL 计数：跨实体一致性、唯一性
        → 决策层（业务流程）
          → 规则引擎/领域服务：需要动作、跨系统、时效判断的规则
```

入口层规则要严，发布层规则要全，决策层规则要少。决策层规则越少，越容易测试和解释。

SHACL 示例：订单校验

turtle

```
@prefix sh:   <http://www.w3.org/ns/shacl#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
@prefix ex:   <https://example.com/onto#> .

ex:OrderShape a sh:NodeShape ;
```

```
    sh:targetClass ex:Order ;
    sh:property [
        sh:path ex:orderNumber ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
        sh:datatype xsd:string ;
        sh:pattern "^ORD-[0-9]{8}$" ;
        sh:message "订单号必填且格式为 ORD-8 位数字"@zh
    ] ;
    sh:property [
        sh:path ex:hasCustomer ;
        sh:minCount 1 ;
        sh:class ex:Customer ;
        sh:message "订单必须关联客户"@zh
    ] ;
    sh:property [
        sh:path ex:orderTotal ;
        sh:minCount 1 ;
        sh:datatype xsd:decimal ;
        sh:minInclusive 0 ;
        sh:message "订单金额必填且不小于 0"@zh
    ] ;
    sh:property [
        sh:path ex:hasOrderLine ;
        sh:minCount 1 ;
        sh:node ex:OrderLineShape ;
        sh:message "订单至少一条明细"@zh
    ] .
```

跨实体规则（明细金额求和等于订单总额）超出基本约束，用 SPARQL 约束表达：

turtle

```
@prefix sh:  <http://www.w3.org/ns/shacl#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ex:  <https://example.com/onto#> .

# SHACL-SPARQL 中的前缀需显式声明
ex: a owl:Ontology ;
    sh:declare [
        sh:prefix "ex" ;
        sh:namespace "https://example.com/onto#"^^xsd:anyURI
    ] .

ex:OrderTotalShape a sh:NodeShape ;
    sh:targetClass ex:Order ;
    sh:sparql [
        sh:message "订单总额必须等于明细金额之和"@zh ;
        sh:severity sh:Violation ;
        sh:prefixes ex: ;
        sh:select """
            SELECT $this (ex:orderTotal AS ?path) (?total AS ?value)
            WHERE {
              $this ex:orderTotal ?declared .
              { SELECT $this (SUM(?amount) AS ?total) WHERE {
                  $this ex:hasOrderLine/ex:lineAmount ?amount .
```

```
            } GROUP BY $this }
          FILTER(ABS(?declared - ?total) > 0.001)
        }
      """
  ] .
```

注意浮点容差：金额比较不要用等号。

规则分类清单

建模时逐类核对，避免遗漏：

- **结构规则**：必填、唯一、类型、格式、基数；
- **取值规则**：范围、枚举、单位、精度、编码规范（如统一社会信用代码校验位）；
- **跨实体规则**：引用完整性、金额合计、状态机前置条件、层级深度限制；
- **时序规则**：生效日期不重叠、结束时间晚于开始、状态变更顺序、有效期覆盖；
- **统计规则**：比例阈值、异常波动，一般不放 SHACL，交给监控平台。

时序规则容易被忽略。用时间区间建模（`ex:validFrom` / `ex:validTo`）配合 SPARQL 重叠检测，比在代码里写一堆日期比较可靠。

行业例子

金融合规

典型规则：客户风险等级与产品风险等级匹配、投资冷静期、关联交易识别、披露文件完整性。做法上，把监管条款拆成“适用对象 + 条件 + 结论 + 依据条款号”，用 SHACL 或规则表实现，并保留条款号用于审计追溯。关键是**规则可追溯到监管原文**，否则监管检查时无法说明依据。

制造 BOM

典型规则：物料替代关系的有效性、BOM 层级与用量校验、工程变更（ECN）影响范围、同一装配件下位物料版本一致性。BOM 天然是图结构，用本体表达“装配-部件”的传递关系与版本约束，比关系表递归查询更直观。注意：**部分-整体关系要区分“组成”与“分类”**，混用会导致层级校验误判，参见 本体设计模式。

规则治理

规则是资产，需要和代码一样管理：

1. **规则有唯一 ID、版本、责任人、生效日期、失效日期**；
2. **变更走审批**，影响范围要评估（哪些数据集、哪些下游）；
3. **每条规则有测试用例**：至少一个通过样例、一个违规样例，进 CI；
4. **例外要显式**：业务允许的例外用白名单记录（含原因、期限、批准人），不要靠关闭规则解决；
5. **违规报告可操作**：包含节点 IRI、属性、实际值、期望值、规则 ID、严重级别；
6. **定期回归**：数据量变化、模型升级后重跑全量校验。

严重级别分三档：`sh:Violation` 阻断、`sh:Warning` 记录、`sh:Info` 提示。全部设成 Violation 的结果是报告没人看。

与 DDD 聚合不变量配合

边界要划清楚：

- **聚合内部不变量**（如订单明细数量上限、状态迁移合法性）由聚合根在事务内保证，不要靠事后 SHACL 检查；
- **跨聚合/跨系统的规则**（订单金额与发票金额一致、客户信用额度）不适合放在单个聚合里，用 SHACL 或独立校验服务；
- **规则本体中的约束是“数据契约”**，聚合是“行为契约”，两者互补。

实现上，领域对象在持久化前可以调用语义校验服务（端口-适配器，见 分层与六边形架构），但不要让业务事务依赖外部推理器的可用性。校验失败的策略要明确：阻断、降级记录、还是异步告警。

失败模式

1. **规则写在三个地方**：数据库约束、代码 if-else、SHACL 各一套，彼此矛盾。
2. **OWL 公理当校验**：期待推理器报错，结果只推出类型，脏数据照样入库。
3. **规则无版本**：业务说“这条规则早改了”，却查不到历史。
4. **校验太晚**：只在月末跑一次，问题数据已经流向下游。
5. **违规不可定位**：报告只写“有 372 条违规”，没有节点和属性，无法修复。
6. **例外靠关规则**：关掉一次就再也开不回来。
7. **规则过度形式化**：简单格式校验也写成 SPARQL，维护成本高于收益。能用 `sh:pattern` 就不要写查询。

语义集成与互操作

系统集成的主流做法是点对点接口或企业服务总线（ESB），两者都能跑通流程，但都不解决**含义**问题：字段叫 `cust_name` 还是 `customerName`、金额单位是元还是分、状态码 `02` 到底表示什么，只能靠文档和口头约定。语义集成把这些问题前移到模型层，用统一 IRI 和显式映射替代逐对谈判。

语义集成解决什么

集成方式	耦合点	新增一个系统	语义一致性
点对点接口	每个调用方与被调方	需要 n 条新映射	每对单独约定
ESB / 消息总线	消息格式与路由	需接入总线并转换格式	转换靠映射组件，语义不显式
语义层（本体 + 映射）	共享词表与 IRI	只需提供到共享模型的映射	由模型统一保证

结论：**点对点适合少量系统、稳定契约；本体适合系统多、语义易变、需要跨组织交换的场景。** 如果只有两三个系统且接口稳定，不要为了“先进”引入语义层，成本不划算。

语义集成有三个层次，投入依次上升：

1. **词表对齐**：统一术语与编码（国家、货币、单位、状态），先解决“词不同”；
2. **数据映射**：把源结构映射到共享本体，产出 RDF，解决“结构不同”；
3. **联邦与虚拟化**：不搬数据，查询时跨源组合，解决“数据不能动”。

多数项目做到第 2 层就有明显收益，第 3 层按需。

映射方式选型

方式	标准状态	适用	注意
R2RML	W3C 推荐标准	关系表 → RDF，映射清晰稳定	只覆盖关系数据库
RML	社区规范	多格式源（CSV/JSON/XML），扩展 R2RML 思路	工具支持不如 R2RML 普及
SPARQL CONSTRUCT	W3C 标准查询语言的一部分	复杂转换、需调用外部逻辑	映射逻辑写在查询里，可读性要控制
程序映射（代码）	无标准	大规模、需调服务、需复杂清洗	必须配测试与来源记录

示例：把源表映射到共享本体的最小 `CONSTRUCT`：

sparql

```
PREFIX ex: <https://example.com/shared#>
PREFIX src: <https://example.com/source/>

CONSTRUCT {
  ?customer a ex:Customer ;
            ex:legalName ?name ;
```

```
            ex:taxId ?taxId ;
            ex:registeredIn ?country .
  ?country a ex:Country .
}
WHERE {
  ?row src:cust_name ?name ;
       src:tax_id ?taxId ;
       src:country_code ?cc .
  BIND(IRI(CONCAT("https://example.com/customer/", ENCODE_FOR_URI(?taxId))) AS ?customer)
  BIND(IRI(CONCAT("https://example.com/country/", ENCODE_FOR_URI(?cc))) AS ?country)
}
```

映射的三条纪律：**IRI 由业务主键派生且稳定、映射文档进版本库并有测试、每条三元组保留来源**（`prov:wasDerivedFrom`）。缺了第三条，数据出错时无法回溯。

联邦查询与数据虚拟化

数据不能搬（合规、时效、体量）时，用 SPARQL 联邦查询：`SERVICE` 把子查询发给远端端点，本地组合结果。

sparql

```
PREFIX ex: <https://example.com/shared#>

SELECT ?customer ?orderCount WHERE {
  SERVICE <https://partner.example.com/sparql> {
    ?customer a ex:Customer ; ex:taxId ?taxId .
  }
  {
    SELECT ?taxId (COUNT(?order) AS ?orderCount) WHERE {
      ?order ex:hasCustomer/ex:taxId ?taxId .
    } GROUP BY ?taxId
  }
}
```

联邦查询的现实约束必须提前说清楚：

- **性能不可控**：受远端端点延迟与限流影响，`SERVICE` 嵌套或无界模式可能超时；
- **端点能力不一**：对方可能只支持部分 SPARQL 1.1，甚至只提供 REST；
- **可用性耦合**：远端挂掉，查询整体失败。关键路径要有降级（缓存上次结果、改走批量同步）；
- **权限与合规**：联邦不搬数据不等于没有数据出境问题，需法务确认。

因此**联邦适合探索性查询与少量关键查询，不适合高频、低延迟、强一致的业务调用。**后者建议做受控同步 + 本地图谱。

API 语义化

对外接口同样可以带语义：

- **JSON-LD**：在现有 JSON 中加 `@context`，把字段映射到 IRI，前端无需改动即可获得可扩语义。适合渐进改造；
- **内容协商**：同一资源同时提供 JSON 与 Turtle，按 `Accept` 头返回，见 RDF 数据模型与序列化；

- **超媒体与链接**：Hydra 等思路用词汇描述可用操作与参数，目前是社区方向，采用前先确认客户端生态支持；
- **版本策略**：语义化 API 的字段变更遵循本体版本策略，见 版本与演进。

一个 JSON-LD 片段：

json

```
{
  "@context": {
    "ex": "https://example.com/shared#",
    "legalName": "ex:legalName",
    "taxId": { "@id": "ex:taxId", "@type": "@id" }
  },
  "@id": "https://example.com/customer/91310000XXXXXXXX",
  "@type": "ex:Customer",
  "legalName": "示例科技有限公司",
  "taxId": "91310000XXXXXXXX"
}
```

行业标准：能复用就别自造

按准确度择用，采用前核对版本与许可：

标准	维护方	覆盖领域	用法建议
schema.org	社区词表（Google、Microsoft、Yahoo、Yandex 发起）	通用实体（产品、组织、事件、文章）	面向公开 Web 数据的轻量标注，可直接复用
FHIR	HL7	医疗健康	数据交换格式强，RDF 表示需借助映射；以官方发布的 RDF 定义为依据
GS1	GS1 组织	商品、条码、供应链（GTIN、GLN、EPCIS）	商品与物流标识优先复用，不要自造编码
IFC	buildingSMART（ISO 16739）	建筑与工程	与 BIM 工具交换时使用，配合其数据模型
DCAT	W3C	数据目录	描述数据集与服务，数据治理必备

采用标准的判断顺序：**先看行业是否已有事实标准 → 再看能否覆盖 80% 需求 → 最后才扩展自定义属性。**扩展时用独立命名空间，不要修改标准词表的语义。

ESB 与语义层的分工

两者不是替代关系：

- **ESB 管流程与路由**：消息可靠投递、协议转换、事务补偿；
- **语义层管含义**：字段与概念对齐、编码统一、跨源实体合并。

现实中常见组合是“ESB 传输 + 语义层解释”：消息体用 JSON-LD 或 RDF 承载，ESB 负责可靠送达，语义服务负责把消息映射进共享模型并校验。若已有 ESB，不必推倒重来，只需在消息体中补上语义标识。

实践建议与坑

1. **先对齐编码，再谈模型。**国家、货币、单位、状态码不统一，映射写得再好也没用。
2. **映射是长期资产。**为每个源系统维护映射文档与测试样例，随源变更同步更新。
3. **不要试图统一所有系统的内部结构。**共享模型只覆盖互操作必需的部分，各系统内部模型保持独立。这与 DDD 的限界上下文思想一致，见 领域驱动设计（DDD）是什么。
4. **实体主键要稳定。**源系统主键变更会让外部引用全部失效，必要时引入代理键。
5. **区分“同一事物”与“相似事物”。**用 `owl:sameAs` 前先确认，错并会顺着推理扩散。
6. **联邦查询设超时与降级**，并在监控里单独统计远端失败率。
7. **标准不是越新越好。**对接方支持哪个版本，往往比标准本身先进与否更重要。

延伸阅读：数据治理与主数据（共享术语基础）、存储选型（本地图谱与混合架构）、RDF 数据模型与序列化（JSON-LD 上下文与序列化）。

运营与生命周期

本体不是交付物，而是长期运行的资产。一个本体活下来的标志不是“文件还在”，而是**有人提需求、有人评审、有自动校验、有人在用。**本页讲运营机制：角色、流程、指标、失败模式，以及如何用最小的组织成本起步。

为什么运营比建模更难

建模是一次性投入，运营是持续成本。本体失活通常经历同样四步：

1. 上线时模型完整、热度高；
2. 源系统改字段，映射滞后，数据开始缺；
3. 消费方发现结果不准，逐渐弃用；
4. 无人维护，模型与数据彻底脱节，成为“历史文档”。

阻断这条滑坡只有两个手段：**把校验自动化、把维护责任落到人。**任何依赖“大家自觉”的治理方案都撑不过半年。

角色与职责

角色	职责	时间投入
本体负责人（Owner）	终审模型变更、对语义一致性负责	每周固定小时数，不能是挂名
领域专家	提出与确认概念、判定业务语义争议	评审会 + 按需答疑
语义/知识工程师	设计模型、写映射与约束、跑校验	主要维护者
数据工程师	源系统接入、映射实现、数据质量	按项目阶段投入
消费方代表	提出能力问题、验收、反馈错例	每迭代一次

关键点：**Owner 必须有决策权。**如果每次语义争议都要上升到委员会，模型就会停滞。小团队里可以由架构师兼任，但要明确写入职责。

提案—评审—发布流程

轻量但明确，四步即可：

1. **提案（Issue）**：谁提的、什么能力问题驱动、影响哪些类/属性/约束、是否破坏兼容；
2. **评审（Review）**：语义工程师审形式正确性（一致性、约束合理性），领域专家审业务正确性，消费方审影响面；
3. **实现与测试**：改模型、改映射、补测试（CQ 查询、SHACL 样例、单元测试），在 CI 中验证；
4. **发布（Release）**：打版本号、写变更说明、更新消费方、必要时并行运行一段时间。

变更分级很重要：

- **补丁级**：加注释、加同义词、修正拼写——直接合并；

- **次版本**：加类、加可选属性、加约束（可能暴露历史数据问题）——需要评审与数据影响评估；
- **主版本**：删类、改语义、改 IRI——需要迁移方案与消费方确认。

版本与兼容策略详见 版本与演进。

用 Git 与 CI 管理本体

本体文件（Turtle、SHACL）当代码管理，是最成熟的实践：

text

```
ontology-repo/
├── ontologies/
│   ├── core.ttl              # 核心概念
│   ├── product.ttl           # 业务模块
│   └── vocab/                # 复用的外部词表（固定版本）
├── shapes/
│   ├── product-shapes.ttl
│   └── order-shapes.ttl
├── queries/                  # 能力问题对应的 SPARQL，作为验收测试
│   └── cq-001-products-by-category.rq
├── mappings/                 # R2RML / CONSTRUCT 映射
├── tests/                    # 样例数据与期望结果
└── CHANGELOG.md
```

CI 里至少跑四件事：

- **语法解析**：所有 Turtle 文件可解析（`riot --validate` 或对应库的解析器）；
- **一致性检查**：推理器加载全量模式，确认无不可满足类、无矛盾（ELK、HermiT 等）；
- **CQ 回归**：跑 `queries/` 下的查询，结果与期望集比对；
- **SHACL 回归**：样例数据必须通过、违规样例必须被抓到。

再加一条：**外部依赖锁版本。**复用 schema.org、FOAF、DCAT 等外部词表时，把文件快照进仓库或固定下载版本，避免上游变更导致构建漂移。

协作编辑上，WebProtégé 之类的浏览器工具适合领域专家参与评审；正式版本仍以 Git 仓库为准。Protégé 本地编辑容易出现格式冲突，提交前统一格式（序列化排序）能减少无意义 diff。

监控指标

运营指标要能回答“本体是否还活着”：

维度	指标	采集方式
模型质量	CQ 通过率、推理一致性状态、SHACL 违规数	CI + 定时任务
数据同步	映射滞后时长、实体覆盖率、重复实体数	加载监控
使用情况	查询次数、活跃消费方数量、API 调用来源	端点日志
变更节奏	提案数、评审时长、平均合并周期	仓库与工单系统
服务性能	查询 P95 延迟、超时率、推理任务耗时	端点监控

最少要盯住三项：**CQ 通过率、映射滞后、活跃消费方数量**。三者中任何一项持续下降，说明本体正在失去价值。

数据侧质量指标的定义见 数据治理与主数据，检索场景的效果指标见 语义检索与 RAG。

常见失败模式

1. **一次性项目**：立项时叫“知识图谱建设”，验收后无人接手。对策：验收标准里包含运营机制与预算。
2. **无人维护**：没有 Owner，或 Owner 只是挂名。对策：明确写入岗位职责与考核。
3. **建模与数据脱节**：模型改了，映射没改；或数据变了，模型不更新。对策：映射与模型同仓库、同 CI。
4. **过度设计**：追求全量公理与完美分类，交付周期拉长，消费方失去耐心。对策：能力问题驱动，增量发布。
5. **只对内不对外**：模型不对外文档化，新成员无法上手。对策：每个类与属性必须有 `rdfs:label` 与 `skos:definition`，术语表可检索。
6. **治理流程过重**：每个小改动都要开会评审，维护者放弃提交。对策：变更分级，按风险区分流程。
7. **指标只看数量**：统计“有多少类和属性”，不看是否被使用。数量增长不等于价值增长。

从小做起的路线

如果组织还没有本体运营经验，按下面顺序推进，每一步都能独立交付：

1. **第 1 个月**：选定一个业务域，建立 Git 仓库与目录结构，写 10 条 CQ，建模 20–40 个核心类；
2. **第 2 个月**：接入 1 个源系统，实现映射；配置 CI 跑解析、一致性、CQ 回归；
3. **第 3 个月**：上线第一个消费方（查询接口或校验流水线），建立 Owner 与双周评审节奏；
4. **第 4–6 个月**：补 SHACL 质量规则、扩展第二个源系统，开始记录运营指标；
5. **半年后**：复盘 CQ 通过率与使用量，决定是扩展域还是加深应用。

这条路线避免了两个极端：一开始就追求平台化（必然烂尾），或者只做一次性交付（必然失活）。

与其他章节的关系

- 运营依赖建模质量，见 质量评估与反模式；
- 版本与兼容性细节，见 版本与演进；
- 组织推广与阶段规划，见 落地路线图；
- 模块划分与复用，见 模块化与复用。

落地路线图

本页面向准备把本体引入组织的人：如何判断从哪一阶段开始、90 天能做什么、需要哪些角色、怎么选试点和验收、如何避免“立项即高潮、验收即终止”。结论先给：**从一个受限业务域、一个明确消费方、一条可自动校验的链路开始，用 90 天做出可验收的最小闭环，再谈平台化。**

成熟度模型

组织在本体上的能力通常经过四个阶段。先定位自己，再决定下一步，不要跳级。

阶段	特征	关键产出	退出条件
探索	有零散尝试，概念验证为主	术语表、小样例本体、技术选型报告	找到一个真实痛点与可行技术路线
试点	单域、单消费方、有明确验收指标	可运行的本体 + 映射 + 校验 + 一个应用	消费方实际使用并给出量化收益
规模化	多域并行，有治理流程与 CI	模块化本体库、共享词表、角色分工	变更可控、多团队复用同一模型
平台化	语义层作为公共基础设施	本体注册表、语义 API 网关、SLA	有稳定运营指标与预算保障

常见错误是“探索阶段直接按平台化设计”：还没人用，就先买商业图数据库、设计多租户语义网关。结果预算花完，产出物没人接。**每一阶段的投入应匹配上一阶段验证过的需求。**

90 天启动计划

以周为单位，可执行、可交付：

第 1–2 周：定义问题与范围

- 访谈 3–5 位业务/技术相关方，收集“因为语义不一致而返工”的具体案例；
- 写出 10 条能力问题（CQ），按价值与可行性排序，选 5 条作为本期验收，写法见 能力问题（CQ）；
- 确定范围：1 个业务域、不超过 3 个源系统、明确不做什么。

第 3–4 周：复用调研与模型初稿

- 盘点可复用标准（schema.org、DCAT、行业标准），列取舍理由；
- 建仓库、定命名空间与 IRI 规则，写第一批类与属性，配 `rdfs:label`、`skos:definition`；
- 搭最小技术栈：三元组库（或轻量 oxigraph/RDFLib 起步）+ Git + CI 骨架，见 技术栈与平台总览。

第 5–7 周：数据接入与映射

- 从优先级最高的源系统开始，写映射（R2RML 或程序映射），导入样例数据；
- 用 CQ 查询验证数据可用性，暴露缺失字段与口径问题；
- 记录每条问题的处理方式，形成“语义决策日志”。

第 8–10 周：校验与消费方对接

- 写 SHACL 形状，覆盖必填、类型、范围、关键跨实体规则，见 SHACL 形状约束；
- 上线第一个消费接口：查询 API、检索服务或校验流水线，择一，不要全做；
- 与消费方联调，收集错例。

第 11–13 周：验收与复盘

- 对照 5 条 CQ 验收；统计使用量、数据质量、查询延迟；
- 写复盘：哪些决策被证明错误、哪些标准复用有效、下一阶段优先级；
- 决定继续、调整或停止。**允许停止也是成功**，及时止损比硬撑更有价值。

团队能力与角色

最小可运行配置是三个人兼职：

- **语义/知识工程师**：建模与形式化、约束设计、CI 配置；
- **数据工程师**：源系统接入、映射实现、数据质量；
- **领域专家**：语义判定与评审，按需参与。

再加一名有决策权的 Owner 负责终审。规模扩大后才需要专职的语义平台团队。

能力建设建议：

- 先用一个真实小项目把“建模 → 映射 → 校验 → 查询”走通，比看教程有效；
- 团队至少有一人熟悉 OWL 与 SHACL 的语义差异（开放世界 vs 封闭世界），这是最常见的知识缺口；
- 不要指望外部顾问交付后就撤场，必须做知识转移，把 CI 与仓库留在自己手里。

工具与预算

工具分四类，按需配置：

类别	起步选择	何时升级
建模编辑	Protégé / WebProtégé	多团队协作时引入注册表与评审流
存储与查询	单机三元组库（如 Apache Jena TDB2 + Fuseki、Oxigraph）	数据量与并发上来后评估商业库
校验与推理	pyshacl、Jena SHACL、ELK/HermiT	需要增量推理或规则引擎时评估商业产品
工程协作	Git + CI	规模化后加本体注册表与发布流程

预算上要区分三类成本：**软件许可**（开源可零许可费，但需要人力运维）、**人力**（建模与映射通常是主要成本）、**集成与培训**（容易被低估）。商业产品在集群、权限、推理性能、技术支持上有优势，是否值得取决于数据规模与运维能力，不要用“开源一定便宜”做决定——运维成本常常超过许可费。

与 DDD 团队、数据团队协作

- **与 DDD 团队**：分工是“业务边界与行为 vs 共享语义与数据互操作”。本体团队不重写领域模型，而是把跨上下文共享的部分抽出来。协作方式：领域模型评审时同步语义决策；防腐层可作为映射入口。参见 领域驱动设计（DDD）是什么 与 分层与六边形架构。

- **与数据团队**：数据团队提供源系统与质量基线，本体团队提供术语与映射。最容易冲突的是“谁是主数据权威源”，这件事必须在启动前书面明确。
- **与业务方**：业务方提能力问题、判定语义、验收结果。不要让他们参与 IRI 命名这类技术细节，会消耗耐心。

试点选择与验收

选试点的五个标准，满足四条以上再启动：

1. 痛点有基线数据（返工工时、对账差异、查询耗时）；
2. 范围能在 90 天内收敛；
3. 有明确的消费方与使用流程；
4. 源数据可得且质量可接受；
5. 有业务方愿意投入评审时间。

验收标准写成可核对的条目，而不是“完成知识图谱建设”这种描述：

- 5 条 CQ 全部可查询且结果正确；
- 核心实体覆盖率、关键属性填充率达到约定值；
- SHACL 规则全部自动化运行，违规可定位；
- 消费方在生产或准生产环境实际使用，并给出满意度反馈；
- 运营机制就位：Owner、评审节奏、CI、监控指标。

推广与组织保障

试点成功后，推广失败仍然是常态。三个保障动作：

- **复用优先**：第二个域优先复用已有模块与词表，避免团队各自建本体。模块化做法见 模块化与复用；
- **流程固化**：变更分级、评审规则、发布说明写进团队规范，见 运营与生命周期；
- **价值可视化**：定期展示使用量、质量提升与节省的工时，让投入可见。没有可见价值的项目，预算会最先被砍。

常见问题

必须买商业图数据库吗？ 不必。试点阶段单机开源方案足够，规模与并发上来再评估。

必须用 OWL 吗？ 从 SKOS 词表起步完全可以，需要推理时再升级。

没有领域专家怎么办？ 那就先不做语义严谨度要求高的场景，从检索/术语治理起步，等专家可用再推进。

和现有数据中台冲突吗？ 不冲突。中台管数据加工与服务，本体管语义定义与互操作，通过 ID 与元数据对接。

怎么衡量成功？ 用消费方使用量、CQ 通过率、质量违规下降、返工工时减少四项组合，不要只看模型规模。

启动 Checklist

检查项	验收证据
已定位成熟度阶段，并明确本阶段目标	一页纸的阶段说明与目标
已有 10 条 CQ，选定 5 条作为验收	CQ 清单与查询草稿
范围限定：业务域、源系统、时间窗口	范围声明，含“不做什么”
复用标准已盘点并给出取舍理由	标准对照表
仓库、命名空间、IRI 规则已定	仓库 README 与命名规范
映射方案与责任人已确认	映射清单与负责人
SHACL 校验已进入 CI	CI 流水线与最近一次报告
消费方已确认并参与联调	联调记录与反馈
Owner 与评审节奏已书面确认	职责说明与例会安排
验收指标有基线值与目标值	指标基线表

这十项里，最容易漏的是最后一项与“消费方参与联调”。缺少前者无法证明价值，缺少后者交付即闲置。

端到端案例：电商商品与订单本体

前面几页分别讲了元素与标准，这一页把它们串起来：从能力问题出发，建一个覆盖商品、客户、订单、订单行、地址的最小可用本体，再用样例数据、SPARQL 和 SHACL 证明它真的能用。案例刻意保持小，目的是让整条链路可读、可复现。

范围与能力问题

范围限于“商品展示与订单查询”，不覆盖库存、支付、优惠券和物流轨迹。先写 6 条能力问题（CQ）：

编号	能力问题	验收方式
CQ1	某客户还有哪些未取消的订单？金额多少？	SPARQL
CQ2	电子类目下有哪些商品（含子类目）？	SPARQL 属性路径
CQ3	已支付超过 24 小时仍未发货的订单有哪些？	SPARQL + 时间比较
CQ4	哪些订单行的成交价与商品标价不一致？	SPARQL 对账
CQ5	客服视图里，一个订单的客户、地址、商品明细是什么？	SPARQL
CQ6	录入的订单数据是否完整、格式是否合规？	SHACL

CQ 的价值在于“可判定”：每条都能落到查询或校验上，模型是否有用一目了然。CQ 也不是一次性文档，每轮迭代先复核 CQ 是否仍成立，再决定加不加概念。写法要点见能力问题（CQ）。

复用取舍：schema.org 与 GS1

电商概念与 schema.org 高度重合（`schema:Product`、`schema:Offer`、`schema:Order`、`schema:OrderItem`），商品条码领域有 GS1 的 GTIN 标准。本案例的选择是：

- **核心业务类自建**：订单、订单行、客户、地址需要承载内部编码、状态机与校验规则，自建才能控制语义。
- **对齐而非 import**：用 `owl:equivalentClass` 或 `skos:exactMatch` 把 `shop:Product` 对齐到 `schema:Product`，供外部检索与交换使用（下方 TBox 给出了 `skos:exactMatch` 的写法）。
- **GS1 只借用标识**：`shop:gtin` 作为数据属性存储 GTIN，不引入 GS1 全套词表。

不直接 `owl:imports` schema.org 的原因很实际：它面向搜索引擎，类多而浅，导入后既不能替你表达业务规则，还会把大量无关的 domain/range 带进推理。取舍原则见模块化与复用。

核心类与关系

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix schema: <https://schema.org/> .
```

```
<https://example.com/shop/ontology> a owl:Ontology ;
  rdfs:label "电商商品与订单本体"@zh ;
  owl:versionInfo "1.0.0" .

shop:Product a owl:Class ;
  rdfs:label "商品"@zh ;
  skos:exactMatch schema:Product .
shop:ElectronicsProduct a owl:Class ;
  rdfs:subClassOf shop:Product ;
  rdfs:label "电子商品"@zh .
shop:Customer a owl:Class .
shop:Order a owl:Class .
shop:OrderLine a owl:Class ;
  rdfs:comment "订单与商品之间的限定关系，承载数量与成交价。"@zh .
shop:Address a owl:Class .
shop:OrderStatus a owl:Class .

shop:placedBy a owl:ObjectProperty ;
  rdfs:domain shop:Order ;
  rdfs:range shop:Customer ;
  owl:inverseOf shop:placedOrder .

shop:hasLine a owl:ObjectProperty ;
  rdfs:domain shop:Order ;
  rdfs:range shop:OrderLine ;
  owl:inverseOf shop:lineOfOrder .

shop:lineProduct a owl:ObjectProperty ;
  rdfs:domain shop:OrderLine ;
  rdfs:range shop:Product .

shop:shipsTo a owl:ObjectProperty ;
  rdfs:domain shop:Order ;
  rdfs:range shop:Address .

shop:orderStatus a owl:ObjectProperty ;
  rdfs:domain shop:Order ;
  rdfs:range shop:OrderStatus .

shop:orderNumber a owl:DatatypeProperty ;
  rdfs:domain shop:Order ;
  rdfs:range xsd:string .
shop:placedAt a owl:DatatypeProperty ;
  rdfs:domain shop:Order ;
  rdfs:range xsd:dateTime .
shop:paidAt a owl:DatatypeProperty ;
  rdfs:domain shop:Order ;
  rdfs:range xsd:dateTime .
shop:totalAmount a owl:DatatypeProperty ;
  rdfs:domain shop:Order ;
  rdfs:range xsd:decimal .
shop:currency a owl:DatatypeProperty ;
  rdfs:domain shop:Order ;
  rdfs:range xsd:string .
shop:customerId a owl:DatatypeProperty ;
  rdfs:domain shop:Customer ;
```

```
  rdfs:range xsd:string .
shop:customerName a owl:DatatypeProperty ;
  rdfs:domain shop:Customer ;
  rdfs:range xsd:string .
shop:sku a owl:DatatypeProperty ;
  rdfs:domain shop:Product ;
  rdfs:range xsd:string .
shop:productName a owl:DatatypeProperty ;
  rdfs:domain shop:Product ;
  rdfs:range xsd:string .
shop:listPrice a owl:DatatypeProperty ;
  rdfs:domain shop:Product ;
  rdfs:range xsd:decimal .
shop:quantity a owl:DatatypeProperty ;
  rdfs:domain shop:OrderLine ;
  rdfs:range xsd:integer .
shop:unitPrice a owl:DatatypeProperty ;
  rdfs:domain shop:OrderLine ;
  rdfs:range xsd:decimal .
shop:lineAmount a owl:DatatypeProperty ;
  rdfs:domain shop:OrderLine ;
  rdfs:range xsd:decimal .
shop:addressLine a owl:DatatypeProperty ;
  rdfs:domain shop:Address ;
  rdfs:range xsd:string .
shop:city a owl:DatatypeProperty ;
  rdfs:domain shop:Address ;
  rdfs:range xsd:string .
shop:postalCode a owl:DatatypeProperty ;
  rdfs:domain shop:Address ;
  rdfs:range xsd:string .
```

订单行是本案例的**限定关系**：订单与商品是多对多关系，还带数量与成交价，因此必须具体化为 `shop:OrderLine`，而不是在订单上挂一个 `shop:buysProduct`。这个套路可复用到“合同-商品-单价”“项目-人员-工时”等场景，见对象属性与关系。

关键公理

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

# 订单至少有一个订单行（存在性，不是必填校验）
shop:Order rdfs:subClassOf [
  a owl:Restriction ;
  owl:onProperty shop:hasLine ;
  owl:someValuesFrom shop:OrderLine
] .

# 四种状态是 OrderStatus 的个体
shop:StatusDraft     a shop:OrderStatus .
shop:StatusPaid      a shop:OrderStatus .
shop:StatusShipped   a shop:OrderStatus .
```

```
shop:StatusCancelled a shop:OrderStatus .

# 一个订单至多有一个状态
shop:orderStatus a owl:FunctionalProperty .

# 四种状态互不相同（注意：互斥的“值”用 AllDifferent，
# AllDisjointClasses 用于互斥的“类”）
[] a owl:AllDifferent ;
  owl:members ( shop:StatusDraft shop:StatusPaid
                shop:StatusShipped shop:StatusCancelled ) .

# 业务唯一标识，用于推出个体同一性
shop:Customer owl:hasKey ( shop:customerId ) .
shop:Product owl:hasKey ( shop:sku ) .
```

注意两条边界：`someValuesFrom` 在开放世界下不会因为“没有行数据”而报错；`owl:hasKey` 只推出 `sameAs`，不拒绝重复。状态用“函数属性 + `AllDifferent`”表达：前者限制至多一个，后者保证四个状态个体互异，合起来才能对“一单两个状态”判不一致。真正的必填与唯一校验交给 SHACL 和数据库，见公理。

样例数据

turtle

```
@prefix data: <https://example.com/shop/data/> .
@prefix shop: <https://example.com/shop/ontology#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

data:product-1 a shop:ElectronicsProduct ;
  shop:sku "SKU-KB-001" ;
  shop:productName "无线键盘"@zh ;
  shop:listPrice "199.00"^^xsd:decimal .

data:product-2 a shop:Product ;
  shop:sku "SKU-BK-002" ;
  shop:productName "本体建模实践"@zh ;
  shop:listPrice "445.00"^^xsd:decimal .

data:customer-1001 a shop:Customer ;
  shop:customerId "C-1001" ;
  shop:customerName "张三" .

data:address-1 a shop:Address ;
  shop:addressLine "示例路 100 号" ;
  shop:city "上海" ;
  shop:postalCode "200000" .

data:order-ORD20250301 a shop:Order ;
  shop:orderNumber "ORD-20250301" ;
  shop:placedAt "2025-03-01T10:15:00+08:00"^^xsd:dateTime ;
  shop:paidAt "2025-03-01T10:20:00+08:00"^^xsd:dateTime ;
  shop:placedBy data:customer-1001 ;
  shop:shipsTo data:address-1 ;
  shop:orderStatus shop:StatusPaid ;
  shop:totalAmount "1288.00"^^xsd:decimal ;
  shop:currency "CNY" ;
```

```
  shop:hasLine data:line-1 , data:line-2 .

data:line-1 a shop:OrderLine ;
  shop:lineOfOrder data:order-ORD20250301 ;
  shop:lineProduct data:product-1 ;
  shop:quantity 2 ;
  shop:unitPrice "199.00"^^xsd:decimal ;
  shop:lineAmount "398.00"^^xsd:decimal .

data:line-2 a shop:OrderLine ;
  shop:lineOfOrder data:order-ORD20250301 ;
  shop:lineProduct data:product-2 ;
  shop:quantity 2 ;
  shop:unitPrice "445.00"^^xsd:decimal ;
  shop:lineAmount "890.00"^^xsd:decimal .
```

数据里刻意保留了 `shop:unitPrice` 与 `shop:listPrice` 两个价格，用来支撑 CQ4 的对账。

SHACL 校验

turtle

```
@prefix shop: <https://example.com/shop/ontology#> .
@prefix sh:   <http://www.w3.org/ns/shacl#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .

shop:OrderShape a sh:NodeShape ;
  sh:targetClass shop:Order ;
  sh:property [
    sh:path shop:orderNumber ;
    sh:minCount 1 ; sh:maxCount 1 ;
    sh:pattern "^ORD-[0-9]{8}$" ;
    sh:message "订单号缺失或格式错误" ;
  ] ;
  sh:property [
    sh:path shop:placedBy ;
    sh:class shop:Customer ;
    sh:minCount 1 ; sh:maxCount 1 ;
  ] ;
  sh:property [
    sh:path shop:totalAmount ;
    sh:datatype xsd:decimal ;
    sh:minInclusive 0 ;
  ] ;
  sh:property [
    sh:path shop:currency ;
    sh:pattern "^[A-Z]{3}$" ;
    sh:message "币种应为三位大写 ISO 代码" ;
  ] ;
  sh:property [
    sh:path shop:hasLine ;
    sh:minCount 1 ;
    sh:node shop:OrderLineShape ;
  ] .

shop:OrderLineShape a sh:NodeShape ;
```

```
  sh:targetClass shop:OrderLine ;
  sh:property [
    sh:path shop:quantity ;
    sh:datatype xsd:integer ;
    sh:minInclusive 1 ;
  ] ;
  sh:property [
    sh:path shop:lineProduct ;
    sh:minCount 1 ;
    sh:class shop:Product ;
  ] .
```

这份形状把 CQ6 落成了可执行的检查：必填、格式、引用类型、数值下界都在封闭世界语义下判定，缺数据就报错。语法细节见SHACL 形状约束，规则治理见业务规则与校验。

CQ 对应的 SPARQL

sparql

```
PREFIX shop: <https://example.com/shop/ontology#>
PREFIX data: <https://example.com/shop/data/>

# CQ1：客户 C-1001 未取消的订单
SELECT ?order ?number ?total WHERE {
  ?order a shop:Order ;
         shop:orderNumber ?number ;
         shop:placedBy ?customer ;
         shop:orderStatus ?status ;
         shop:totalAmount ?total .
  ?customer shop:customerId "C-1001" .
  FILTER (?status != shop:StatusCancelled)
}
ORDER BY DESC(?total)
```

sparql

```
PREFIX shop: <https://example.com/shop/ontology#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

# CQ2：电子类目（含子类）的商品
SELECT ?sku ?name ?price WHERE {
  ?product rdf:type/rdfs:subClassOf* shop:ElectronicsProduct ;
           shop:sku ?sku ;
           shop:productName ?name ;
           shop:listPrice ?price .
}
```

sparql

```
PREFIX shop: <https://example.com/shop/ontology#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>

# CQ3：支付超过 24 小时仍未发货
SELECT ?order ?number ?paidAt WHERE {
```

```
  ?order a shop:Order ;
         shop:orderNumber ?number ;
         shop:orderStatus shop:StatusPaid ;
         shop:paidAt ?paidAt .
  FILTER (?paidAt < (NOW() - "PT24H"^^xsd:dayTimeDuration))
}
```

sparql

```
PREFIX shop: <https://example.com/shop/ontology#>

# CQ4：成交价与标价不一致的订单行（对账）
SELECT ?number ?sku ?sold ?listed WHERE {
  ?order shop:orderNumber ?number ;
         shop:hasLine ?line .
  ?line shop:lineProduct ?product ;
        shop:unitPrice ?sold .
  ?product shop:sku ?sku ;
           shop:listPrice ?listed .
  FILTER (?sold != ?listed)
}
```

CQ5 的客服视图只是把 `placedBy`、`shipsTo`、`hasLine/lineProduct` 组合起来，不再重复。属性路径与聚合的更多用法见SPARQL 查询与更新。

在系统里怎么用

选择两个落地场景，覆盖“写入”和“读取”两端：

写入校验：订单数据进入图之前跑 `OrderShape`，违规订单进入修复队列。实现上可以用服务层封装校验，与聚合内不变量形成双层防护——聚合负责“总额等于行金额之和”，SHACL 负责“字段完整、格式与引用合法”。

日终对账：每天用 CQ4 扫描成交价与标价不一致的订单行，输出差异清单给运营。它体现本体的核心价值：跨系统数据用同一套语义对齐后，对账不再靠字段名猜含义。

若继续扩展，下一步通常是商品检索（把类目与属性用于筛选和召回）和个性化推荐（依赖客户历史订单），分别见语义检索与 RAG与知识图谱构建。

与 DDD 模型的对齐

把本体映射到代码时，边界要清楚：`shop:Order` 是聚合根，`shop:OrderLine` 是聚合内实体，`shop:Address` 与金额是值对象；聚合内的不变量（总额等于行金额之和、状态迁移合法）仍在代码里维护，本体只负责共享语义、查询与校验。`shop:OrderStatus` 的四个个体对应代码里的状态枚举，SHACL 校验对应应用层的 DTO 校验与发布检查。这样的映射让两边各司其职，不需要把推理器塞进聚合，见实现本体总览。

迭代经验

1. **先跑通一条 CQ 再加概念**：本案例第一版没有地址和币种，CQ5 出现后才补，避免了提前设计。
2. **订单行必须一开始就具体化**：把数量、成交价挂在订单或商品上，后期几乎没有无痛迁移路径。

3. **样例数据是测试用例**：每个 CQ 配一条样例数据，比评审类图有效得多。
4. **公理克制**：只保留状态至多一个、状态个体互异、订单行存在性与 key，其余交给 SHACL，推理规模可控。
5. **命名保持稳定**：`shop:` 命名空间与 local name 一旦发布，只增不改，废弃用 `owl:deprecated` 标记。
6. **把 CQ 查询与形状一起纳入版本库**：本体、形状、验收查询同批评审与回归，避免模型改了查询悄悄失效。